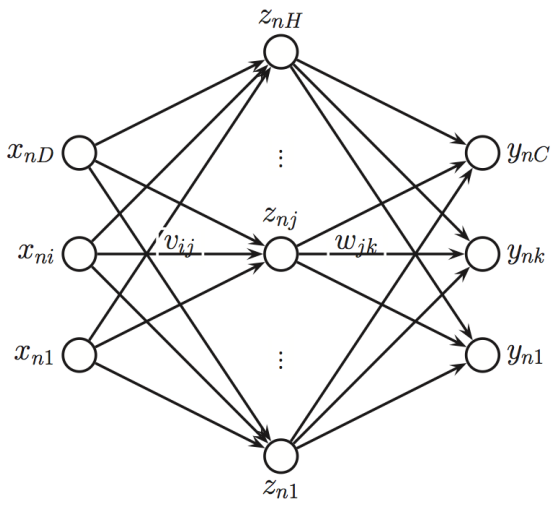
Machine Learning



Lecture 3: representation learning & Neural networks

Iasonas Kokkinos

Imperial College London

i.kokkinos@ic.ac.uk

Meeting hours and course feedback

Monday, 6–7 pm

Office 450

Questions, clarifications and feedback

forms.cloud.microsoft/e/De8UK4392r

Week 1-2: ML foundations

Loss functions

Squared L2 loss for regression, cross-entropy for classification

Regularisation

L2 shrinks coefficients. L1 encourages sparsity. Probabilistic interpretation.

Minimizers

Closed form, second-order methods, gradient descent and SGD

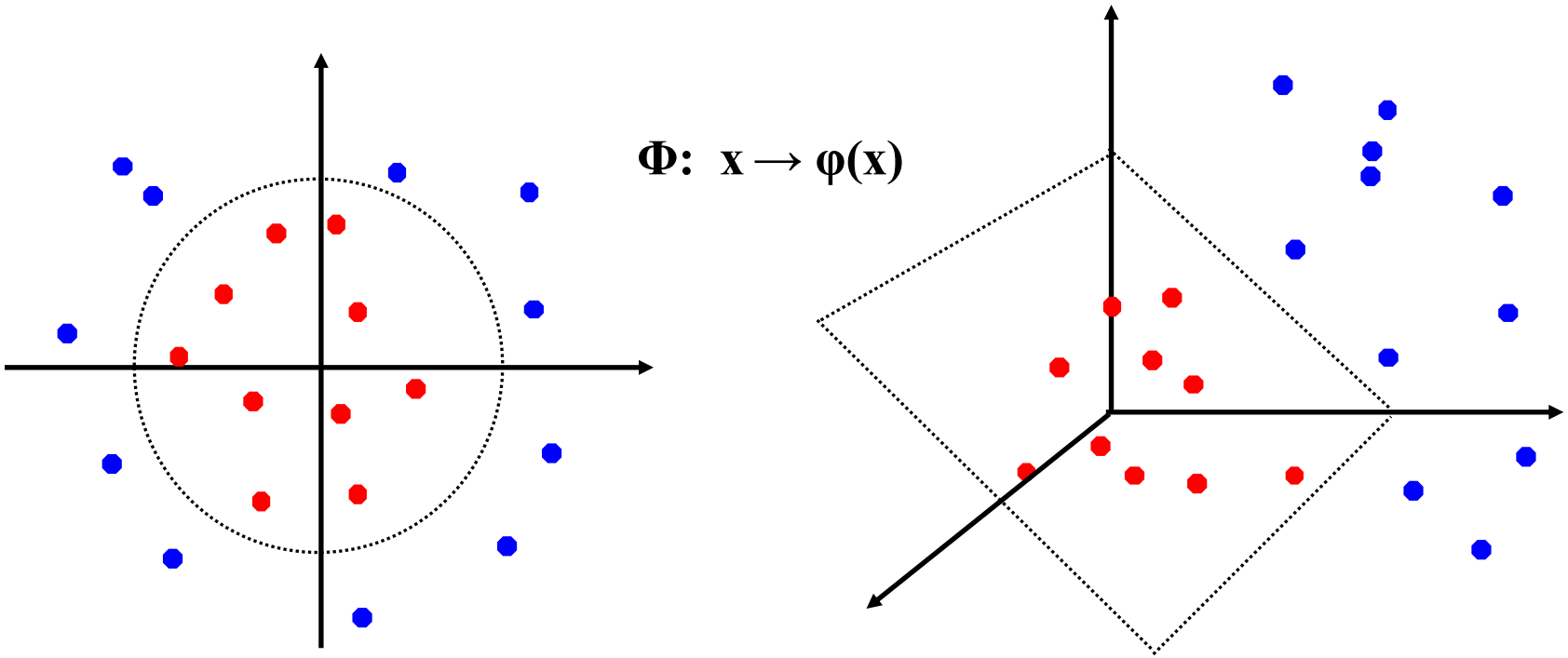
ML fundamentals: rooted in probability, statistics and optimisation.

Historical context: 1763–1970

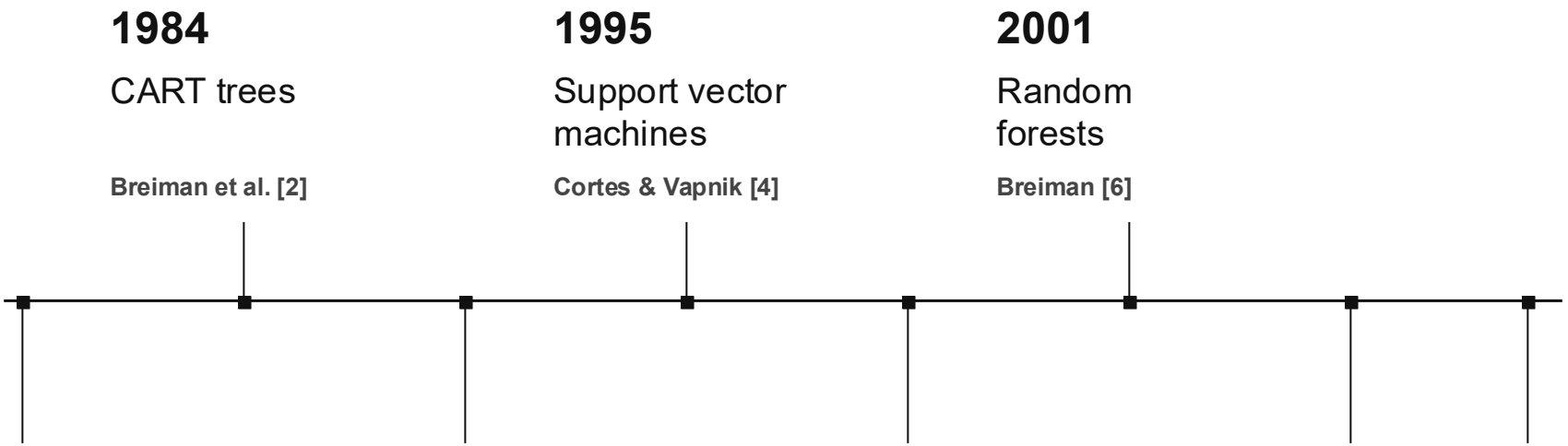
Hastie, Tibshirani & Friedman (2009), The Elements of Statistical Learning.

Beyond linear boundaries

- Weeks 1-2: “More features” & regularization

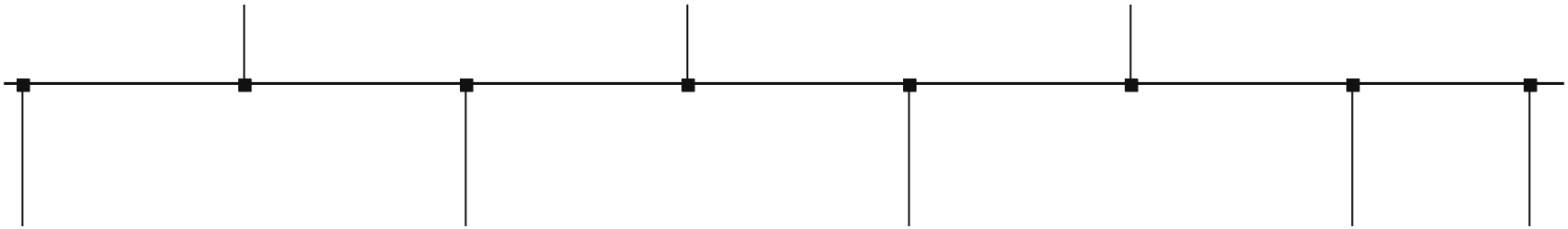


ML milestones



Selected publications. [n] refers to the full references at the end.

ML milestones: good/smart/fast features



1984
CART trees

1995
Support vector machines

2001
Random forests

1958
Perceptron

1986
Multilayer perceptrons

1998
Convolutional networks

2012
AlexNet

2017
Transformer

Decision trees

Decision trees

Bagging and random forests

Kernels and SVMs

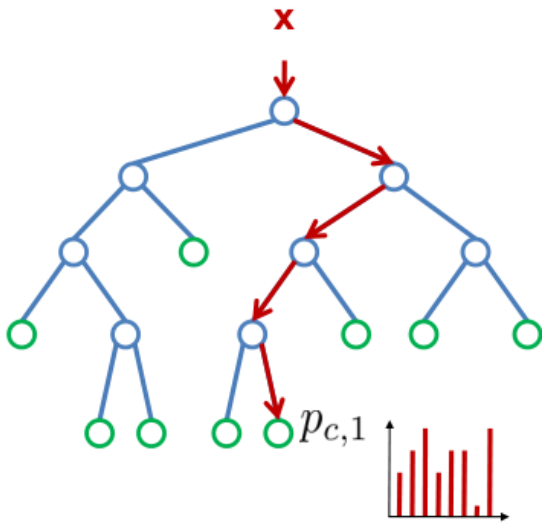
Neural networks

Expressiveness and representations

Backpropagation

1984 CART trees

Breiman, Friedman, Olshen and Stone

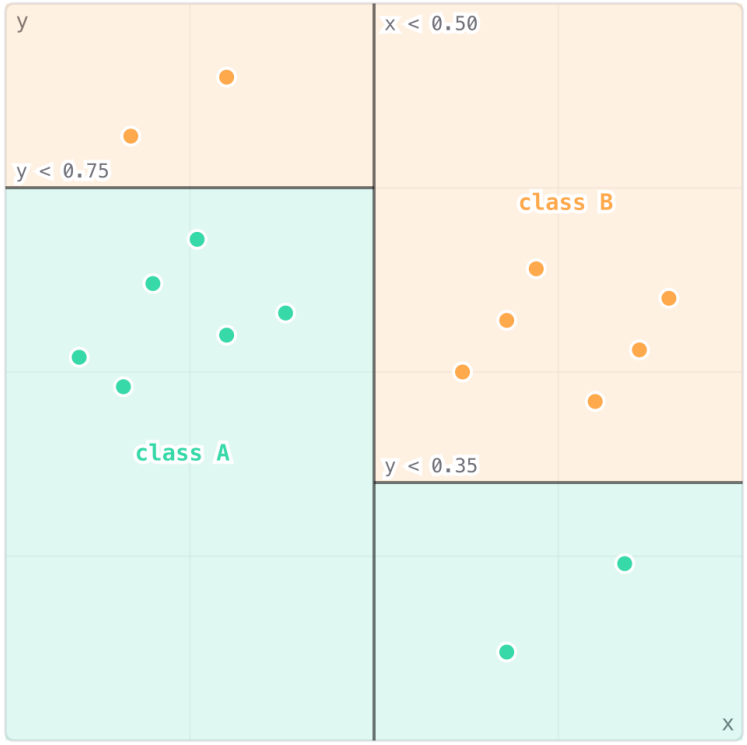


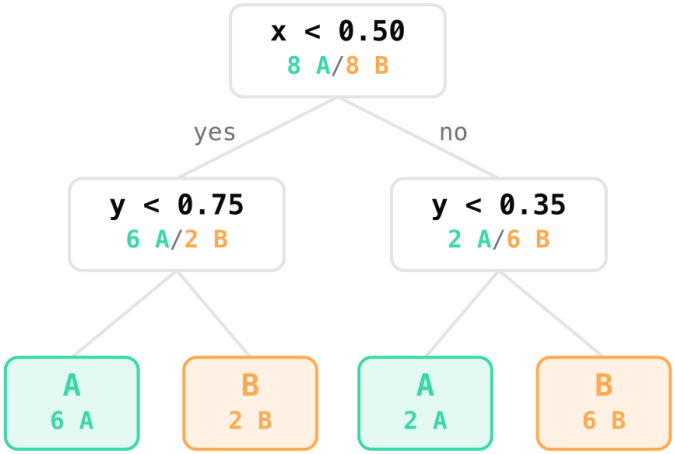
Reading: Decision trees

K. Murphy (2022), §18.1 Classification and regression trees (CART)

A path acts as a learned feature

AlgoMaster: live decision-tree animation: https://algomaster.io/animations/ai-ml/ml-decision-tree



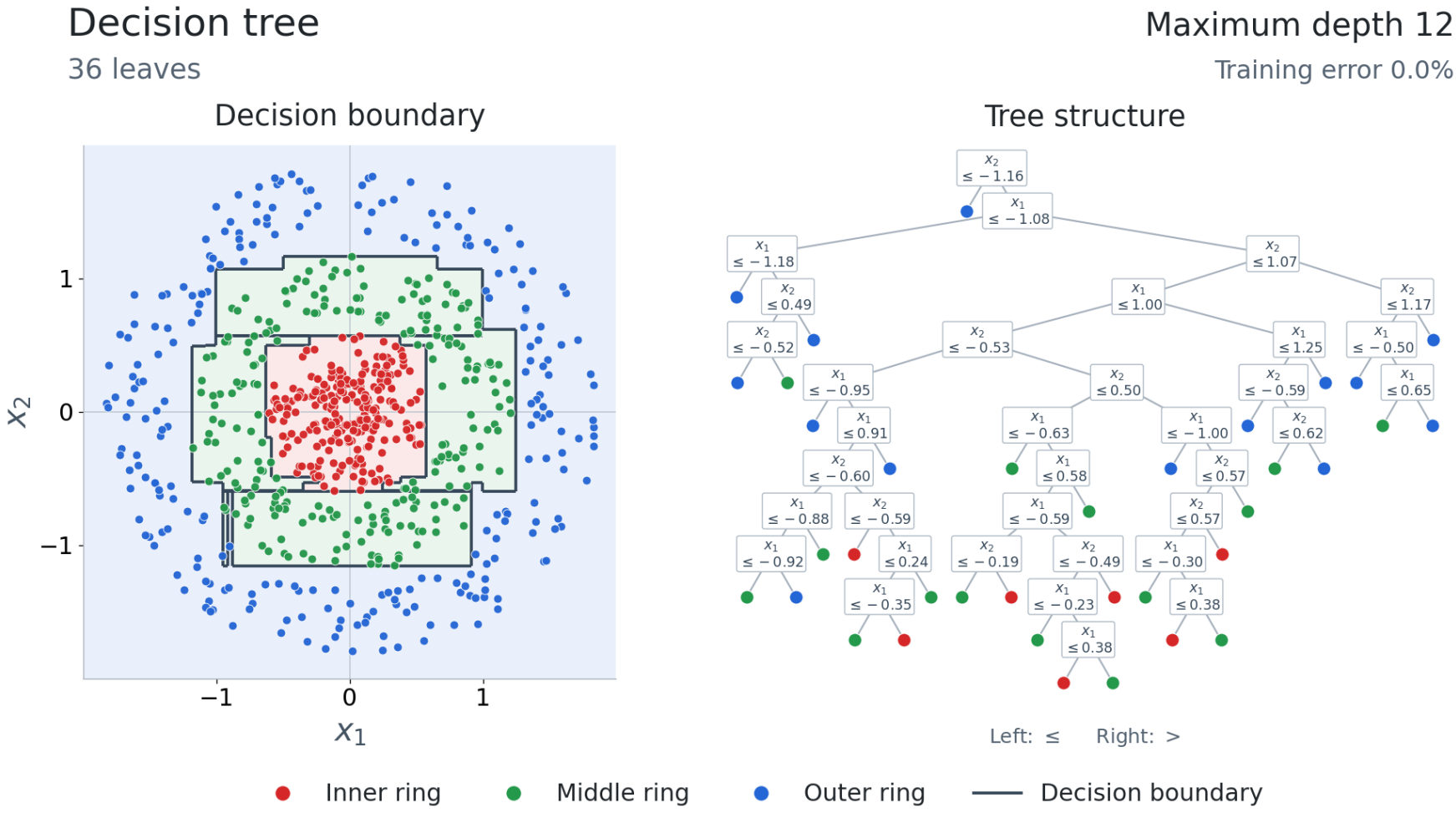


$$\phi_{a,b}(x, y) = \mathbf{1}[x < a,\ y < b]$$

$$\phi_{0.5,0.75}(x, y) = \mathbf{1}[x < 0.5,\ y < 0.75]$$

A path feature is 1 exactly when every test on that path is satisfied.

Tree depth and overfitting

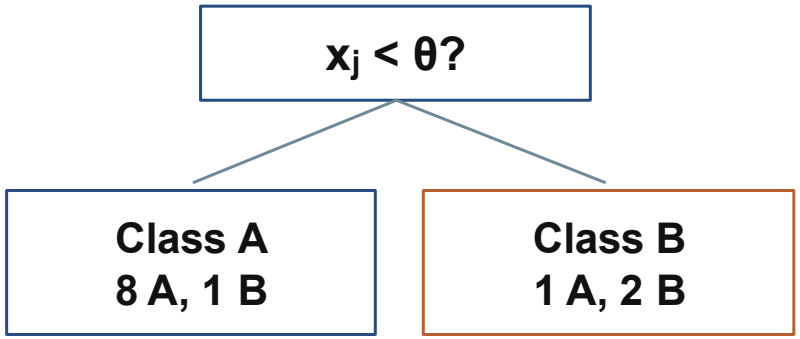


Beyond a useful depth, fitting training detail can hurt unseen performance.

Pruning a decision tree

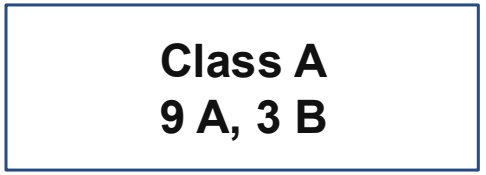
Grow the tree, then remove splits that do not help on validation data.

Before pruning



Validation errors: 4 / 20

After pruning



Validation errors: 3 / 20

1. Visit a node whose children are leaves.
2. Replace its split with a leaf fitted on its training cases.
3. Keep the change if validation performance improves.
4. Work upwards and repeat.

Training error and validation error

Tree construction	Training	Validation	Diagnosis
Early stopping	0.2975	0.4588	Underfit
Exhaustive growth	0.0142	0.3763	Overfit
Pruning	0.1025	0.1997	Best of these

Decision trees: strengths and limits

What makes trees useful

Greedy training is fast. Prediction follows a short path of tests.
Tests can use different feature types, such as colour, size or shape.

How we control complexity

Choose depth, minimum leaf size and pruning using validation data.
Compare held-out performance with the training error.

Bagging and random forests

Decision trees

Bagging and random forests

Kernels and SVMs

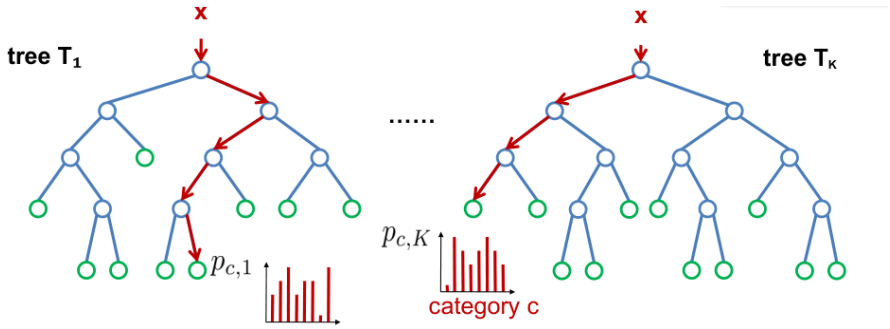
Neural networks

Expressiveness and representations

Backpropagation

2001 Random forests

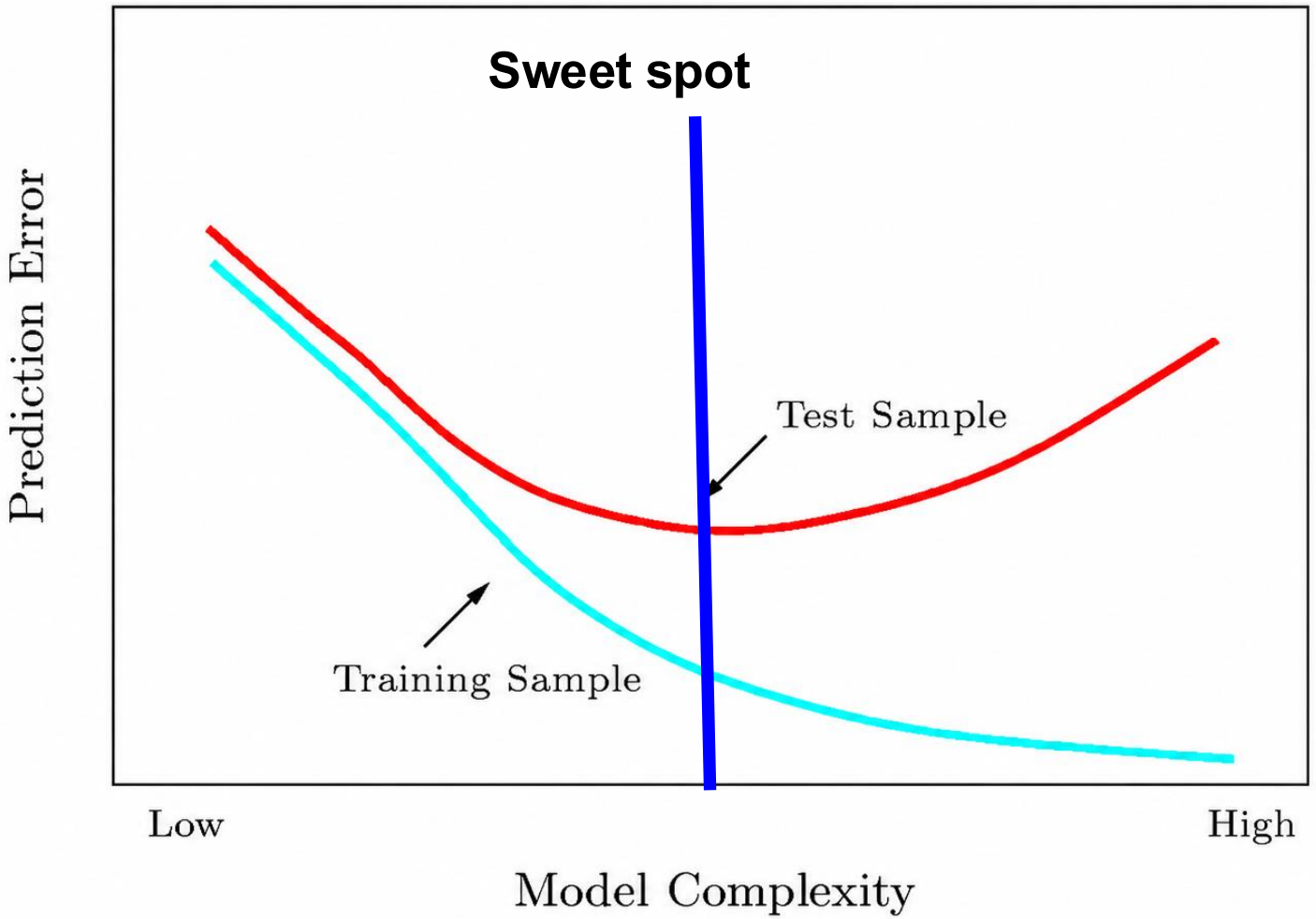
Breiman



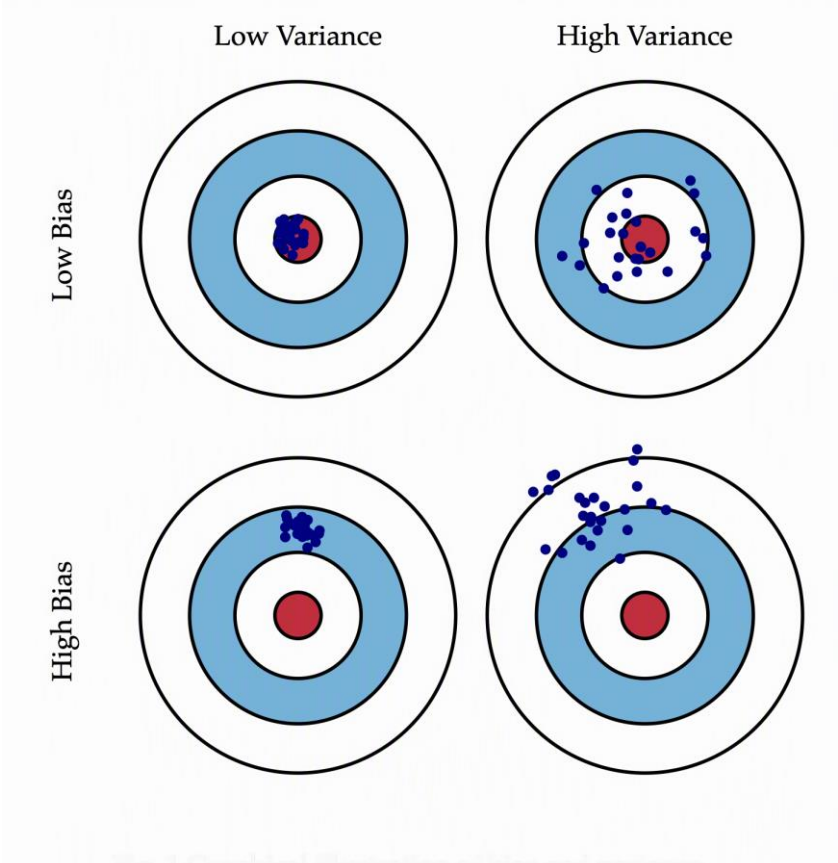
Reading: Bagging and random forests

K. Murphy (2022), §§18.3–18.4 Bagging and random forests

Model complexity and generalization error



Bias and variance - intuition



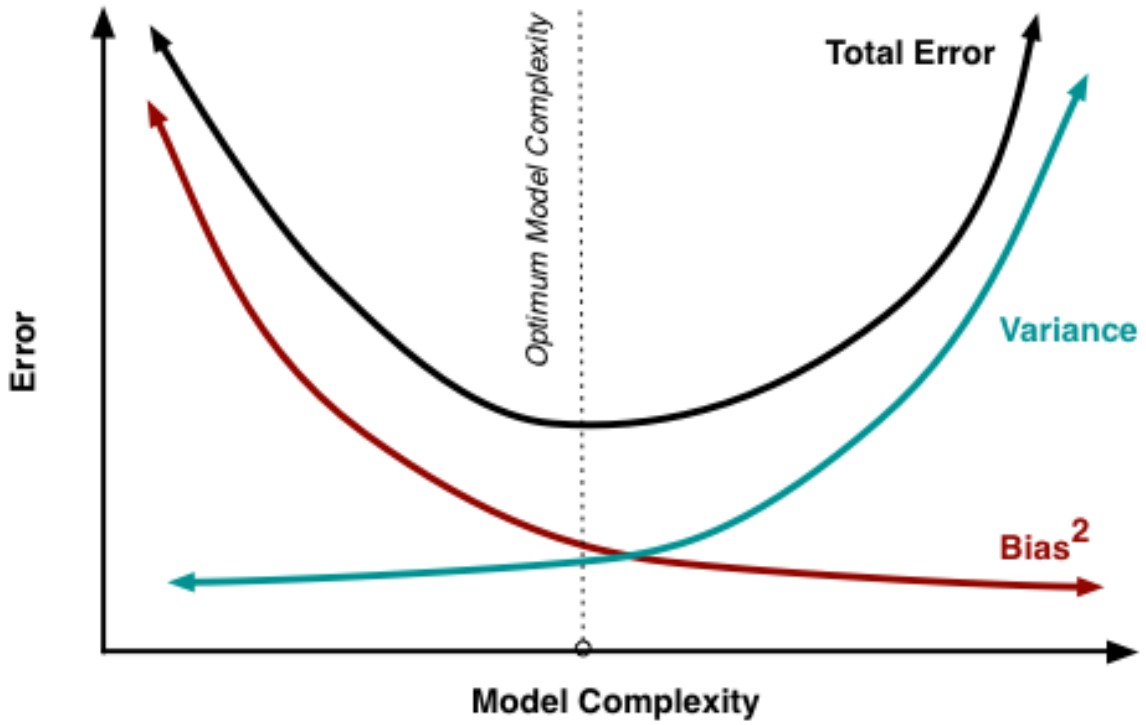
Bias: can your model represent the underlying function?

Variance: will your learned solution depend on the particular training set? (e.g. influence of noise)

https://scott.fortmann-roe.com/docs/BiasVariance.html

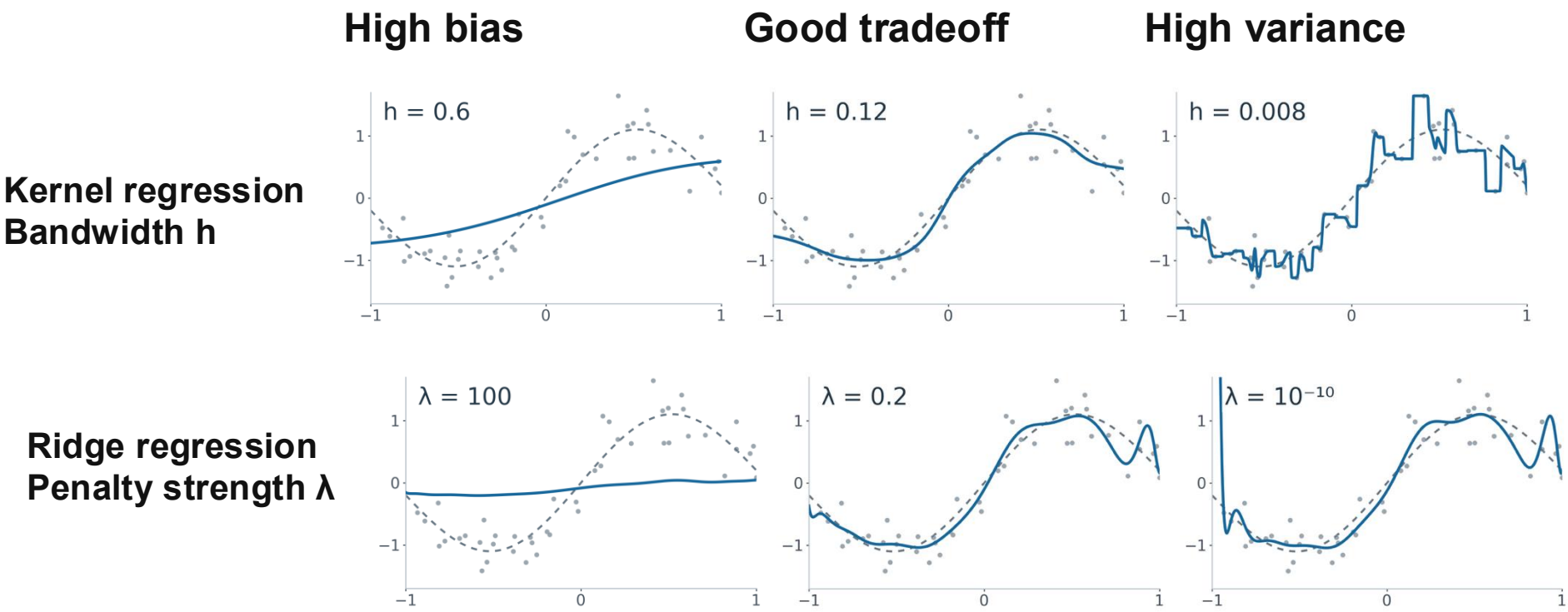
Bias-variance decomposition

$$\mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})] = \mathbb{E}_{\mathbf{X}}[\text{bias}^2(\mathbf{X}) + \text{variance}(\mathbf{X}) + \text{noise}(\mathbf{X})]$$



Schematic; noise is not shown.

Bias and variance: regression examples

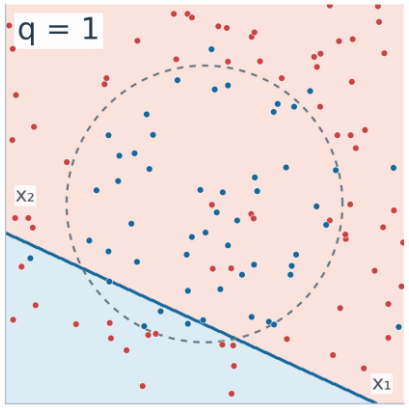
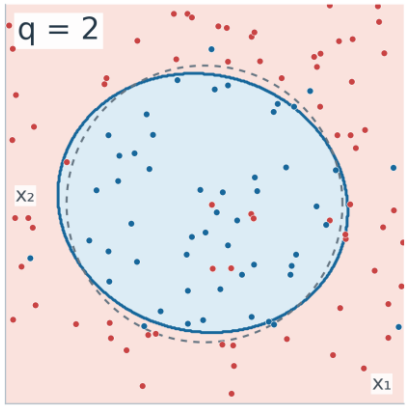
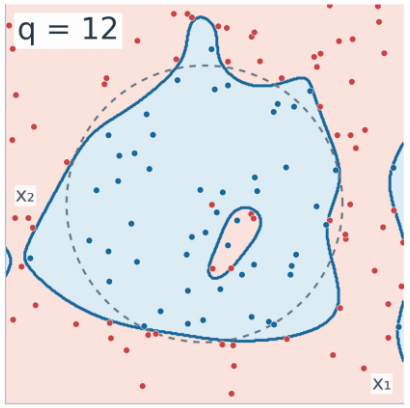
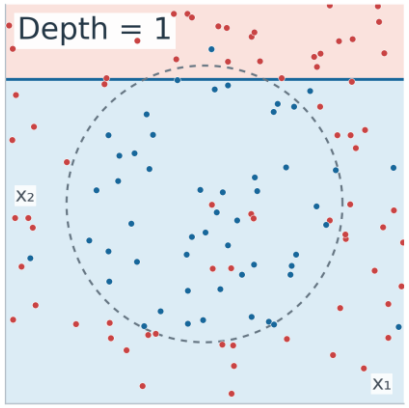
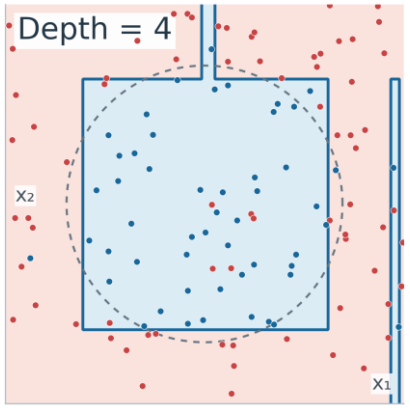
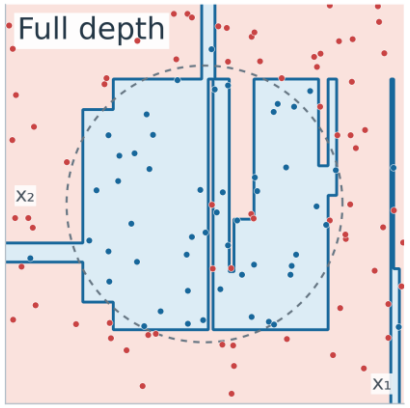


Dots: training data. Dashed: true model. Blue: fitted predictor.

Bias and variance: classification examples

Lecture 2
Logistic regression
Feature degree q

Lecture 3
Decision trees
Maximum depth

High bias	Good tradeoff	High variance
		
		

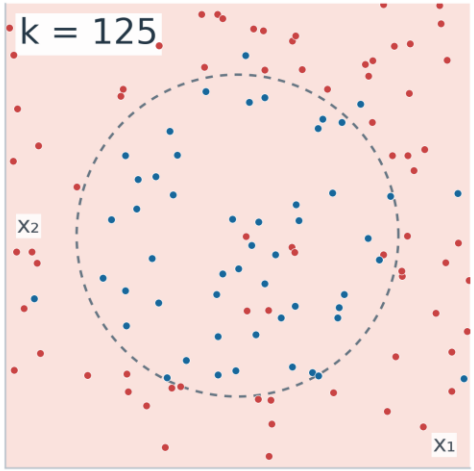
Colour: predicted class. Dots: training labels. Dashed: true boundary.

Illustrative fits · Increasing complexity can fit training noise

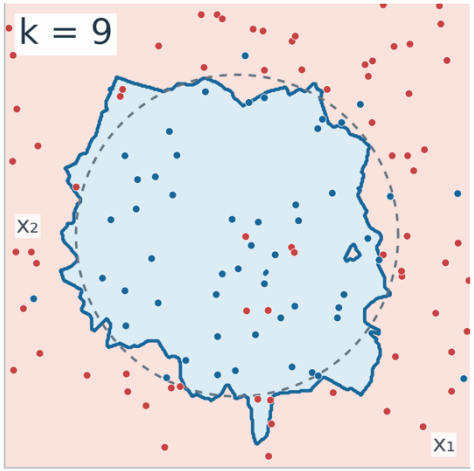
Bias and variance: kNN classification

The same training data, with different neighbourhood sizes.

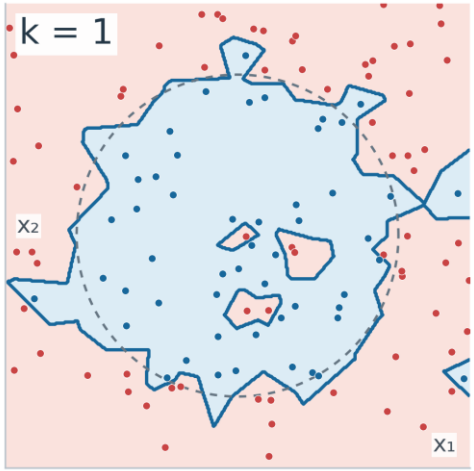
High bias



Good tradeoff



High variance

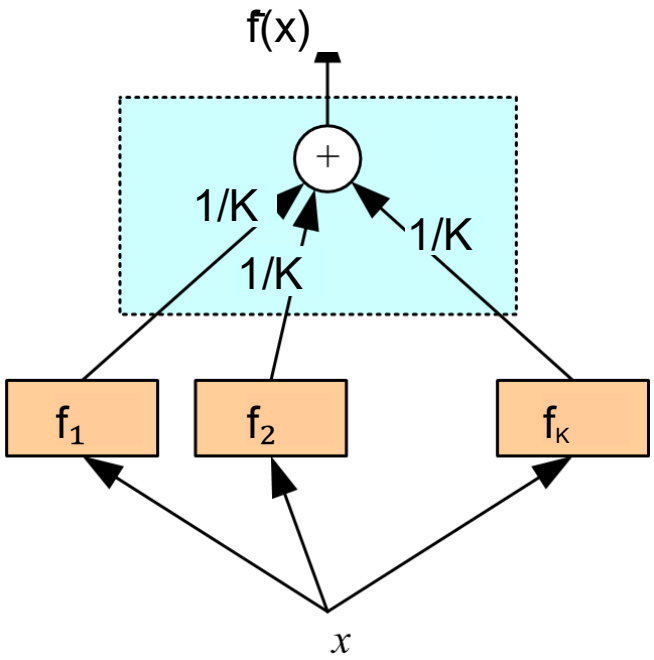


A large k smooths the boundary. A small k can fit individual labels.

Choose k using validation data.

Colour: prediction. Dots: training labels. Dashed: true boundary.

A committee can reduce variance



At input x, average K learners.
$f_k(x)$ is learner k's prediction.

$$\bar{f}(x) = \frac{1}{K}\sum_{k=1}^{K} f_k(x)$$

Averaging models with the same expected prediction leaves their bias unchanged.

For uncorrelated fluctuations with equal variance:

$$\mathrm{Var}[\bar{f}(x)] = \frac{\mathrm{Var}[f_k(x)]}{K}$$

Positive correlation reduces the gain. We need diverse learners.

Error of an average predictor

K predictors $f_k(x)$ form a committee.
$f^*(x)$ is the true function and $\varepsilon_k(x)$ the error of predictor k.

$$\overline{f}(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} f_k(\mathbf{x}), \qquad f_k(\mathbf{x}) = f^*(\mathbf{x}) + \epsilon_k(\mathbf{x})$$

$$E_k = \mathbb{E}_{\mathbf{x}}[\epsilon_k(\mathbf{x})^2], \qquad E_{\mathrm{AV}} = \frac{1}{K}\sum_{k=1}^{K} E_k$$

E_k is expected squared error. E_{AV} averages it over the K predictors.

$$E_{\mathrm{COM}} = \mathbb{E}_{\mathbf{x}}[(\overline{f}(\mathbf{x}) - f^*(\mathbf{x}))^2]$$

$$= \mathbb{E}_{\mathbf{x}}\left[\left(\tfrac{1}{K}\sum_{k=1}^{K}\left(f_k(\mathbf{x}) - f^*(\mathbf{x})\right)\right)^2\right]$$

$$= \mathbb{E}_{\mathbf{x}}\left[\left(\tfrac{1}{K}\sum_{k=1}^{K}\epsilon_k(\mathbf{x})\right)^2\right]$$

Uncorrelated errors can average out

Assume zero-mean errors, uncorrelated across different predictors.

$$\mathbb{E}_{\mathbf{x}}[\epsilon_k(\mathbf{x})\epsilon_{k'}(\mathbf{x})] = 0 \quad \text{for } k \neq k'$$

Expand the square. Cross terms vanish for k ≠ k′.

$$E_{\mathrm{COM}} = \frac{1}{K^2}\left(\sum_k \mathbb{E}_{\mathbf{x}}[\epsilon_k^2] + \sum_k \sum_{k' \neq k} \mathbb{E}_{\mathbf{x}}[\epsilon_k \epsilon_{k'}]\right)$$

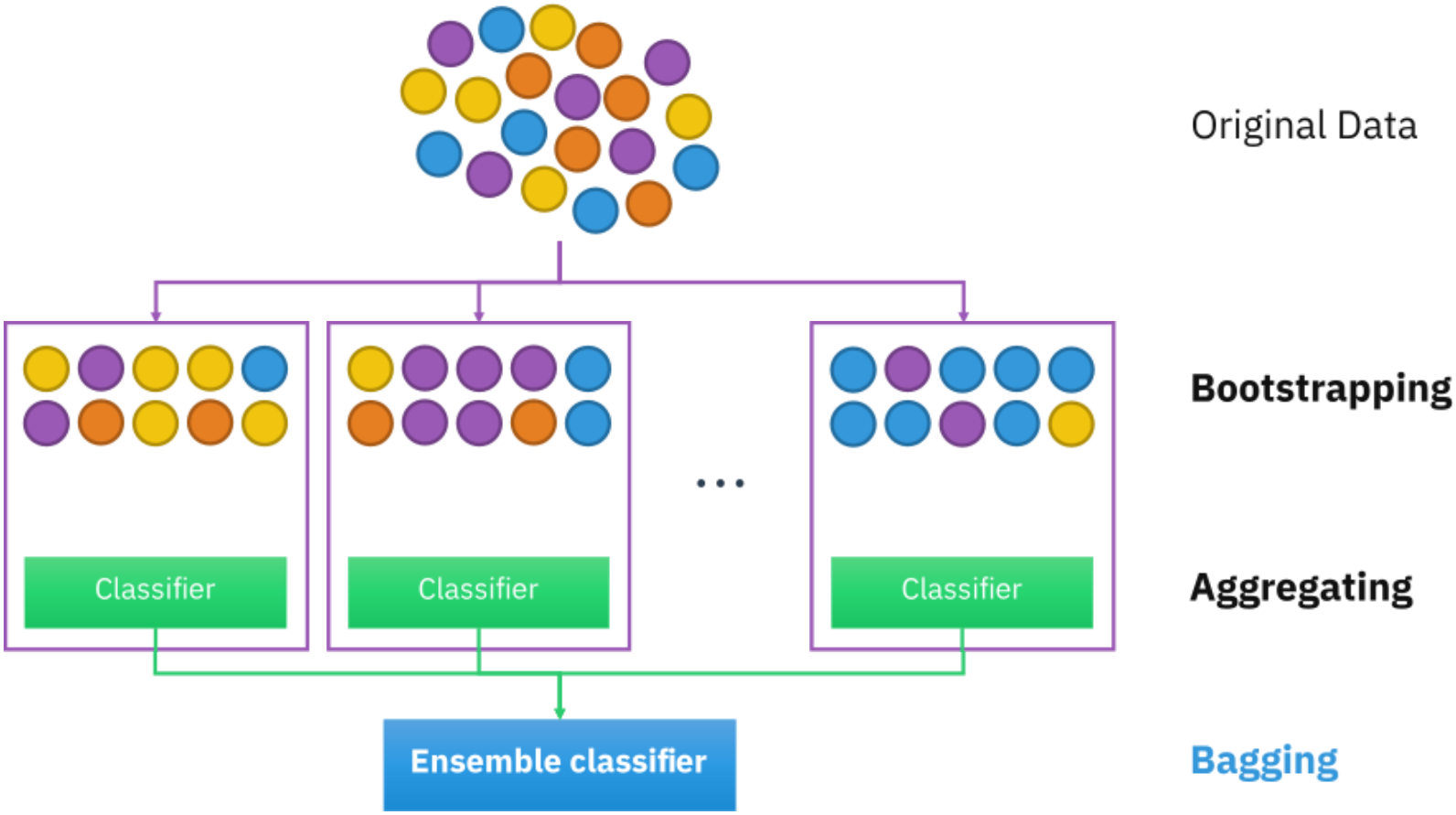
$$= \frac{1}{K^2}\sum_k E_k = \frac{1}{K} E_{\mathrm{AV}}$$

Averaging reduces independent variation. Shared bias remains.

Bagging uses bootstrap samples

Draw N training examples with replacement for each resample.

Fit one base learner per resample, then combine votes or probabilities.



A bootstrap sample

Six training examples:

1 2 3 4 5 6

Six draws with replacement:

1 1 2 4 4 6 1 and 4 repeat. 3 and 5 are omitted.

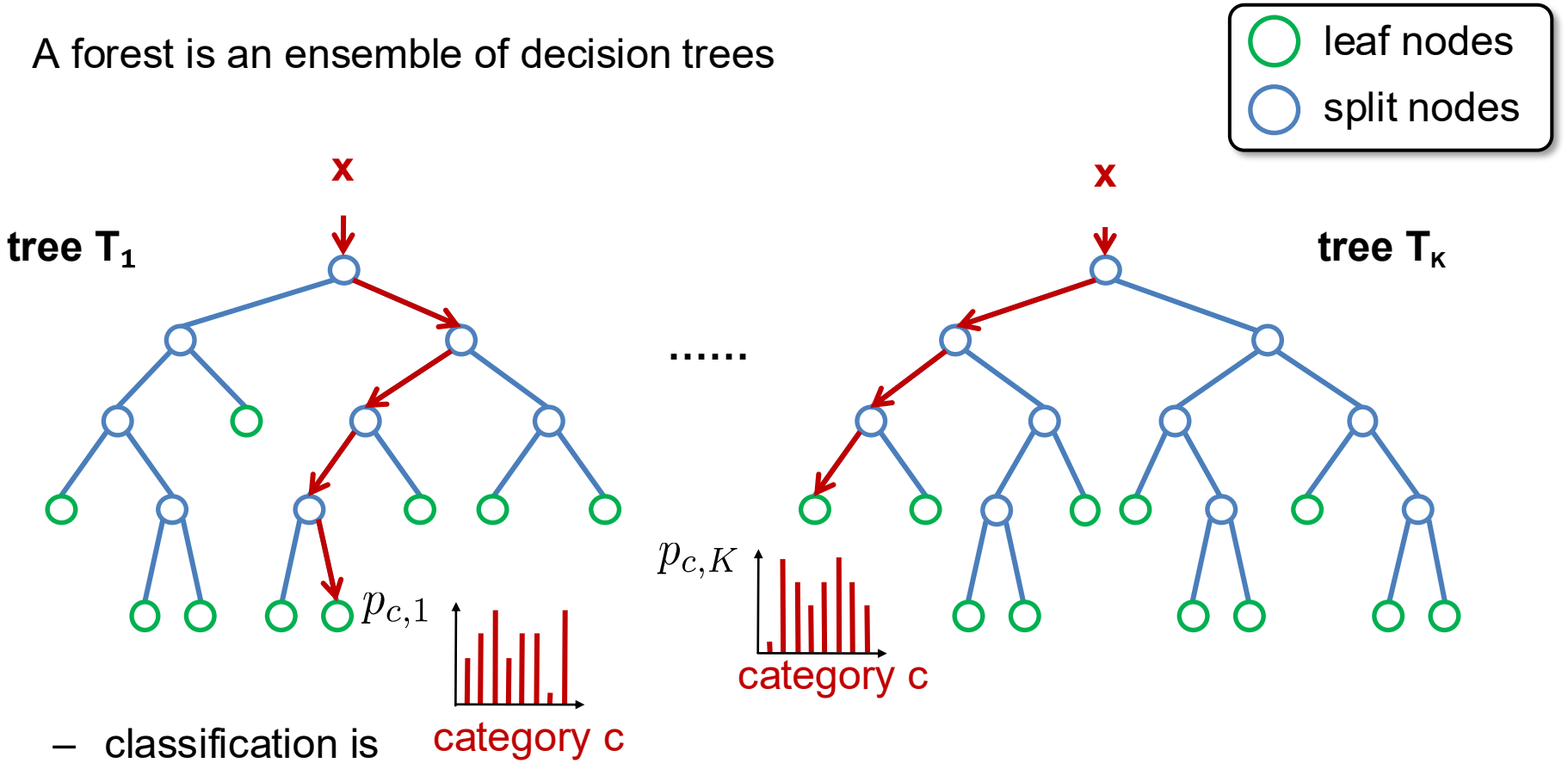
Out-of-bag examples: 3 and 5.
Evaluate each using only models that omitted it during fitting.

Resamples share observations, so their model errors can be correlated.

Random forests also vary the candidate features to reduce correlation.

A forest combines trees

- A forest is an ensemble of decision trees



  - classification is

$$p_c(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} p_{c,k}(\mathbf{x})$$

Training a random forest

- Draw a bootstrap sample to train each tree
- Train each tree:
  - At each node, search a random subset of measured features.

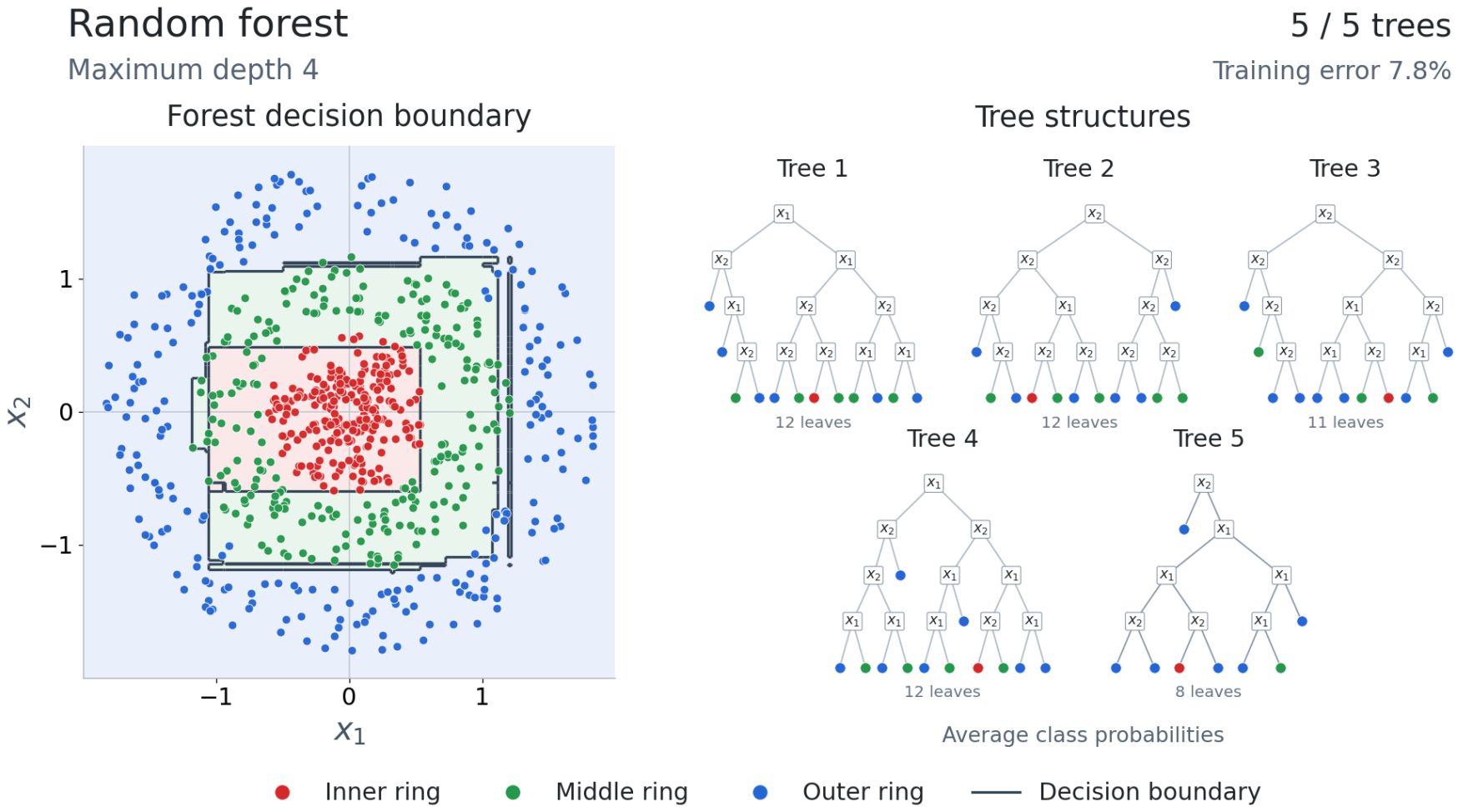
$$\mathcal{F} = \{h_j(\mathbf{x}) = x_j : j = 1, \ldots, d\}, \qquad \mathcal{F}_n \subset \mathcal{F}, \quad |\mathcal{F}_n| = m < d$$

  - Choose the best split within that subset, then recurse.

$$(h_n^*, \theta_n^*) = \underset{h \in \mathcal{F}_n,\, \theta}{\arg\max}\, \mathrm{IG}(I_n; h, \theta)$$

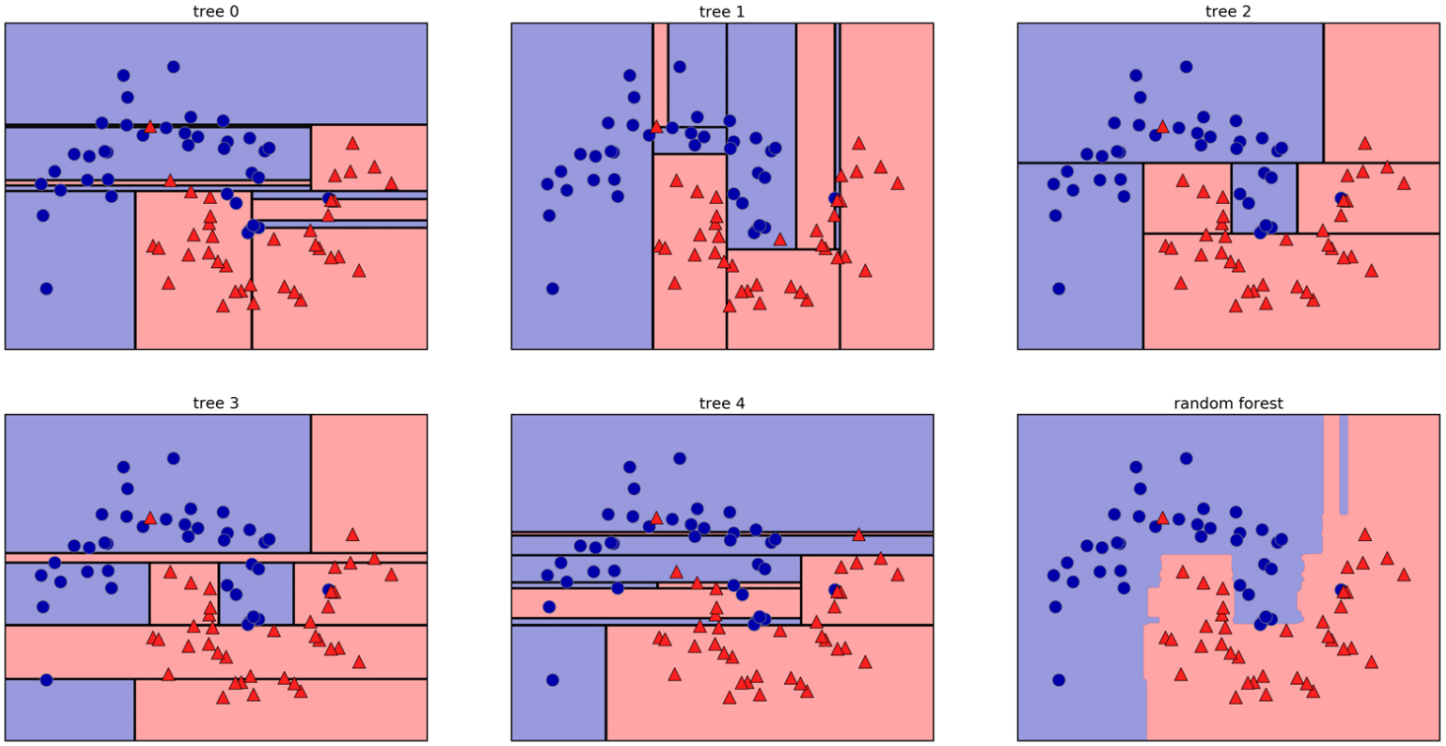
- Feature variation reduces correlation between tree predictions.
- Combine probabilities or votes.
- Choose complexity using validation or out-of-bag predictions.

A forest composed of five trees



Five trees, maximum depth 4. Average their class probabilities.

A forest composed of 5 trees



Limits of greedy feature selection

Extremely flexible classifiers (power of composition)

Low-cost (simple comparisons, no kernel evaluations)

But: learned greedily

Later splits do not jointly retune the earlier tests.

An alternative: use a vast feature space through a kernel.

Kernels and SVMs

Decision trees

Bagging and random forests

Kernels and SVMs

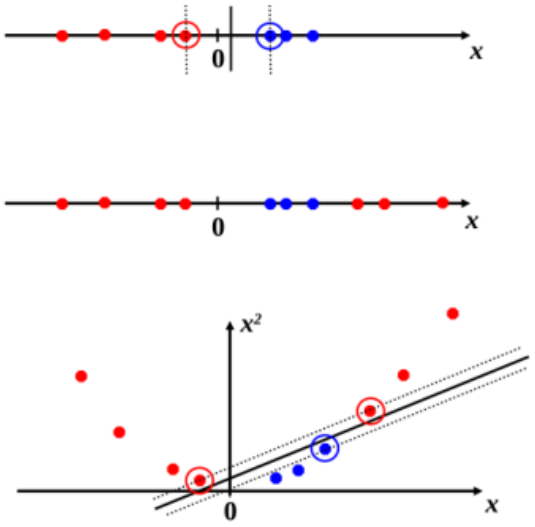
Neural networks

Expressiveness and representations

Backpropagation

1995 Support vector machines

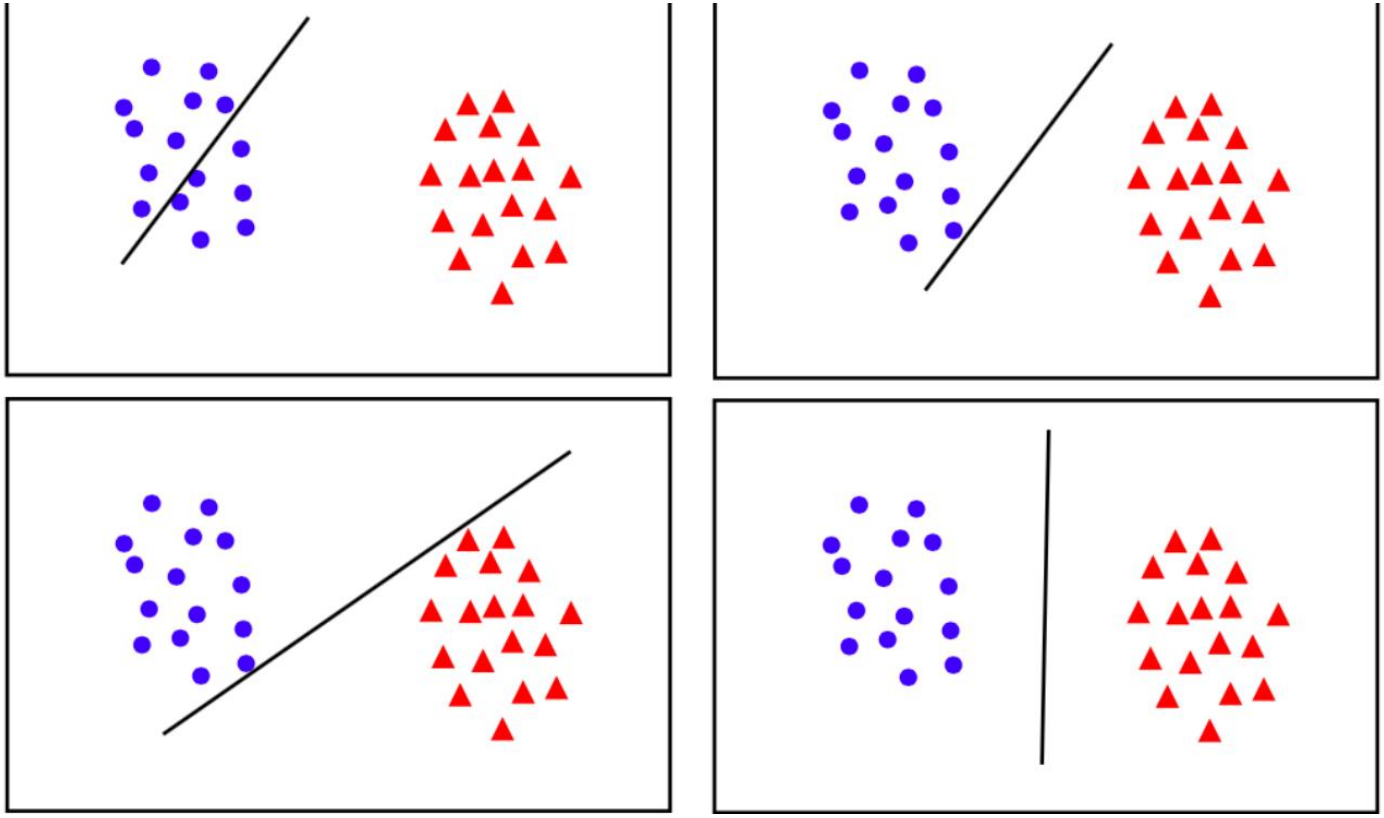
Cortes and Vapnik



Reading: Kernels and SVMs

K. Murphy (2022), §§17.1, 17.3 Kernels and support vector machines

Choosing a separating hyperplane



Which boundary has greatest clearance from the training data?

A support vector machine (SVM) chooses a large-margin separator.
First use the input space; then introduce a kernel.

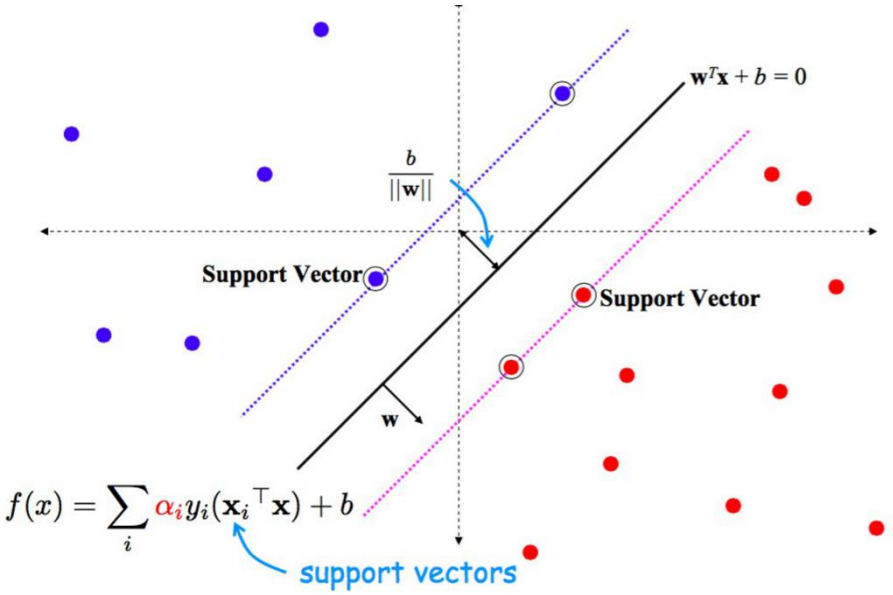
Support vectors determine the classifier

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i$$

$$y_i \in \{-1, +1\}$$

$\alpha_i \geq 0$ is learned.
Only $\alpha_i > 0$ contribute:
the **support vectors**.

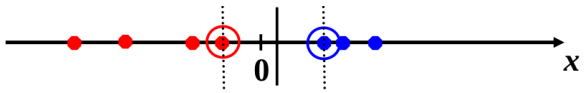
$$\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$



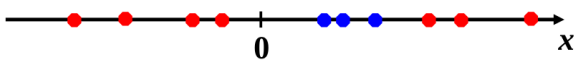
Optimisation chooses the signed coefficients $\alpha_i y_i$ and the bias b.
We will not derive the optimisation problem in this course.

Nonlinear separation through features

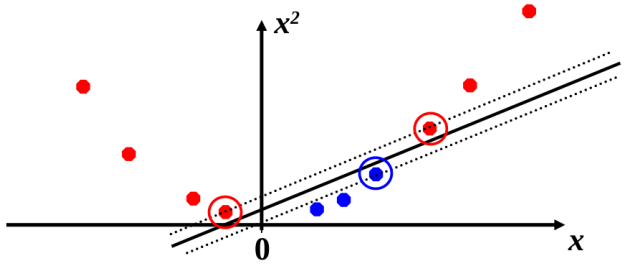
A linear boundary in the original input may be insufficient.



$$\mathbf{x} \mapsto \phi(\mathbf{x})$$



A linear classifier in the feature space can be nonlinear in the original input.



A classifier as a kernel expansion

The learned weight vector is a combination of support-vector features.

$$\mathbf{w} = \sum_{i \in \mathrm{SV}} \alpha_i y_i \, \boldsymbol{\phi}(\mathbf{x}_i)$$

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) + b$$

$$= \sum_{i \in \mathrm{SV}} \alpha_i y_i \underbrace{\boldsymbol{\phi}(\mathbf{x}_i)^T \boldsymbol{\phi}(\mathbf{x})}_{k(\mathbf{x}_i, \mathbf{x})} + b$$

SV: support vectors. Labels $y_i = \pm 1$. Coefficients $\alpha_i \geq 0$ are learned.

Prediction only needs kernel evaluations at the support vectors.

Polynomial kernels and explicit features

$$k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z})^3$$

For two input coordinates, a cubic feature map has four terms:

$$\mathbf{x} \in \mathbb{R}^2 : \quad \phi(\mathbf{x}) = (x_1^3, \sqrt{3}x_1^2 x_2, \sqrt{3}x_1 x_2^2, x_2^3)^T$$

$$\phi(\mathbf{x})^T \phi(\mathbf{z}) = (x_1 z_1 + x_2 z_2)^3$$

In d dimensions, the number of cubic monomials grows rapidly:

$$\mathbf{x} \in \mathbb{R}^d : \qquad \dim \phi(\mathbf{x}) = \binom{d+2}{3} \quad (d = 100 : \ 171{,}700)$$

Explicit features: evaluate each input's monomials, then take their inner product.

Kernel: take one d-term inner product and cube the result.

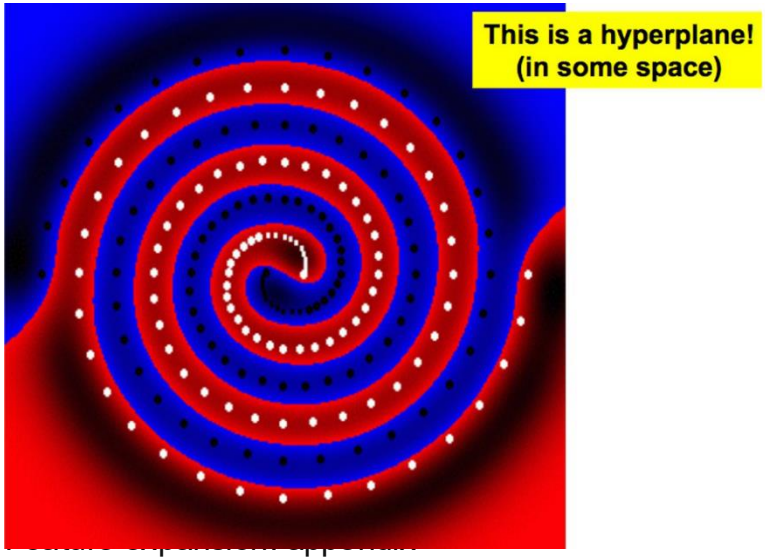
Gaussian kernels and flexible boundaries

As in kernel regression, a Gaussian gives nearby inputs more influence.

$$k(\mathbf{x}, \mathbf{z}) = \exp\left[-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}\right], \qquad \sigma > 0$$

The SVM combines Gaussians centred on its support vectors.

The feature space is infinite-dimensional (see appendix), but each kernel evaluation is finite.



A linear decision in feature space can produce a complicated boundary in input space.

Learning the features and the predictor

A rich feature space can make a simple final decision powerful.

Trees and forests

Select paths of tests from many possible combinations.
Choose splits greedily, without jointly retuning earlier tests.

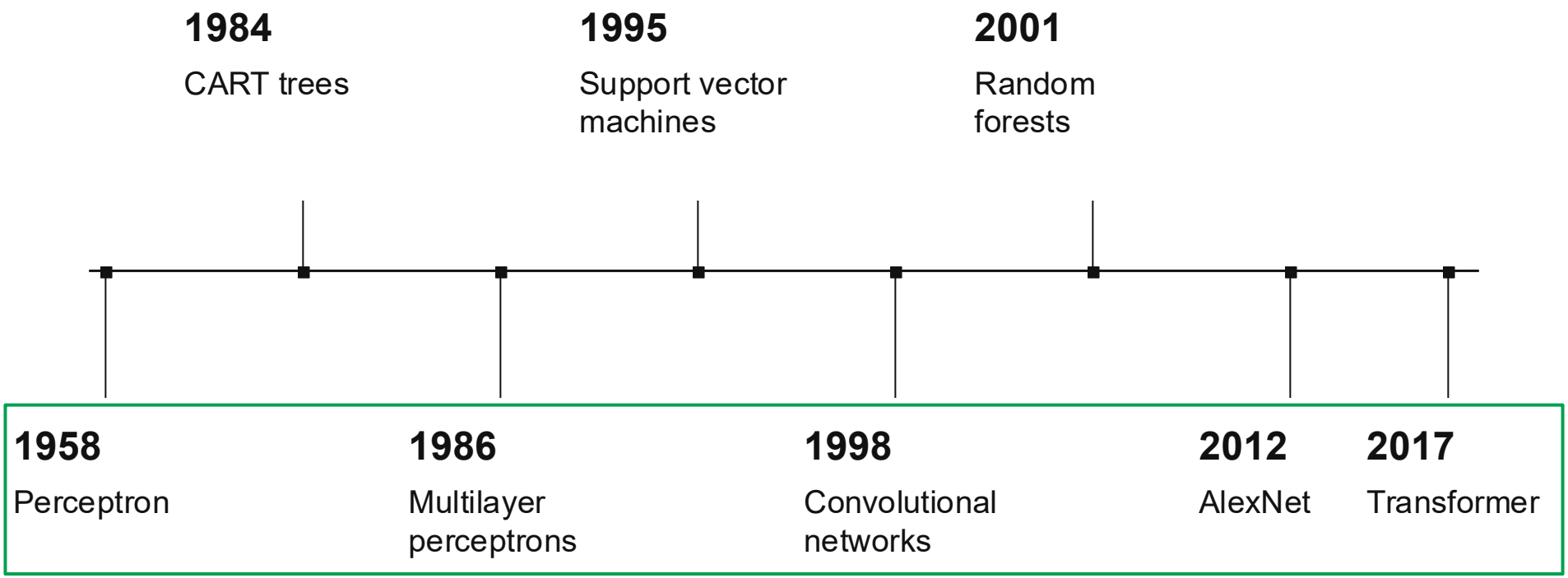
Fixed-kernel methods

Use implicit features through kernel evaluations.
Fit classifier coefficients while the chosen feature map stays fixed.

Neural networks

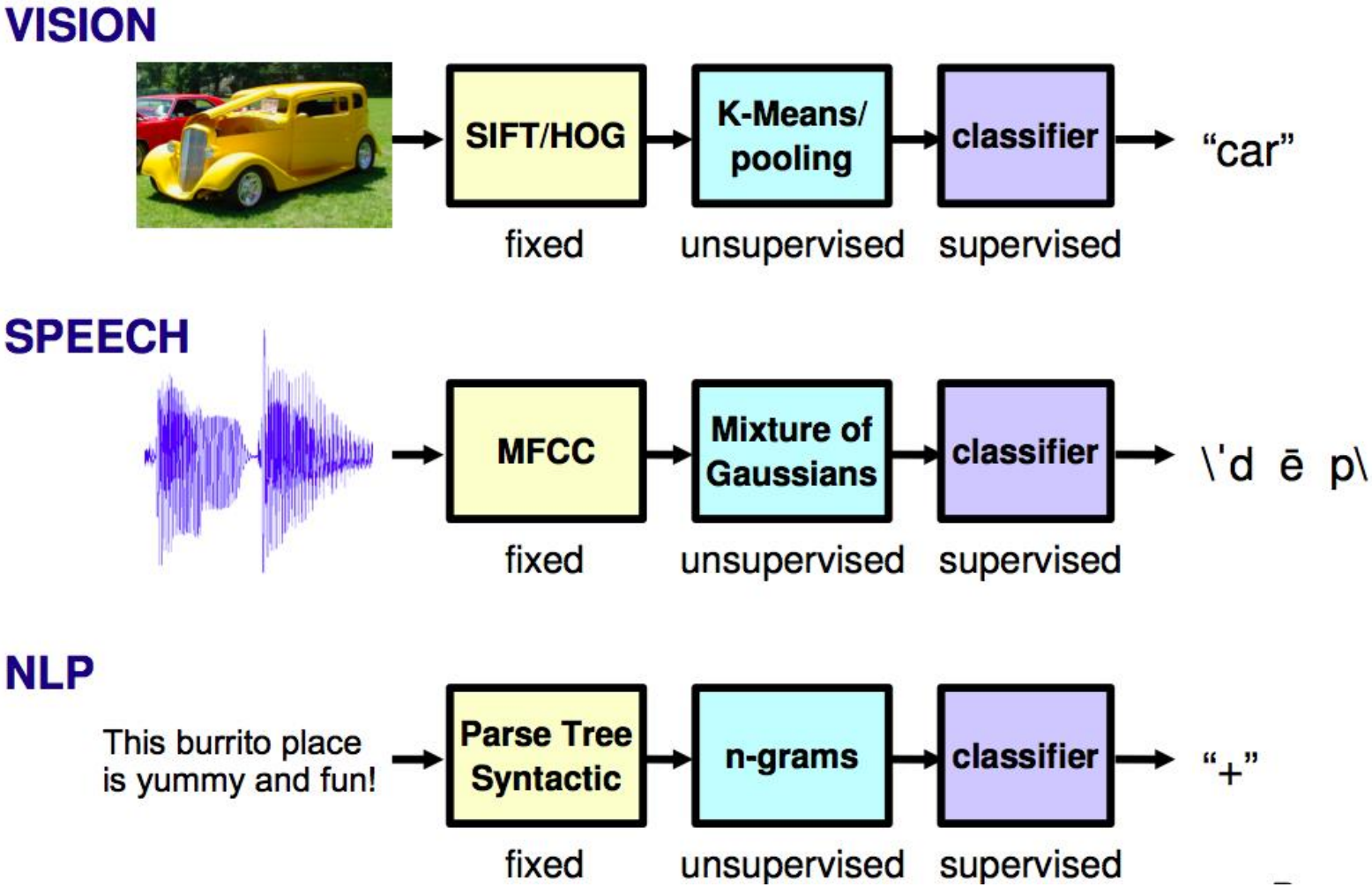
Optimise intermediate features and the final predictor together, using the final prediction loss.

Selected milestones in machine learning



Deep Learning

Machine Learning pre Deep Learning



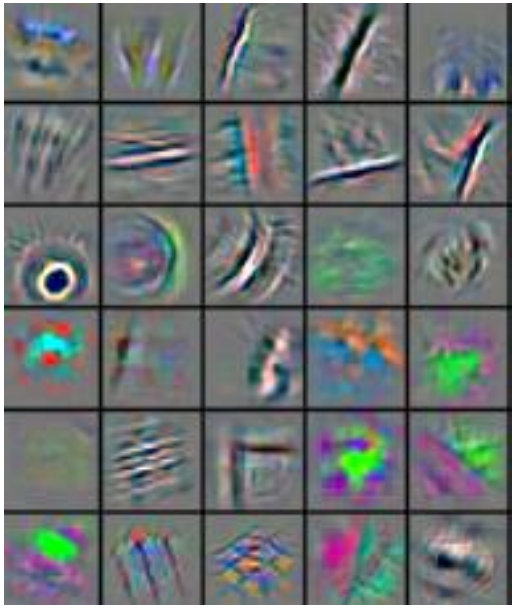
Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

Deep Learning = learning feature compositions



Low-Level Feature → Mid-Level Feature → High-Level Feature → Trainable Classifier → "car"



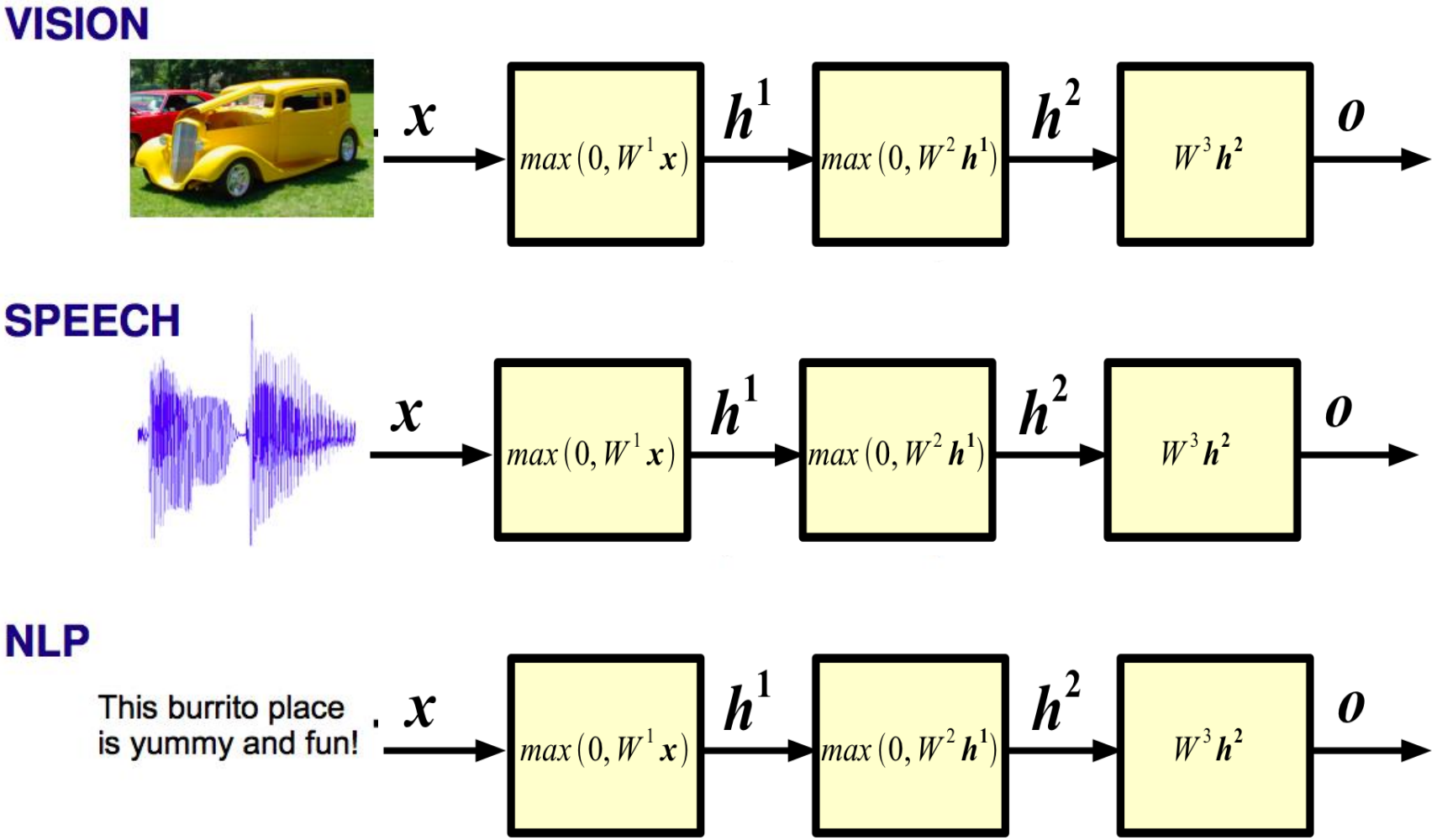




Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

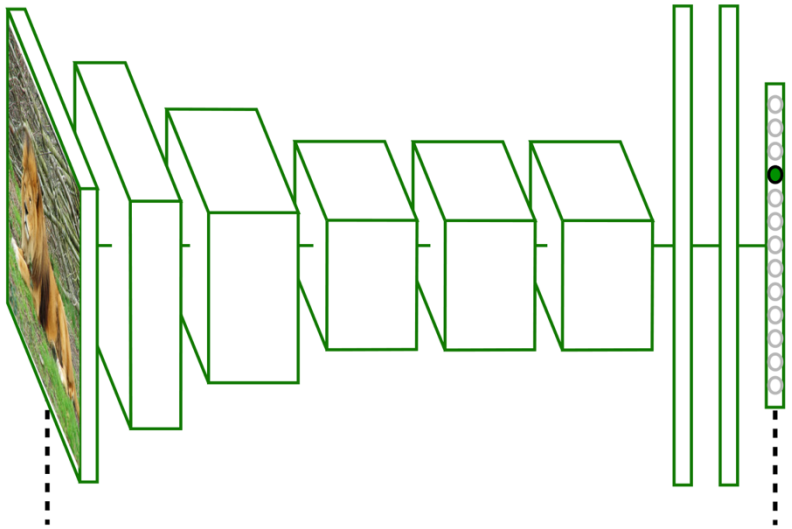
Deep Learning: learn the features!



Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

2012: AlexNet

AlexNet used a deep convolutional network for ImageNet classification.



ImageNet top-5 error

AlexNet: 16.4%
Best shallow competitor: 26%

Next week: convolutional networks and ImageNet

Krizhevsky, Sutskever and Hinton (2012). Comparison reused from Lecture 4.

Course schedule

- 1st lecture: Intro & Regression
- 2nd lecture: Classification
- 3rd lecture: Neural Networks
- 4th lecture: Convolutional Neural Networks
- 5th lecture: Transformers
- 6th lecture: Unsupervised learning
- 7th lecture: Generative modelling
- 8th lecture: Revision & seminar

Lectures 1–2: supervised problems. Lectures 3–5: deep learning.
Buildup to unsupervised learning in 6 and generative models in 7.

Neural networks

Decision trees

Bagging and random forests

Kernels and SVMs

Neural networks

Expressiveness and representations

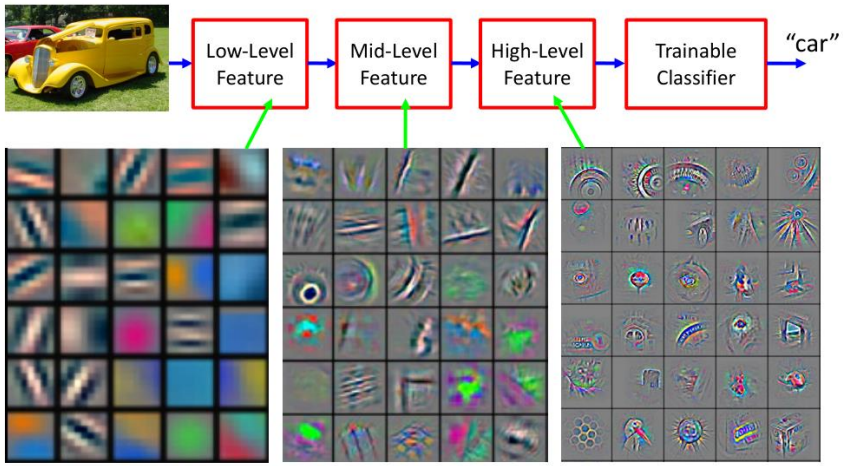
Backpropagation

1958 Perceptron

Rosenblatt

1986 Multilayer perceptrons

Rumelhart, Hinton and Williams



Ranzato and LeCun; Zeiler and Fergus (2013)

Reading: Multilayer perceptrons

K. Murphy (2022), §§10.2.5, 13.2

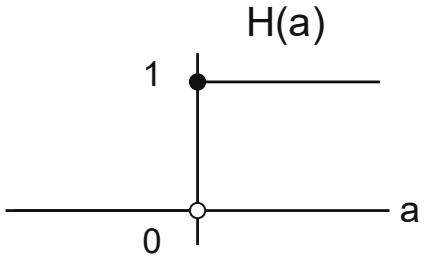
The perceptron: a hard decision

Lecture 2: logistic regression outputs a probability

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

A perceptron uses a hard threshold (Heaviside unit)

$$\hat{y} = H(\mathbf{w}^T \mathbf{x} + b)$$



For the same weights, threshold the probability at 0.5

$$\sigma(\mathbf{w}^T \mathbf{x} + b) \geq 0.5 \iff \mathbf{w}^T \mathbf{x} + b \geq 0$$

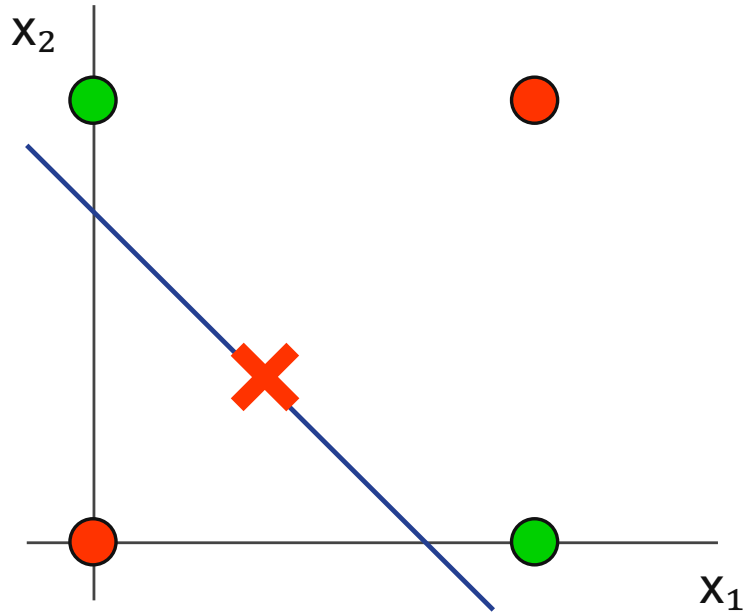
The perceptron is a deterministic classifier.
Its learning rule differs from logistic regression (not covered).

The XOR problem

XOR is 1 when the two binary inputs differ, and 0 otherwise.

$$\hat{y} = H(b + w_1 x_1 + w_2 x_2)$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



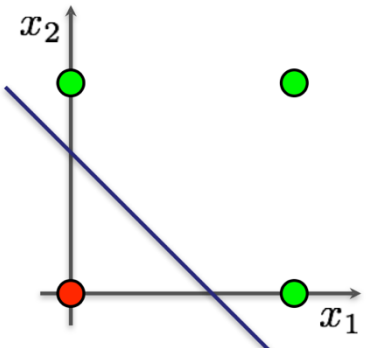
No straight line separates the two classes.
A single perceptron cannot represent XOR.

XOR: two useful features

Compute two useful features: OR and AND.

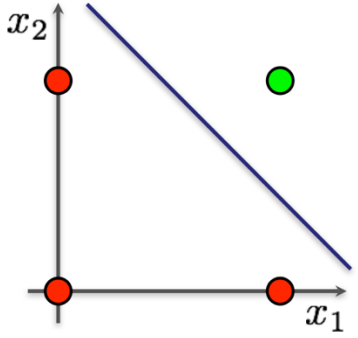
OR feature

x_1	x_2	z_1
0	0	0
0	1	1
1	0	1
1	1	1



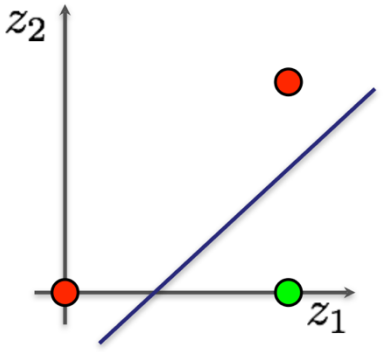
AND feature

x_1	x_2	z_2
0	0	0
0	1	0
1	0	0
1	1	1



OR, but not AND

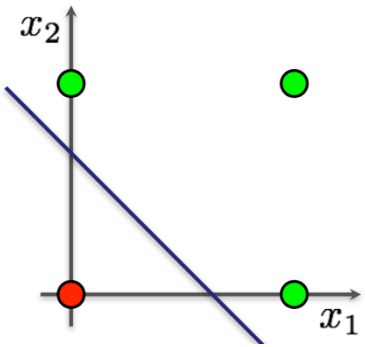
z_1	z_2	y
0	0	0
1	0	1
1	0	1
1	1	0

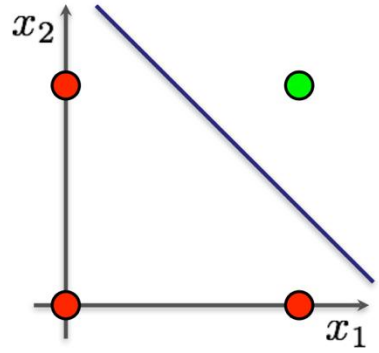


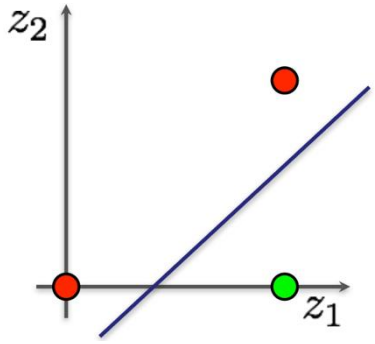
The two positive XOR inputs both become $(z_1, z_2) = (1, 0)$.

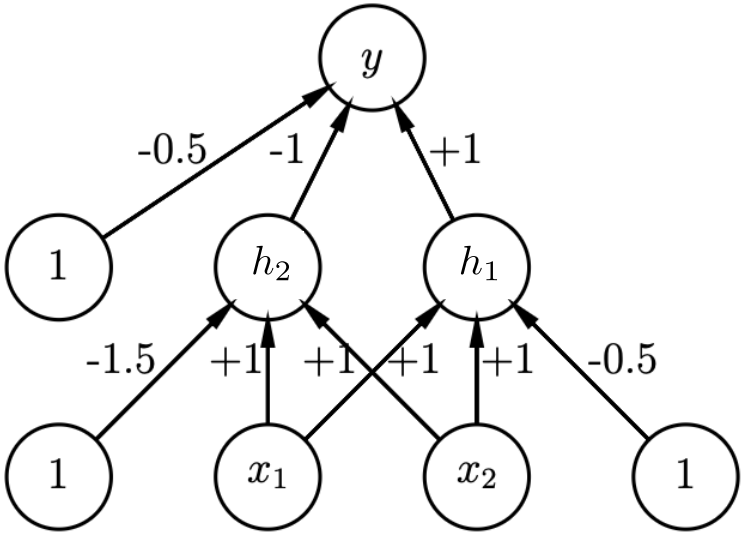
A hand-crafted network for XOR

$$z_1 = H(x_1 + x_2 - 0.5) \qquad z_2 = H(x_1 + x_2 - 1.5) \qquad y = H(z_1 - z_2 - 0.5)$$







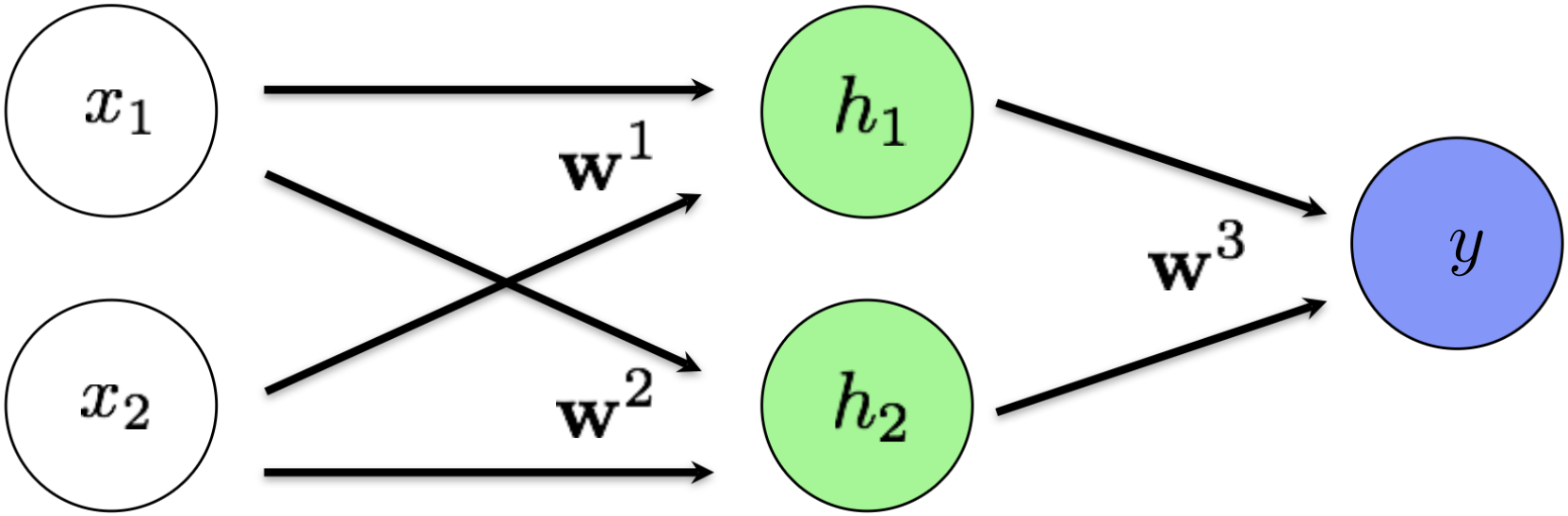


K. Murphy (2022), Fig. 13.1(b). $h_1 = z_1$ (OR), $h_2 = z_2$ (AND).

XOR: the composition is a network

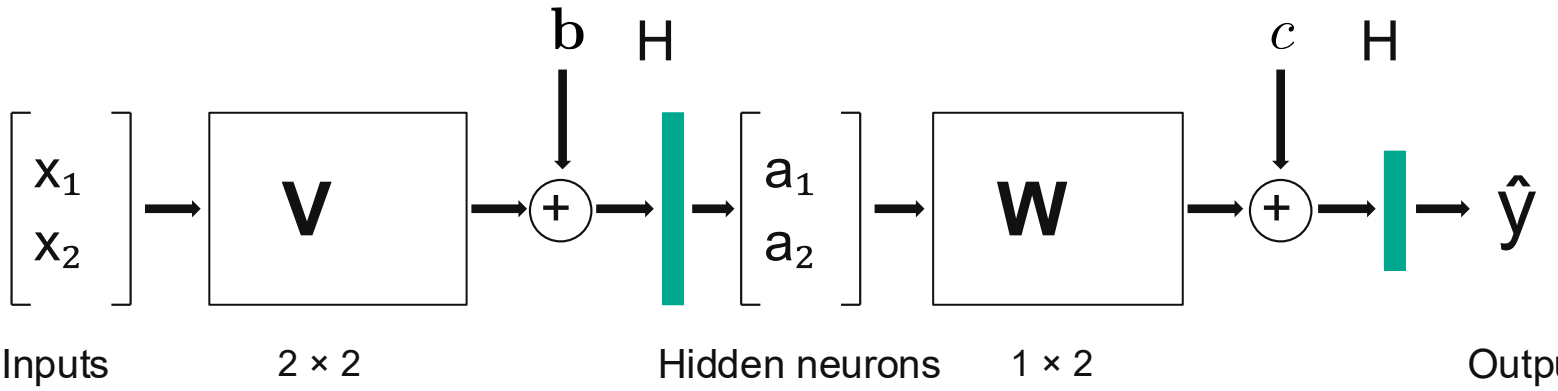
$$y = f\big(g_1(x_1, x_2), g_2(x_1, x_2)\big)$$

The hidden functions g_1 and g_2 compute OR and AND.
One output unit combines these two hidden activations.



Hidden unit 1: $h_1 = z_1$ (OR). Hidden unit 2: $h_2 = z_2$ (AND).
The output y combines these two features to compute XOR.

A two-layer network for XOR



$$a_1 = H(V_{11}x_1 + V_{12}x_2 + b_1) \qquad \hat{y} = H(W_{11}a_1 + W_{12}a_2 + c)$$

$$a_2 = H(V_{21}x_1 + V_{22}x_2 + b_2)$$

$$\mathbf{V} = \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix} \qquad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} \qquad \mathbf{W} = \begin{bmatrix} W_{11} & W_{12} \end{bmatrix}, \qquad c \in \mathbb{R}$$

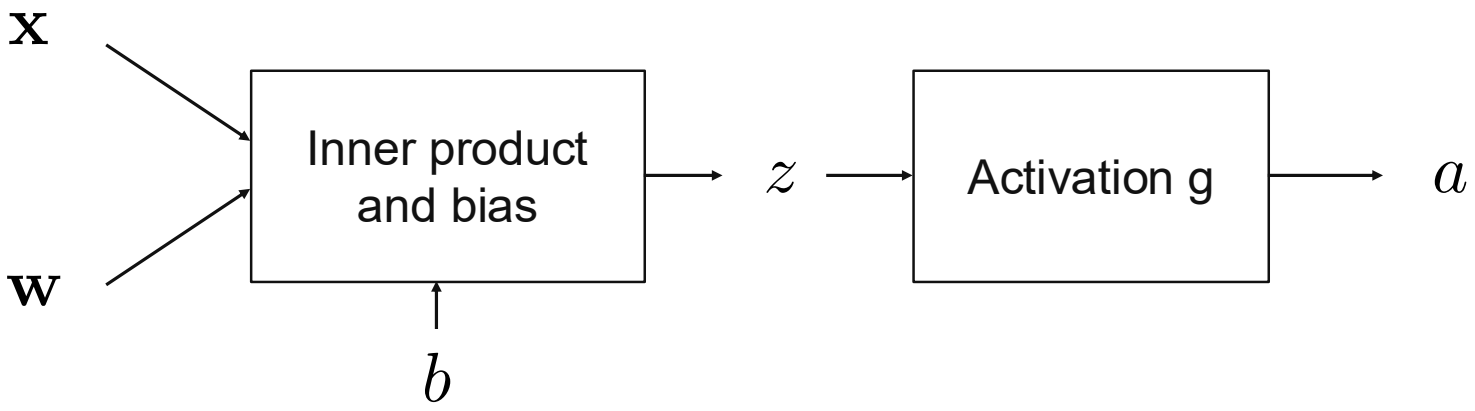
For compact notation, an affine map and activation form one layer.

$$\mathbf{a} = H(\mathbf{V}\mathbf{x} + \mathbf{b}), \qquad \hat{y} = H(\mathbf{W}\mathbf{a} + c)$$

With the XOR weights: a_1 = OR, a_2 = AND, and $\hat{y}$ = XOR.

The basic computation unit: a neuron

$$\mathbf{x}, \mathbf{w} \in \mathbb{R}^d, \quad b \in \mathbb{R}$$



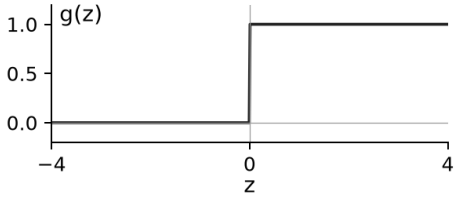
$$z = \mathbf{w}^T \mathbf{x} + b \in \mathbb{R} \qquad a = g(z)$$

A neuron takes a weighted sum of its inputs, adds a bias, then applies a scalar nonlinearity.

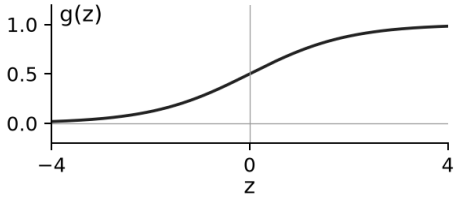
Activation functions

$$z = \mathbf{w}^T \mathbf{x} + b, \qquad a = g(z)$$

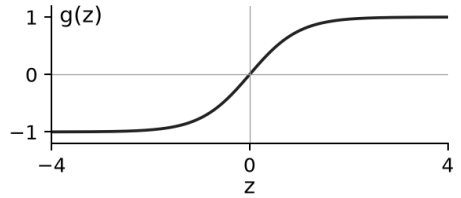
Step (perceptron) $$g(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$$



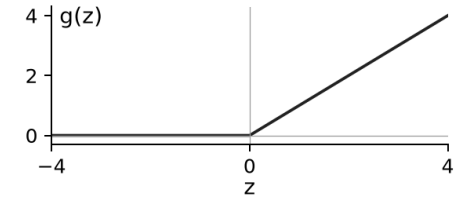
Sigmoid $$g(z) = \frac{1}{1 + \exp(-z)}$$



Tanh $$g(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$



ReLU $$g(z) = \max(0, z)$$



Why the nonlinearity matters

Two affine layers collapse into one affine map:

$$\mathbf{W}(\mathbf{V}\mathbf{x} + \mathbf{b}) + c$$

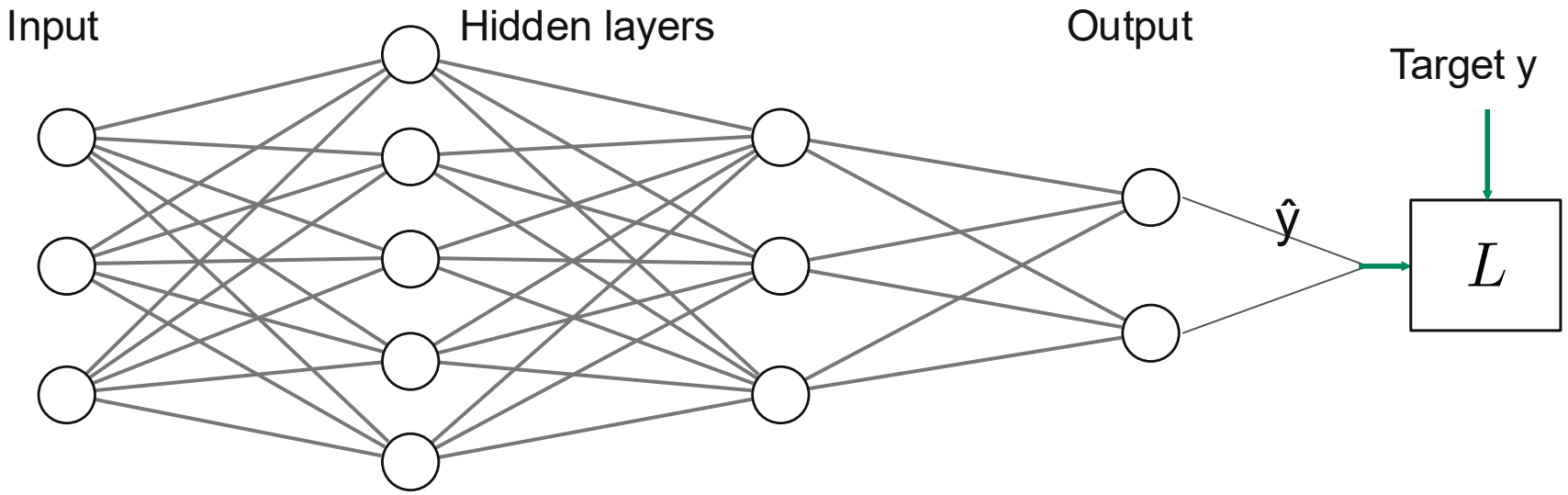
$$= (\mathbf{W}\mathbf{V})\mathbf{x} + (\mathbf{W}\mathbf{b} + c)$$

V and W are weights. The hidden-layer bias b is a vector; the output intercept c is a scalar.

Nonlinear activations make composition more expressive.

Gradients throughout the network

Learn every weight and bias by differentiating the scalar loss L.



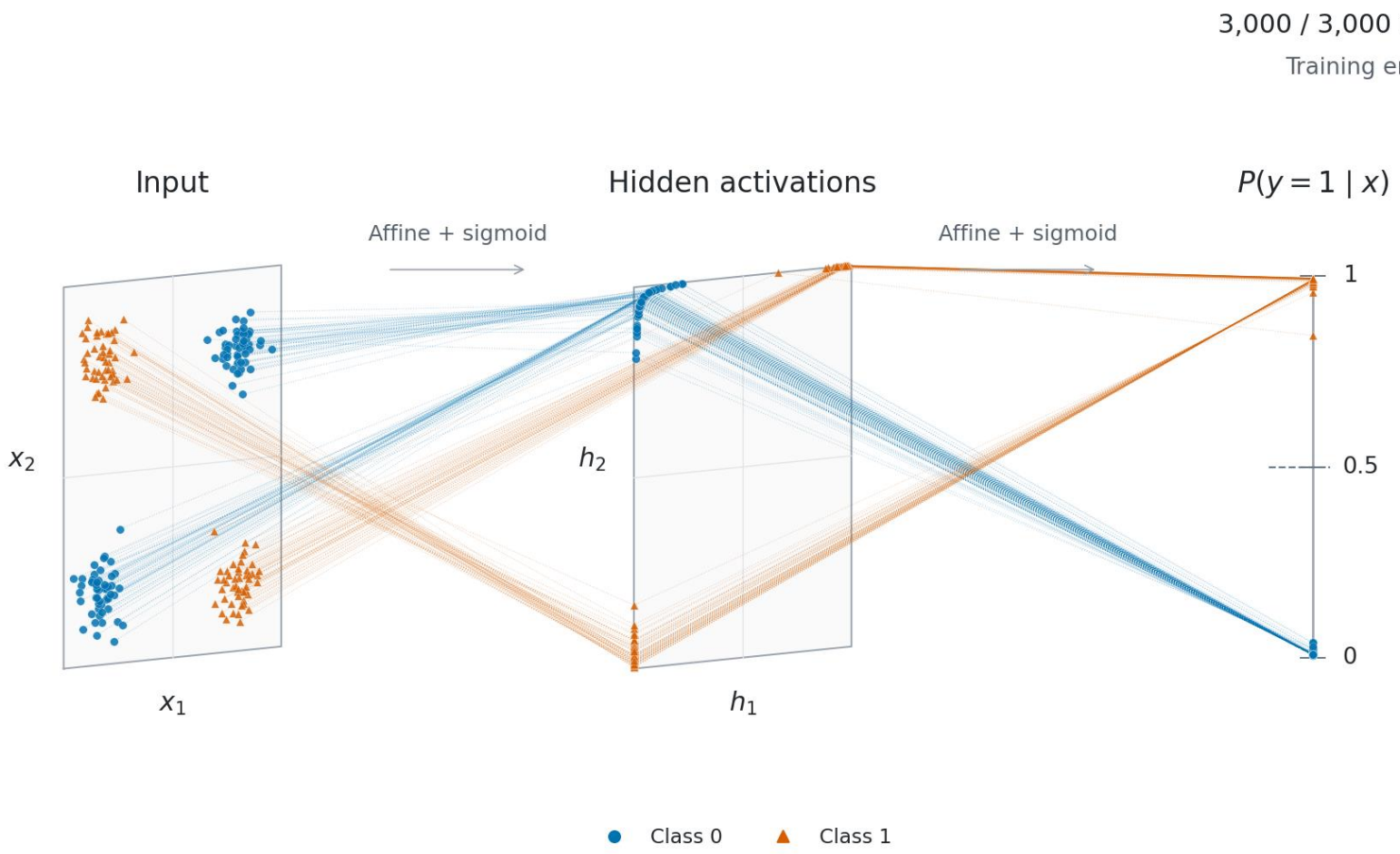
Derivatives for every weight and bias

$$\frac{\partial L}{\partial W_{ij}^{(l)}}, \qquad \frac{\partial L}{\partial b_i^{(l)}}$$

Adapted from G. Hinton

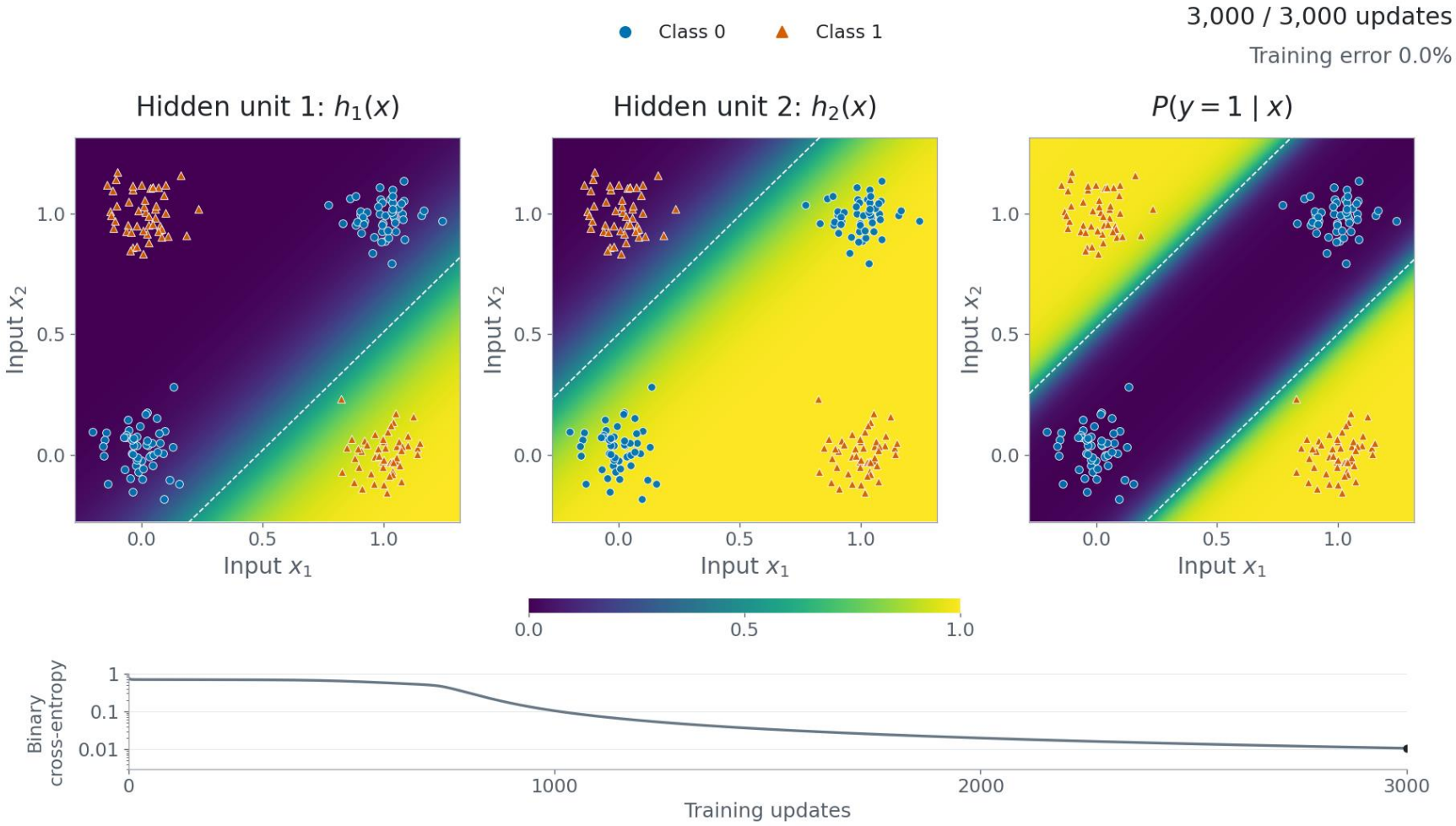
Learning XOR: representations during training

Two sigmoid hidden units feed a single sigmoid output.



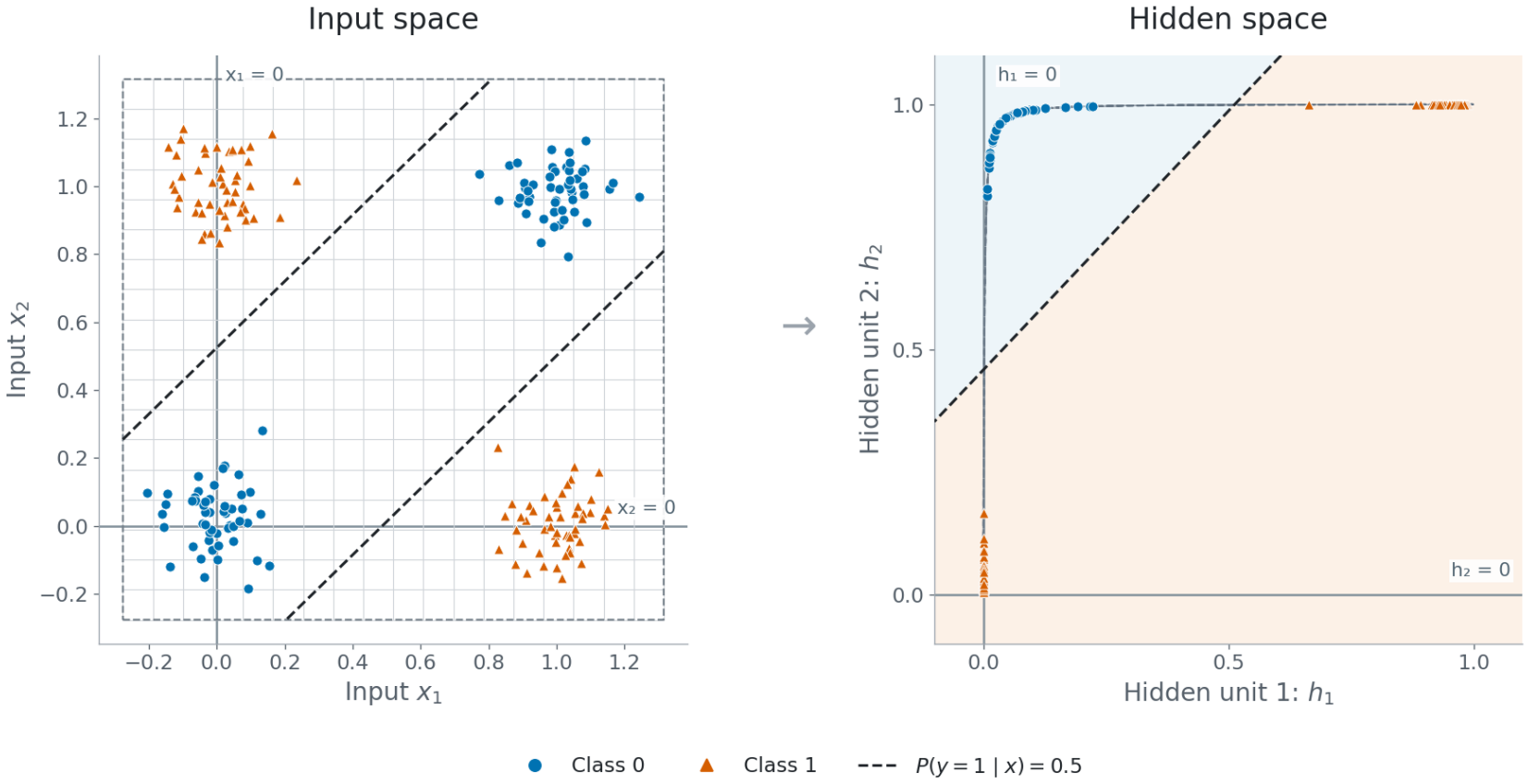
Hidden activations and output probability

The same network evaluated densely over the original input space.



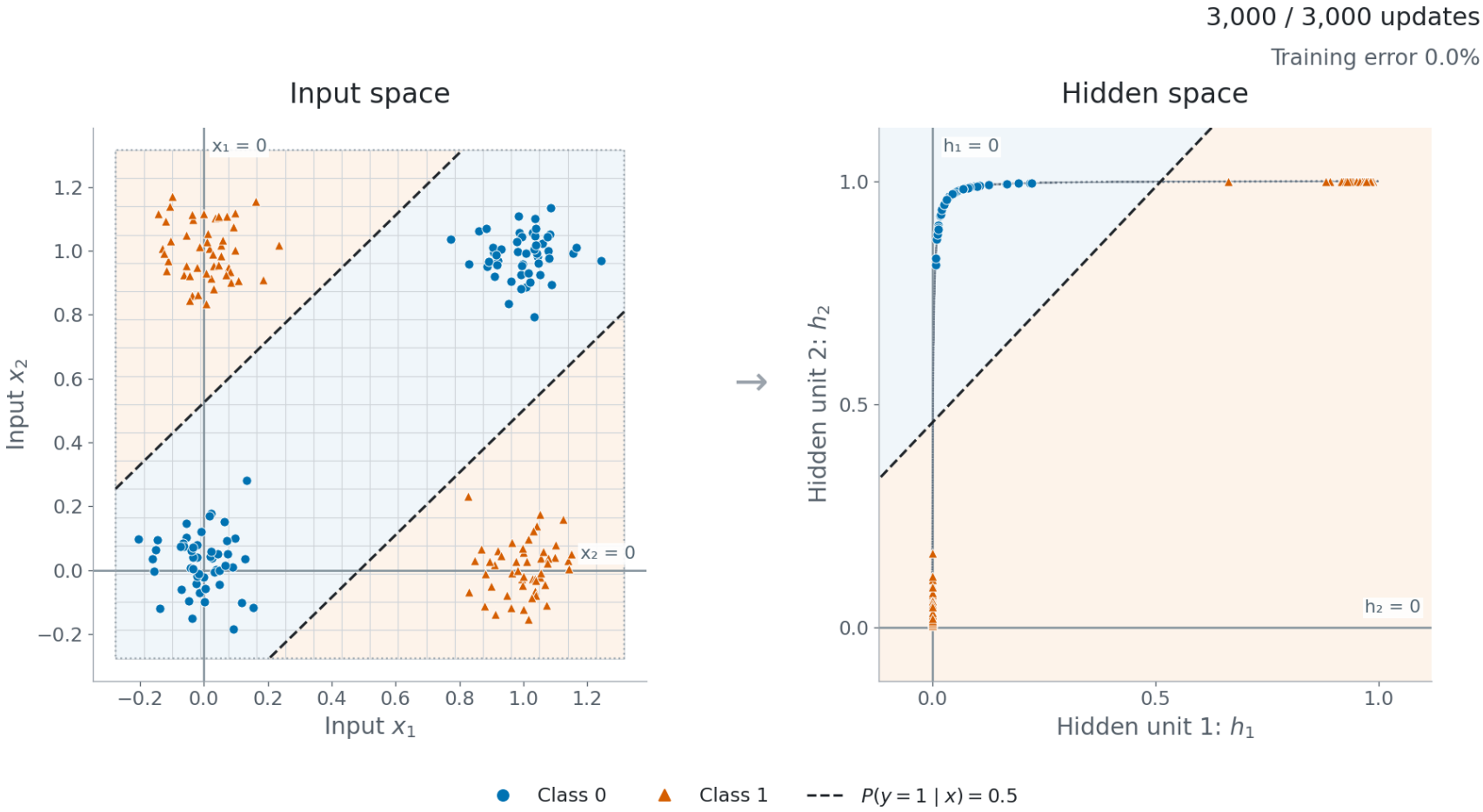
How a trained network deforms space

Fixed weights transform the grid through an affine map and sigmoid.



Learning the deformation and separating line

The hidden grid and decision boundary evolve over 3,000 updates.



Neural networks: expressiveness

Decision trees

Bagging and random forests

Kernels and SVMs

Neural networks

Expressiveness and representations

Backpropagation

Selected publication milestones

1998 Convolutional networks

2012 AlexNet

2017 Transformer

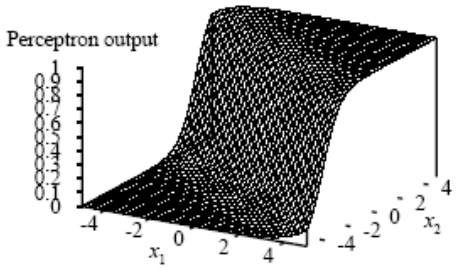
Later lectures

Reading: Expressiveness and representations

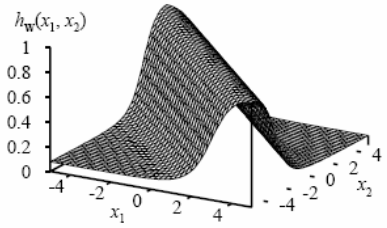
K. Murphy (2022) §§13.2.4–13.2.5 Example models and the importance of depth

Expressiveness of perceptrons

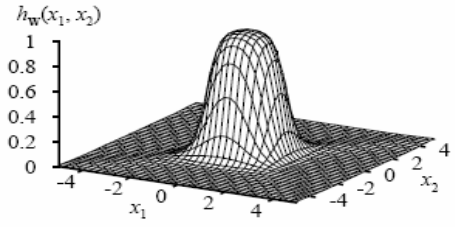
Single sigmoid neuron: ‘soft threshold’ function



Two opposite `soft threshold’ functions: a ridge



Two ridges: a bump



Universal function approximation

For every continuous f on compact K and every ε > 0,
one sufficiently wide hidden layer can achieve:

$$\widehat{f}(\mathbf{x}) = b_0 + \sum_{j=1}^{m} \alpha_j \, \sigma(\mathbf{w}_j^T \mathbf{x} + b_j)$$

$$\max_{\mathbf{x} \in K} |f(\mathbf{x}) - \widehat{f}(\mathbf{x})| < \epsilon$$

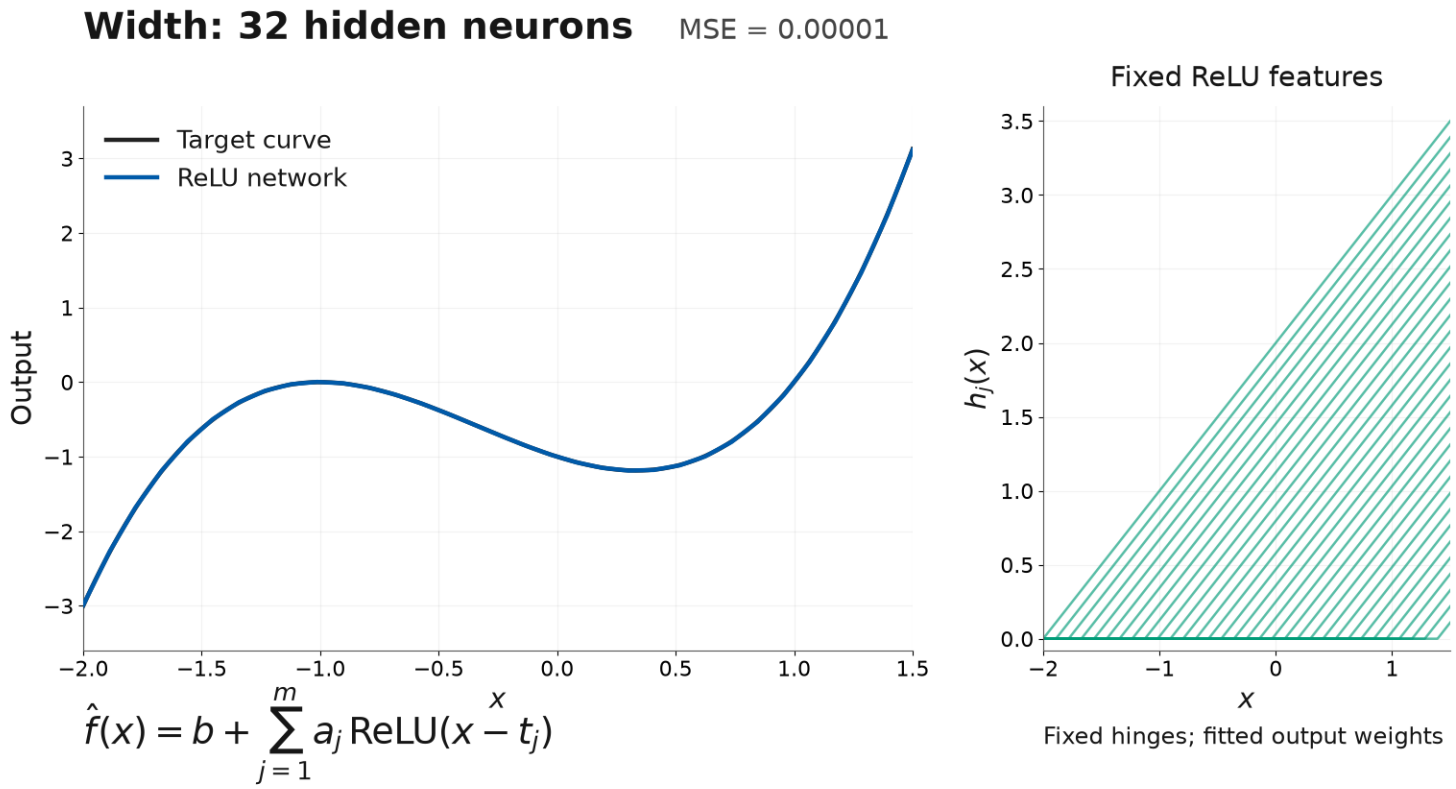
Use biases, a linear output and a continuous non-polynomial activation, such as sigmoid or ReLU.

m is the number of hidden neurons: the width.

This does not guarantee successful learning or generalization!

G. Cybenko (1989); M. Leshno, V. Lin, A. Pinkus and S. Schocken (1993)

Width builds a learned feature space



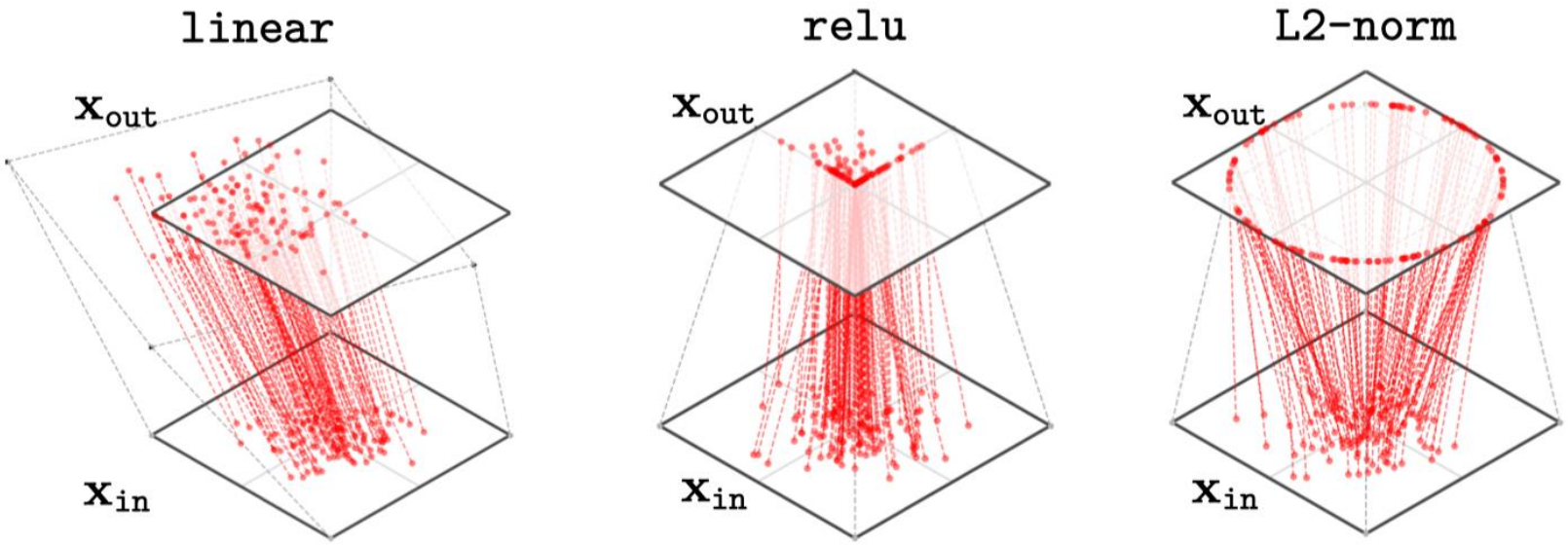
$$\phi(x) = (h_1(x), \ldots, h_m(x))^T \in \mathbb{R}^m$$

For some functions, depth uses far fewer units than width alone.

ReLU construction: B. Fortuner (2017), Can neural networks solve any problem?

Depth: M. Telgarsky (2016), Benefits of depth in neural networks.

Layers transform a cloud of data



Both input and output are 2D, so we can visualise each as a plane.

Lower plane: input. Upper plane: transformed output.

$$\mathbf{x}_{out} = \mathbf{W}\,\mathbf{x}_{in} + \mathbf{b}$$

Stretch and shift.
W: 2 × 2; b: 2 entries.

$$\mathbf{x}_{out,j} = \max(0, \mathbf{x}_{in,j})$$

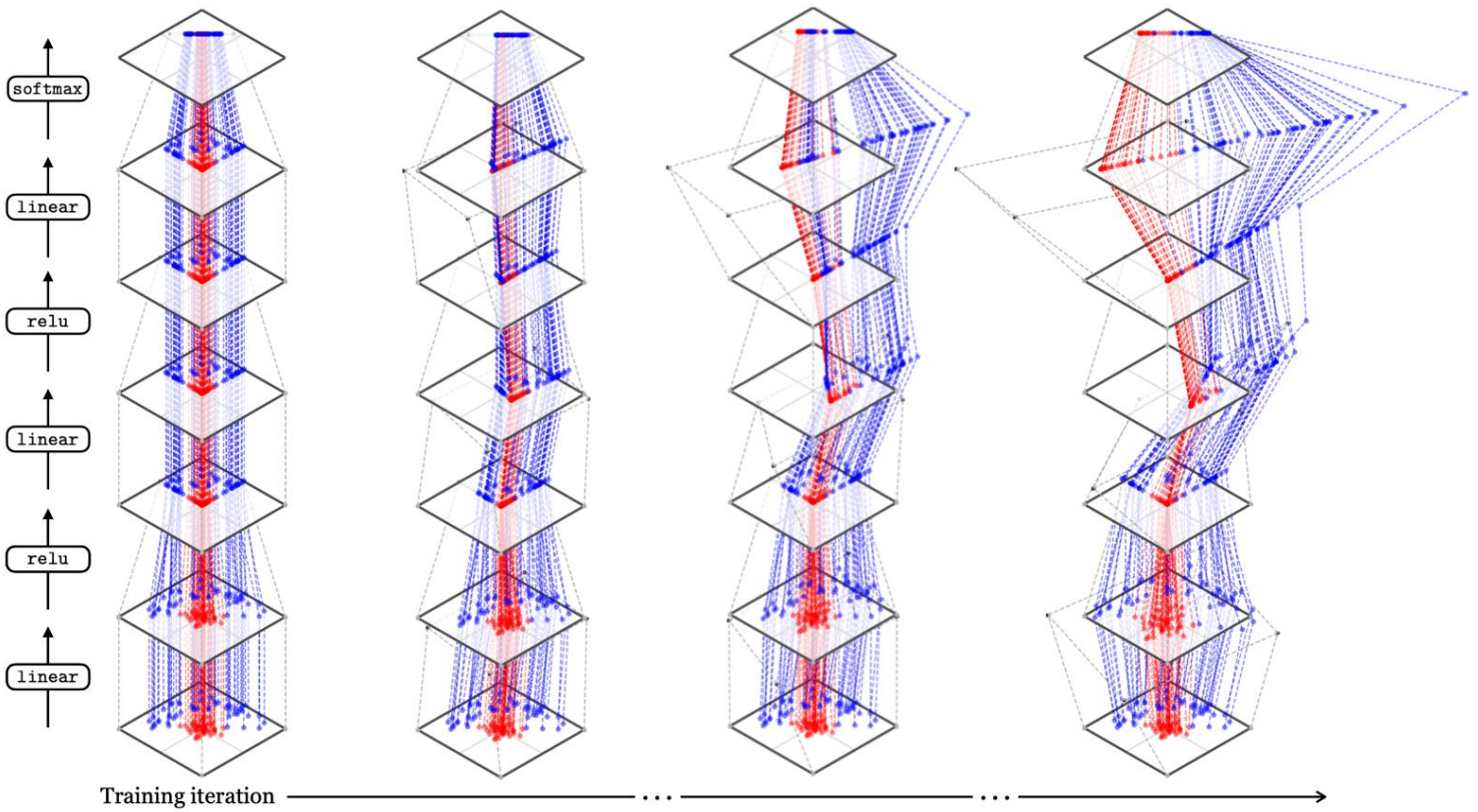
Set negatives to zero.
j = 1, 2 (each coordinate)

$$\mathbf{x}_{out} = \mathbf{x}_{in} \,/\, \|\mathbf{x}_{in}\|_2$$

$$\|\mathbf{x}\|_2 = \sqrt{(\mathbf{x}_1^2 + \mathbf{x}_2^2)}$$

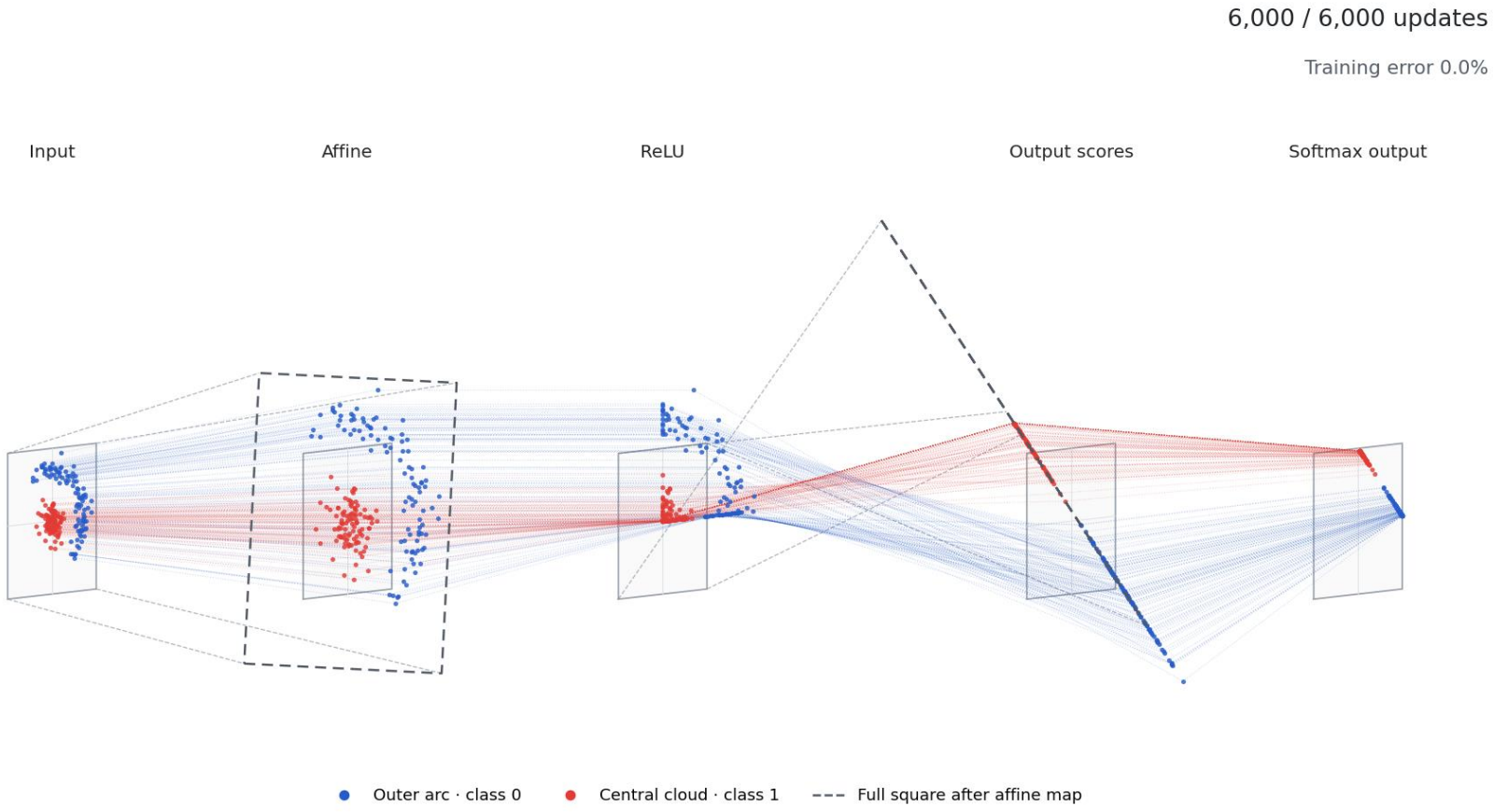
Nonzero input → unit circle

Learning changes the whole transformation



Upwards: layers. Across: training checkpoints. Colours: two classes. Training pulls each output towards its own class target.

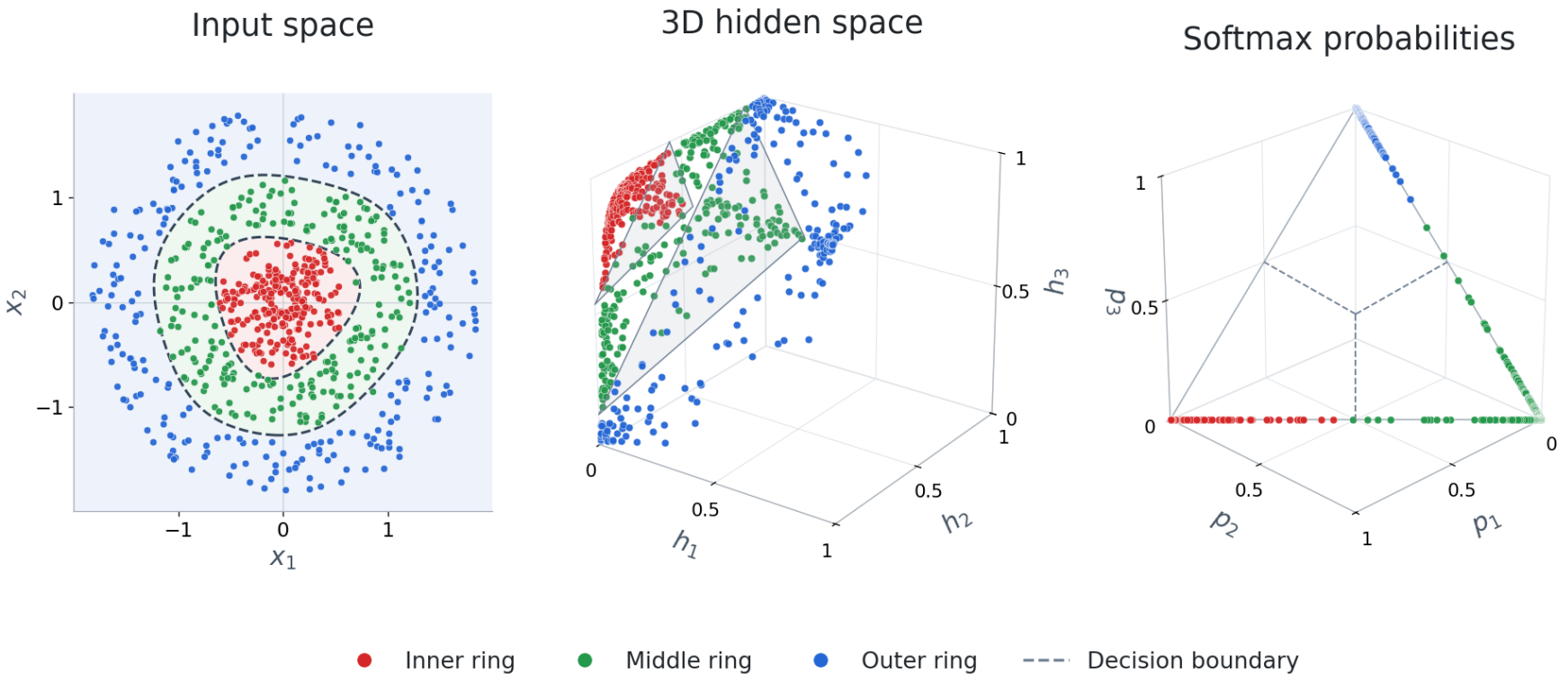
Learning a nonlinear classifier



Learning a three-dimensional representation

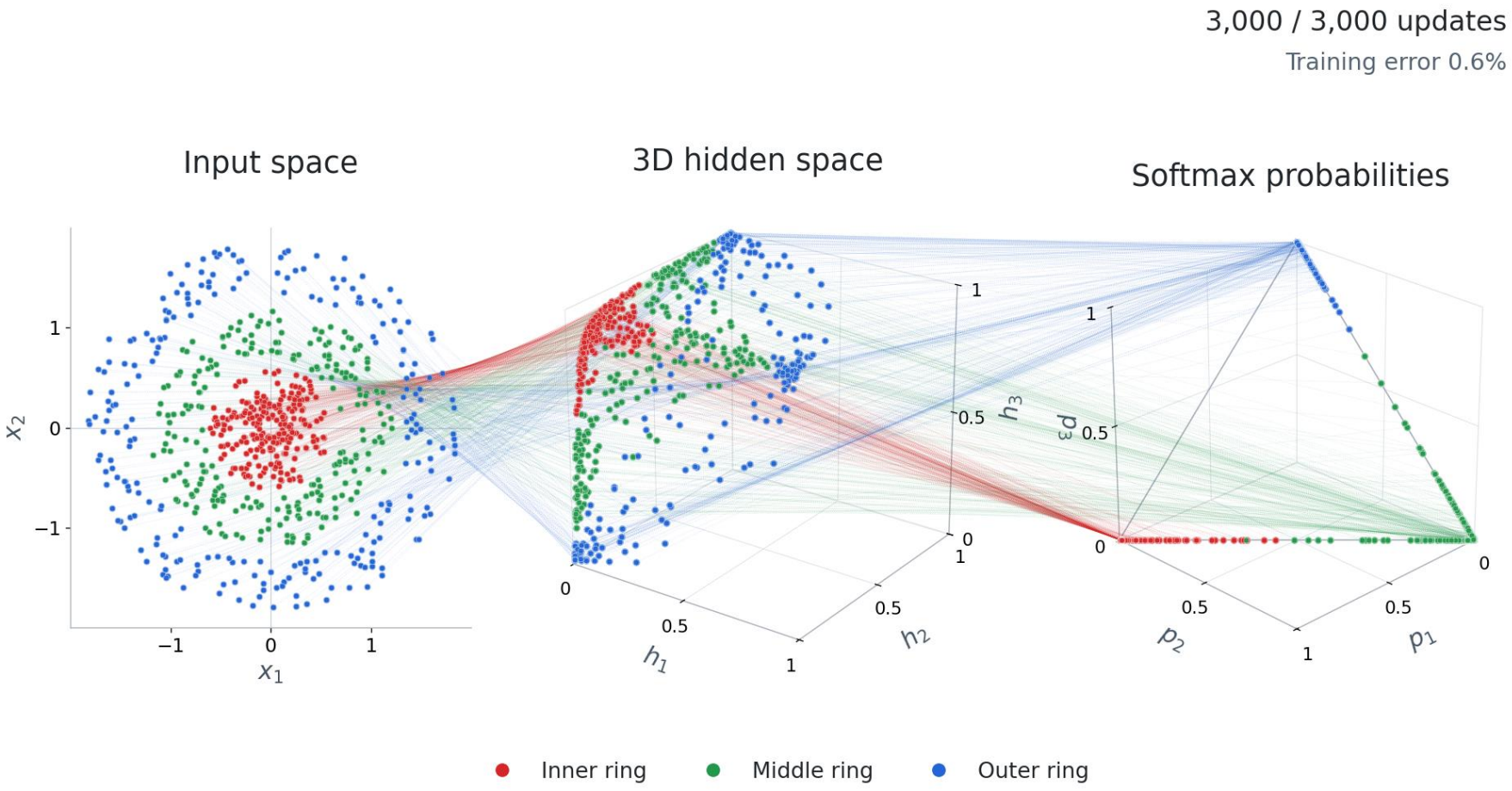
3,000 / 3,000 updates

Training error 0.6%



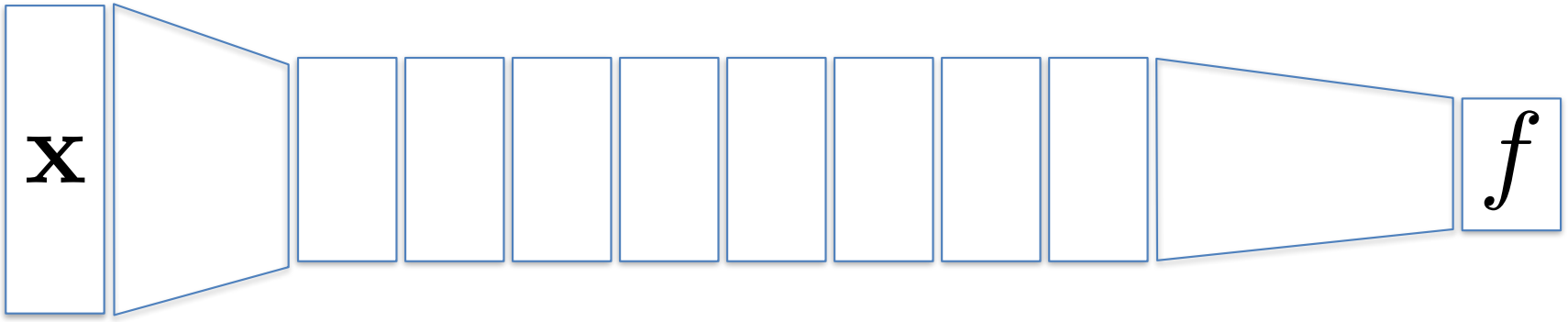
Three hidden activations give 3D features with linear class boundaries.

Following points across representations



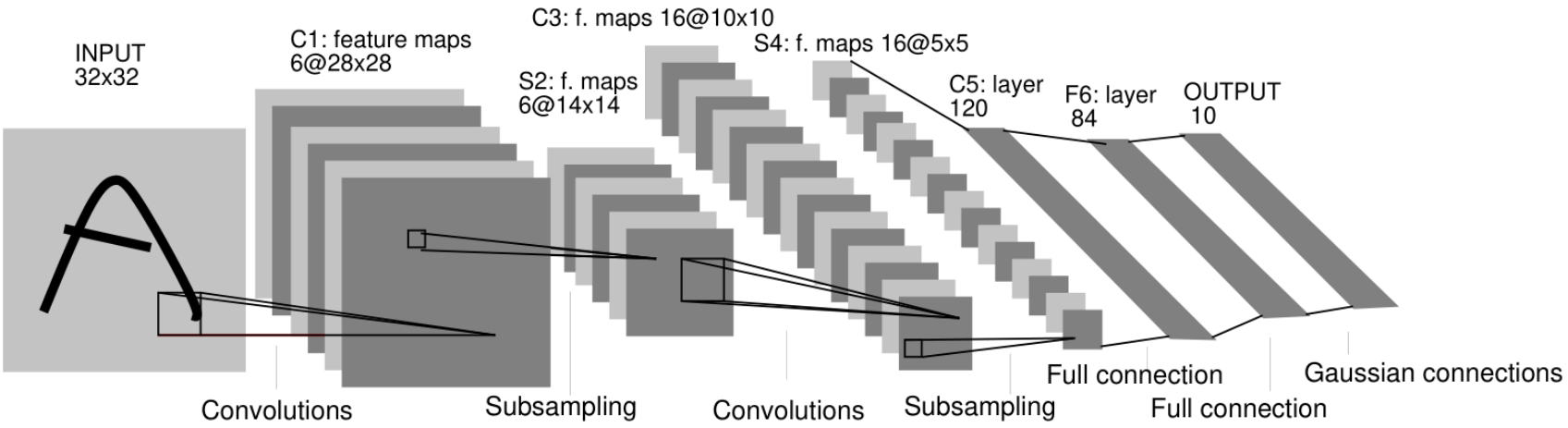
Links track each point. Softmax probabilities sum to one.

Deep networks



$$f = W_i \max(0, W_{i-1} \max(0, \cdots W_0 \mathbf{x})$$

Deep networks: LeNet-5

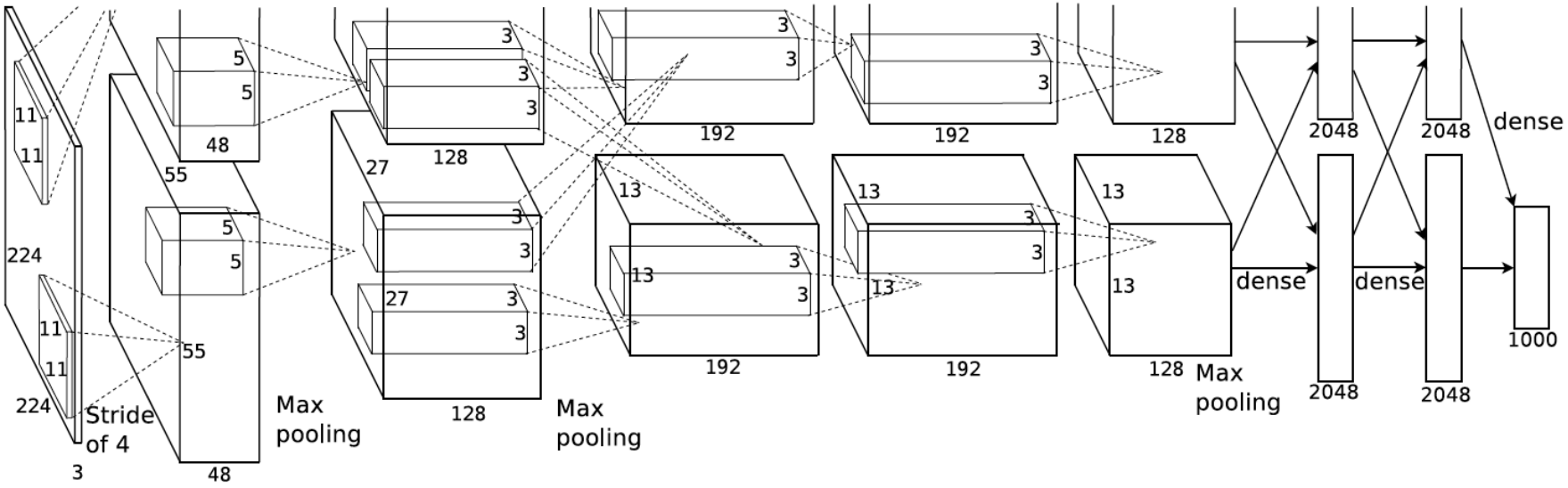


LeNet-5 recognises handwritten digits.

Convolutional networks in more detail: Week 4.

Y. LeCun, L. Bottou, Y. Bengio and P. Haffner (1998), Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE 86(11), 2278–2324. Fig. 2.

Deep networks: AlexNet

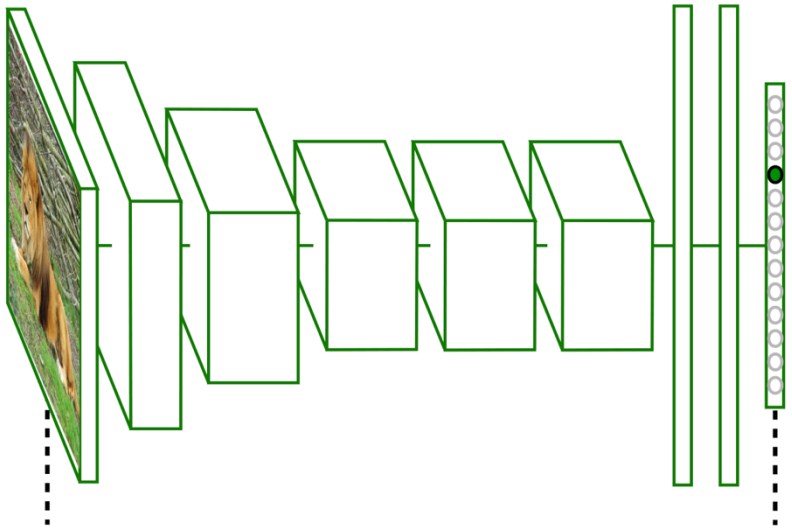


Convolutional networks in more detail: Week 4.

A. Krizhevsky, I. Sutskever and G. E. Hinton (2012), ImageNet Classification with
Deep Convolutional Neural Networks, Advances in Neural Information Processing Systems 25. Fig. 2.

2012: AlexNet

AlexNet used a deep convolutional network for ImageNet classification.



ImageNet top-5 error

AlexNet: 16.4%
Best shallow competitor: 26%

Next week: convolutional networks and ImageNet

Krizhevsky, Sutskever and Hinton (2012). Comparison reused from Lecture 4.

Meeting hours and course feedback

Monday, 6–7 pm

Office 450

Questions, clarifications and feedback



forms.cloud.microsoft/e/De8UK4392r

Backpropagation

Decision trees

Bagging and random forests

Kernels and SVMs

Neural networks

Expressiveness and representations

Backpropagation

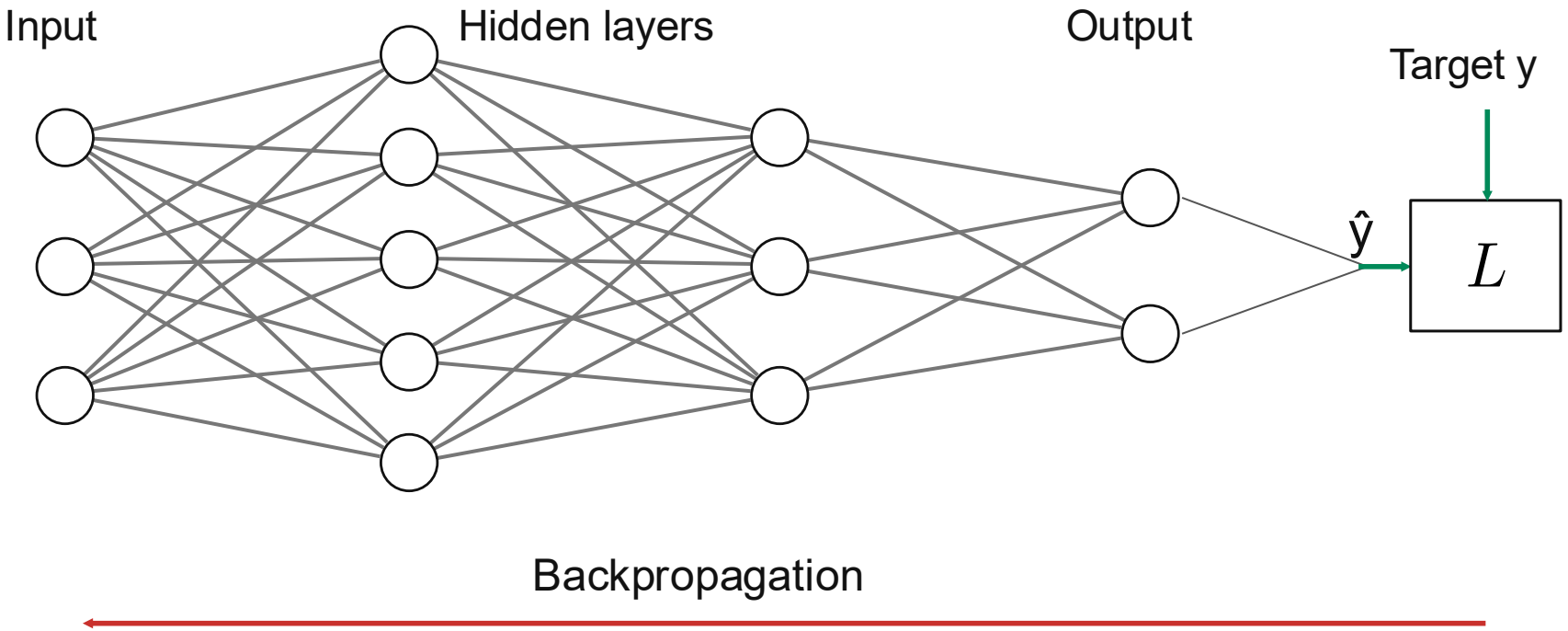
$$x \xrightarrow{g} u \xrightarrow{J} y$$

Reading: Backpropagation

K. Murphy (2022), §§13.3.2–13.3.4: backpropagation and local backward rules

Learning the network's parameters

θ_l collects all weights and biases in layer l. M is the final layer.



Derivatives for every weight and bias

$$\frac{\partial L}{\partial \theta_1}, \quad \cdots, \quad \frac{\partial L}{\partial \theta_M}$$

Parameters and activations

Parameters are the weights and biases shared across examples.

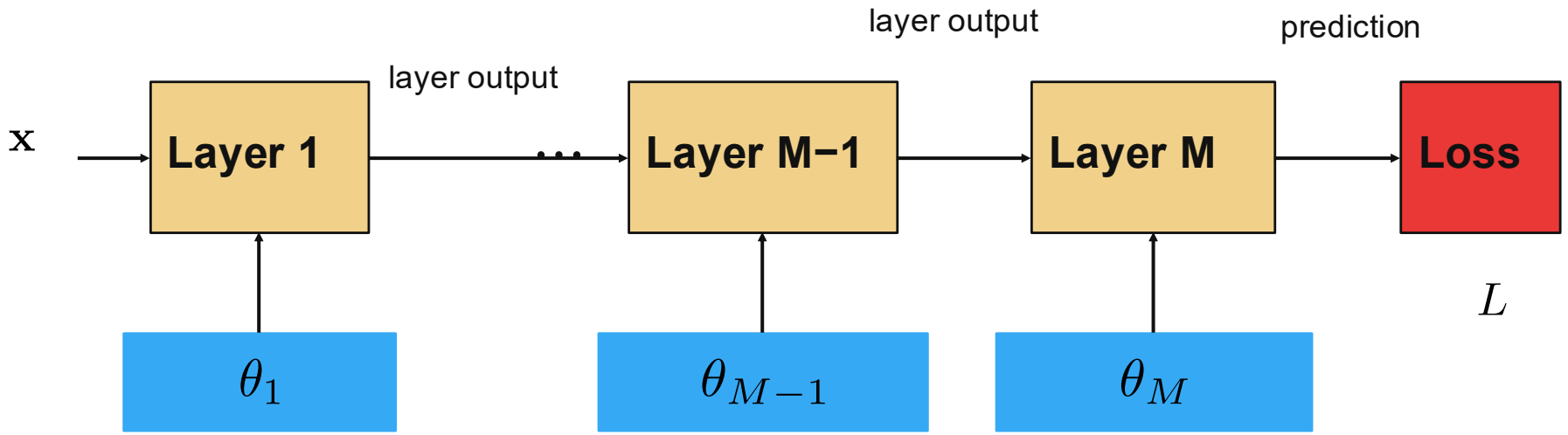
Activations are the intermediate values computed for one example.

Learning changes the parameters using a training dataset.

Inference holds those parameters fixed and computes new activations.

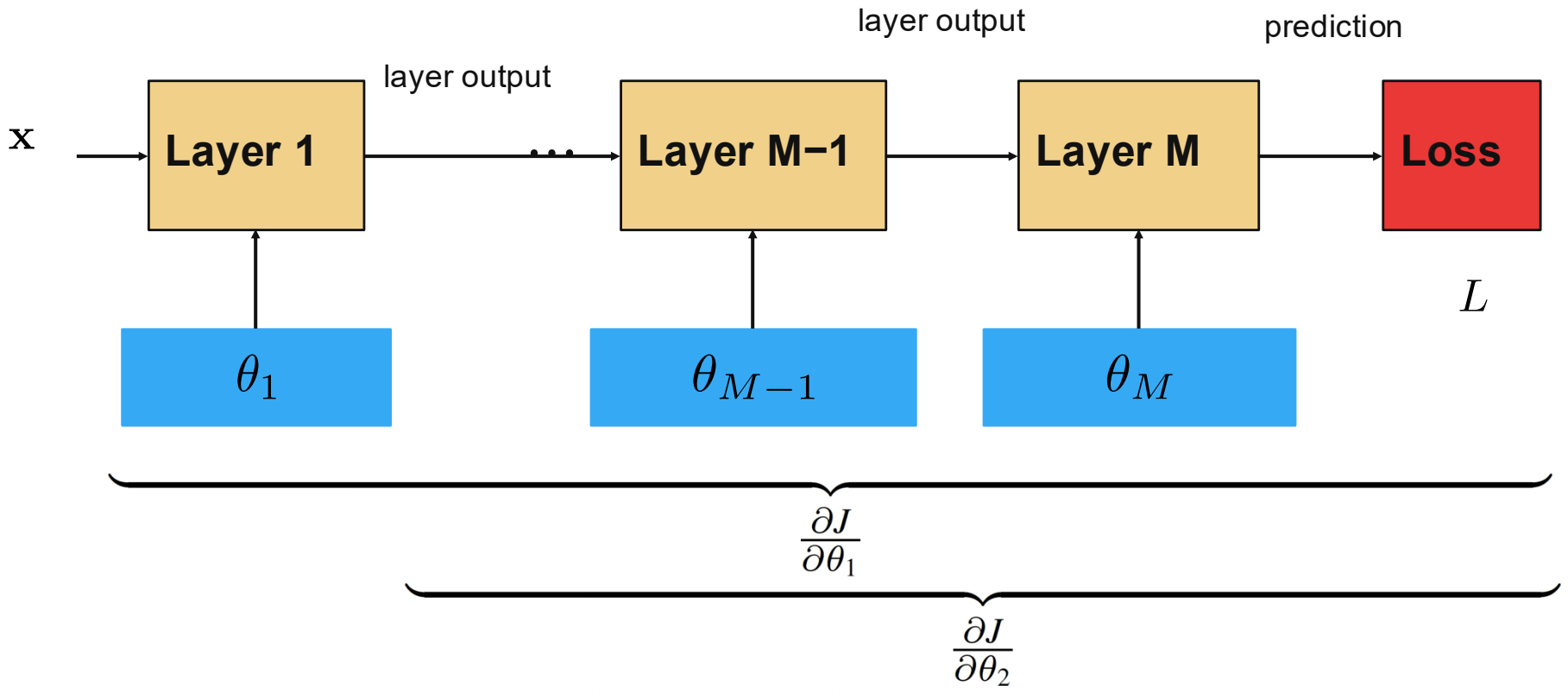
Shared computation in a multilayer network

θ_l collects all weights and biases in layer l. M is the final layer.



Shared computation in a multilayer network

θ_l collects all weights and biases in layer l. M is the final layer.



Gradients for earlier layers pass through the same later layers.

Calculating every parameter derivative separately would repeat this work.

Backpropagation reuses derivatives

x_l is layer l's scalar output; θ_l is its parameter. The loss is $L = g(x_3)$.

$$\frac{\partial L}{\partial \theta_3} = \frac{dL}{dx_3}\frac{\partial x_3}{\partial \theta_3}$$

$$\frac{\partial L}{\partial \theta_2} = \frac{dL}{dx_3}\frac{\partial x_3}{\partial x_2}\frac{\partial x_2}{\partial \theta_2}$$

$$\frac{\partial L}{\partial \theta_1} = \frac{dL}{dx_3}\frac{\partial x_3}{\partial x_2}\frac{\partial x_2}{\partial x_1}\frac{\partial x_1}{\partial \theta_1}$$

Compute each downstream derivative once and reuse it.

Partial derivatives and gradients

A partial derivative changes one input while holding the others fixed.

$$\begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \xrightarrow{\quad g \quad} L = u_1 u_2$$

$$\frac{\partial L}{\partial u_1} = u_2, \qquad \frac{\partial L}{\partial u_2} = u_1$$

The gradient collects these derivatives into a column.

$$\mathbf{g_u} := \nabla_{\mathbf{u}} L = \frac{\partial L}{\partial \mathbf{u}} = \begin{bmatrix} \partial L / \partial u_1 \\ \partial L / \partial u_2 \end{bmatrix} = \begin{bmatrix} u_2 \\ u_1 \end{bmatrix}$$

“Gradient of loss L w.r.t. u”

Bold letters denote vectors. T denotes transpose.

The scalar chain rule

A scalar loss depends on a scalar input through an intermediate value.

$$x \xrightarrow{\quad f \quad} r \xrightarrow{\quad g \quad} L$$

$$\frac{dL}{dx} = \frac{dL}{dr}\frac{dr}{dx}$$

Multiply the local derivatives along the chain.

Example

$$r = 2x + 1, \quad L = r^2$$

$$\frac{dL}{dx} = 2r \cdot 2 = 4(2x + 1)$$

The chain rule adds contributions

If one input affects several intermediate values, include every path.

$$x \xrightarrow{\mathbf{f}} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \xrightarrow{g} L$$

$$u_1 = x, \qquad u_2 = x^2, \qquad g(u_1, u_2) = u_1 u_2$$

$$\frac{dL}{dx} = \frac{\partial L}{\partial u_1}\frac{du_1}{dx} + \frac{\partial L}{\partial u_2}\frac{du_2}{dx}$$

$$= u_2 \cdot 1 + u_1 \cdot 2x = 3x^2$$

The same sum is a dot product:

$$\frac{dL}{dx} = \mathbf{g}_{\mathbf{u}}^T \frac{d\mathbf{u}}{dx} = \begin{bmatrix} u_2 & u_1 \end{bmatrix} \begin{bmatrix} 1 \\ 2x \end{bmatrix}$$

Logistic regression: forward computation

Fixed input x and binary target y. The weights w determine the loss.

$$\mathbf{w} \xrightarrow{f} z \xrightarrow{\sigma} p \xrightarrow{g} L$$

Score $$z = f(\mathbf{w}) = \mathbf{w}^T \mathbf{x}$$

Probability $$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss $$L = g(p) = -y \log p - (1 - y) \log(1 - p)$$

x and w are d-dimensional columns. The target y is 0 or 1.

Logistic regression: local derivatives

Differentiate each step with respect to its own input.

$$\mathbf{w} \xrightarrow{f} z \xrightarrow{\sigma} p \xrightarrow{g} L$$

Loss

$$\frac{dL}{dp} = -\frac{y}{p} + \frac{1-y}{1-p}$$

Sigmoid

$$\frac{dp}{dz} = p(1-p)$$

Score

$$\frac{\partial z}{\partial w_j} = x_j, \qquad \nabla_{\mathbf{w}} z = \mathbf{x}$$

The backward pass combines these local derivatives.

Logistic regression: the backward pass

Apply the chain rule from the loss back to the weights.

$$\mathbf{w} \xrightarrow{f} z \xrightarrow{\sigma} p \xrightarrow{g} L$$

Loss to score

$$\frac{dL}{dz} = \frac{dL}{dp}\frac{dp}{dz}$$

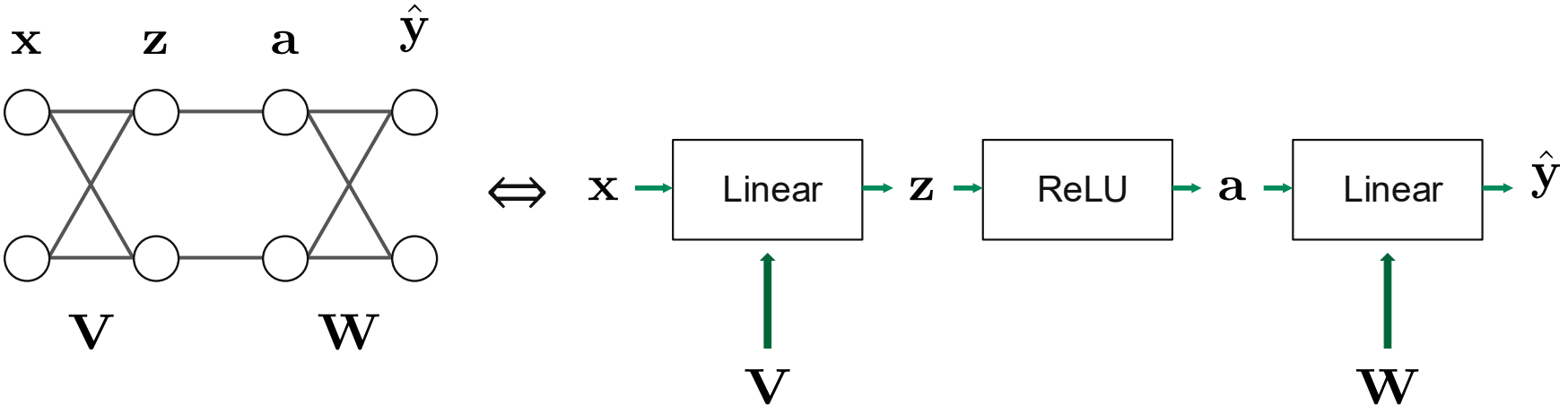
$$= \left(-\frac{y}{p} + \frac{1-y}{1-p}\right) p(1-p) = p - y$$

Score to weights

$$\nabla_{\mathbf{w}} L = \frac{dL}{dz} \nabla_{\mathbf{w}} z = (p-y)\mathbf{x}$$

Every weight reuses the same scalar factor p − y.

An MLP as a computation graph



$$\mathbf{z} = \mathbf{V}\mathbf{x}, \quad \mathbf{a} = \mathrm{ReLU}(\mathbf{z}), \quad \hat{\mathbf{y}} = \mathbf{W}\mathbf{a}$$

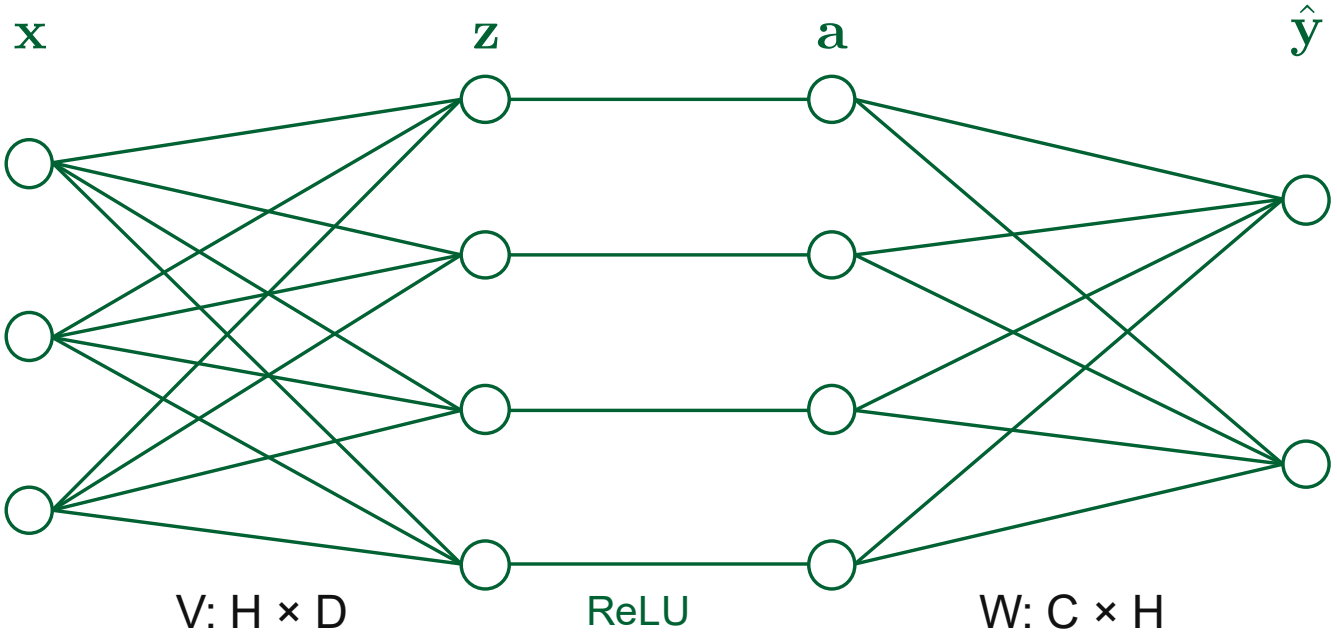
$$\mathbf{V} \in \mathbb{R}^{H \times D}, \quad \mathbf{W} \in \mathbb{R}^{C \times H}, \quad \mathbf{z}, \mathbf{a} \in \mathbb{R}^{H \times 1}$$

x: input z: hidden scores a: hidden activations
ŷ: prediction y: target L: scalar loss

D: inputs H: hidden units C: outputs or classes

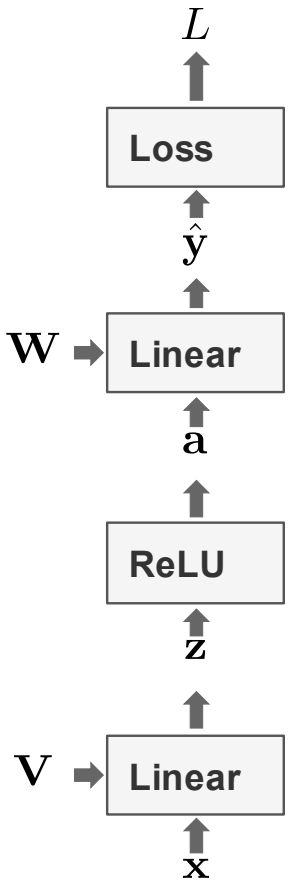
The worked regression MLP

D inputs, H hidden units, C outputs. Biases are omitted.



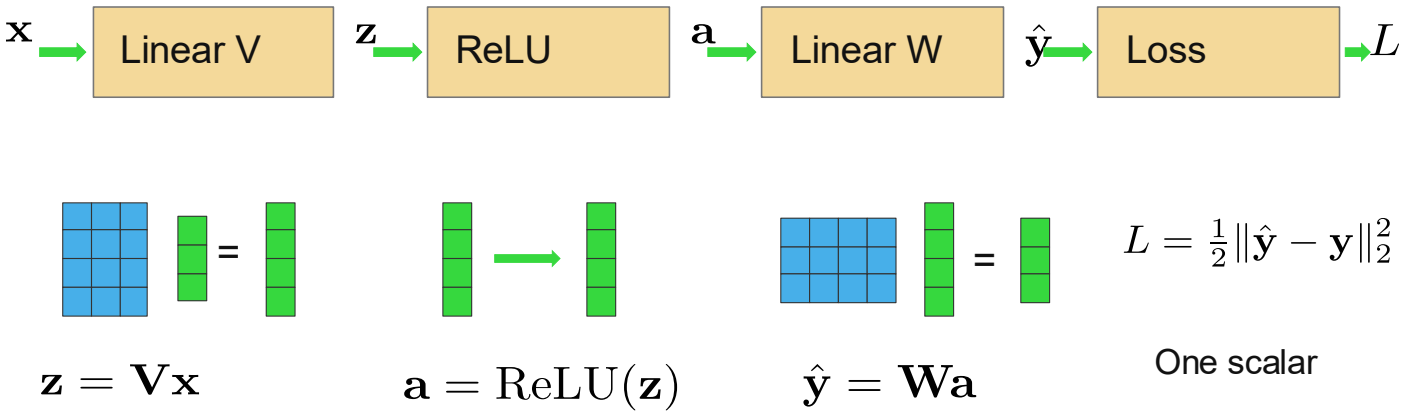
$$\mathbf{z} = \mathbf{V}\mathbf{x} \qquad \mathbf{a} = \mathrm{ReLU}(\mathbf{z}) \qquad \hat{\mathbf{y}} = \mathbf{W}\mathbf{a}$$

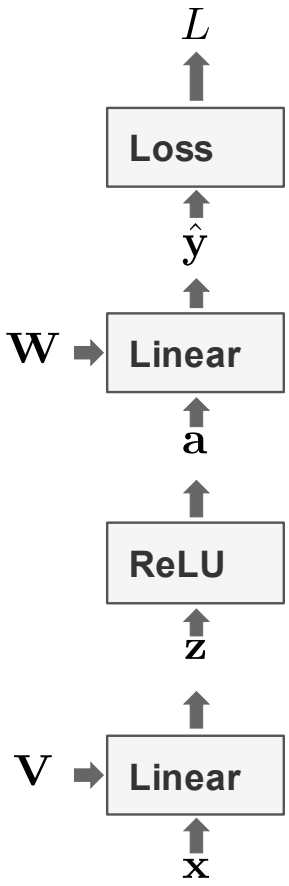
$$L = \tfrac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$$



Forward pass through a ReLU MLP

The same cached values will be reused in the backward pass.





Cache x, z and a; the prediction is ŷ = Wa.

The loss starts the backward pass

One training point with C real-valued output coordinates.

Loss

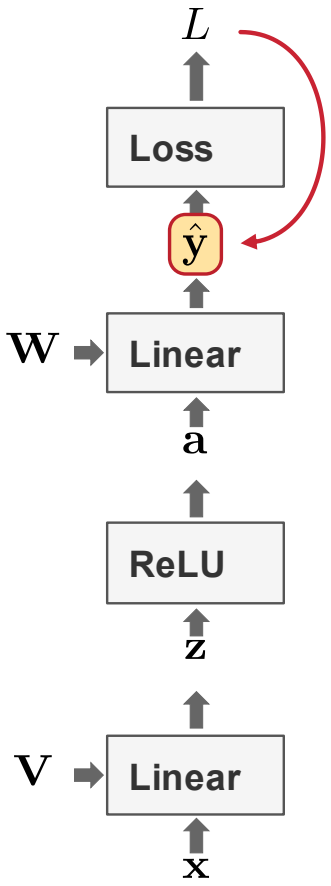
$$L = \frac{1}{2}\sum_{c=1}^{C}(\hat{y}_c - y_c)^2$$

One coordinate

$$\frac{\partial L}{\partial \hat{y}_c} = \frac{1}{2}\,2(\hat{y}_c - y_c) = \hat{y}_c - y_c$$

Column gradient

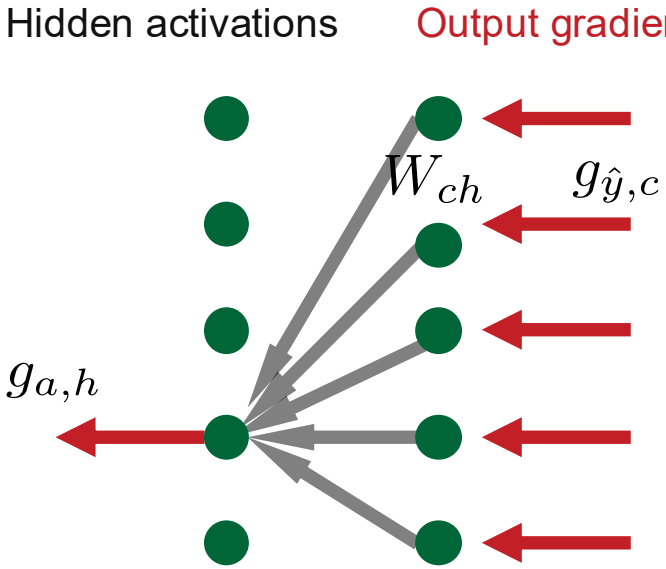
$$\underbrace{\mathbf{g}_{\hat{y}}}_{C\times 1} := \nabla_{\hat{\mathbf{y}}} L = \hat{\mathbf{y}} - \mathbf{y}$$



The residual is the incoming column gradient for the output linear map.

Backward through the output linear map

Each activation gradient sums the loss sensitivities along all output paths.

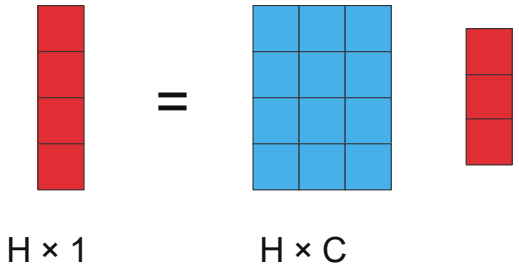


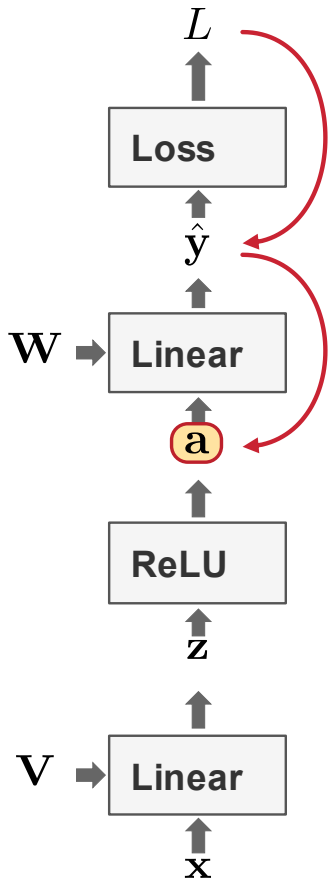
$$\hat{y}_c = \sum_{j=1}^{H} W_{cj} a_j$$

$$\frac{\partial \hat{y}_c}{\partial a_h} = W_{ch}$$

$$g_{a,h} = \sum_{c=1}^{C} W_{ch}\, g_{\hat{y},c}$$

$$\underbrace{\mathbf{g}_a}_{H\times 1} = \underbrace{\mathbf{W}^T}_{H\times C} \underbrace{\mathbf{g}_{\hat{y}}}_{C\times 1}$$

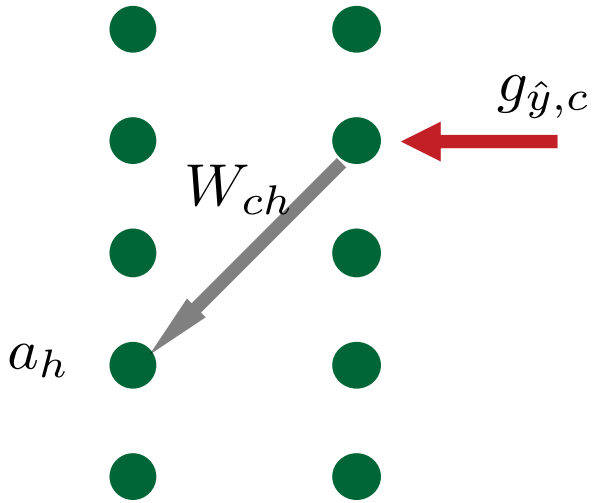




The gradient of one output weight

Each weight uses two factors: an input activation and an output gradient.

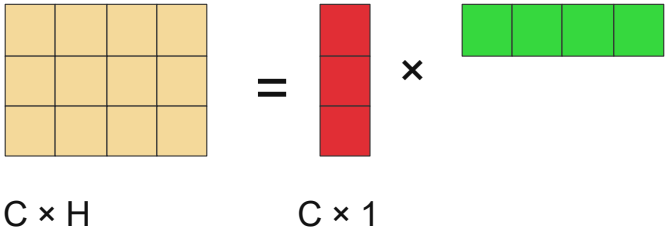
One selected connection



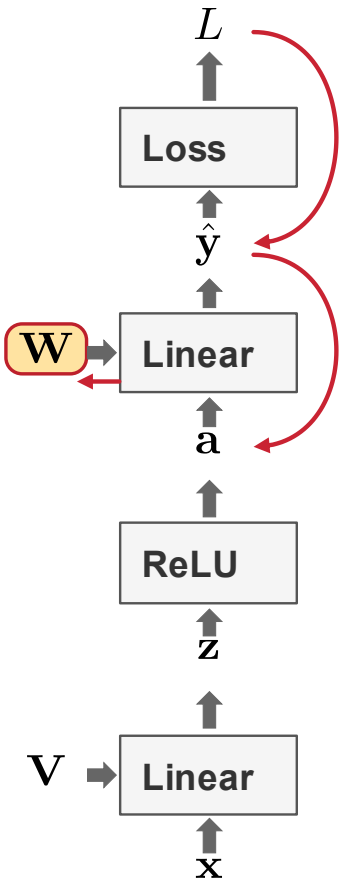
$$\hat{y}_c = \sum_{j=1}^{H} W_{cj} a_j$$

$$\frac{\partial \hat{y}_c}{\partial W_{ch}} = a_h$$

$$\frac{\partial L}{\partial W_{ch}} = g_{\hat{y},c}\, a_h$$



$$\underbrace{\nabla_{\mathbf{w}} L}_{C \times H} = \underbrace{\mathbf{g}_{\hat{y}}}_{C \times 1} \underbrace{\mathbf{a}^T}_{1 \times H}$$



The gradient of one output weight

Each output weight connects hidden activation h to output c.

Local derivative

$$\frac{\partial \hat{y}_j}{\partial W_{ch}} = \begin{cases} a_h & j = c, \\ 0 & j \neq c, \end{cases}$$

Chain rule

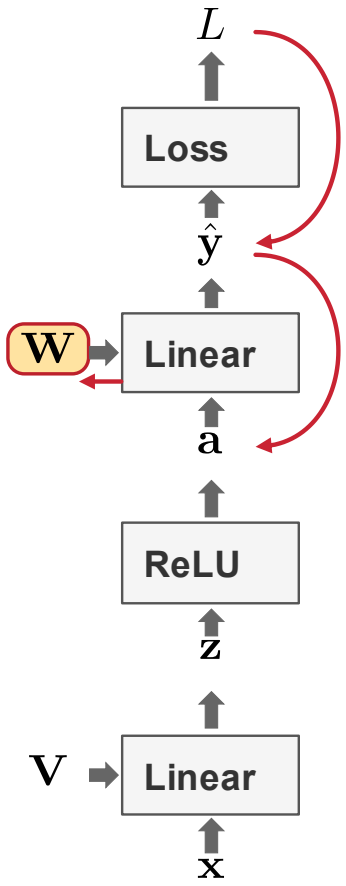
$$\frac{\partial L}{\partial W_{ch}} = \sum_{j=1}^{C} \frac{\partial L}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial W_{ch}}$$

Substitution

$$\frac{\partial L}{\partial W_{ch}} = \frac{\partial L}{\partial \hat{y}_c} a_h$$

Gradient

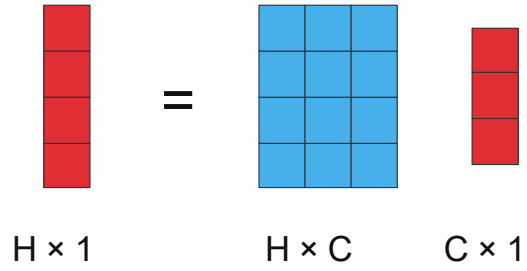
$$\frac{\partial L}{\partial W_{ch}} = g_{\hat{y},c}\, a_h$$



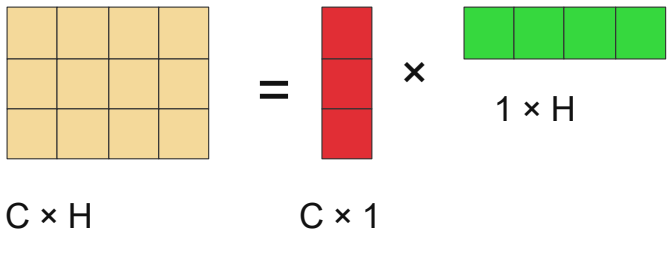
Only one output depends directly on this coefficient.

Linear layers and the gradients to their inputs

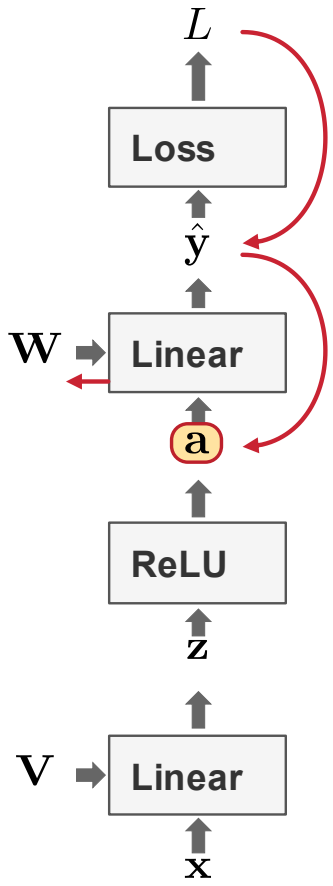
A column gradient is multiplied by the transpose of the weight matrix.



$$\underbrace{\mathbf{g}_a}_{H\times 1} = \underbrace{\mathbf{W}^T}_{H\times C}\underbrace{\mathbf{g}_{\hat{y}}}_{C\times 1}$$

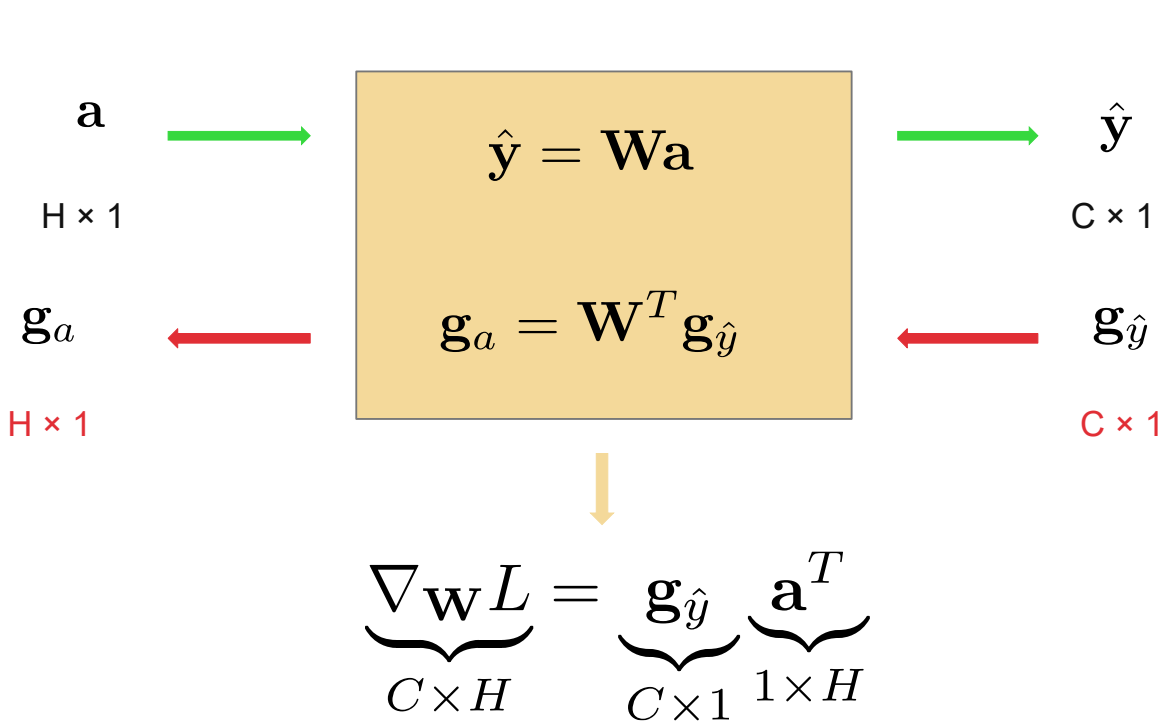


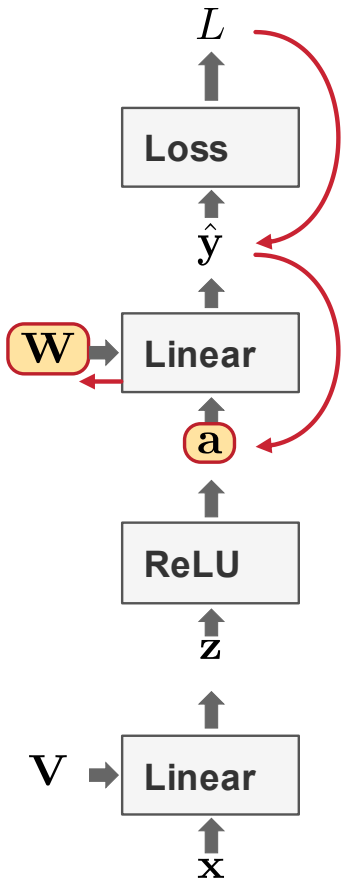
$$\underbrace{\nabla_{\mathbf{W}} L}_{C\times H} = \underbrace{\mathbf{g}_{\hat{y}}}_{C\times 1}\underbrace{\mathbf{a}^T}_{1\times H}$$



Output linear layer: the backward block

One incoming column gradient produces both required gradients.





Return activation sensitivities to earlier layers.
Store weight sensitivities for the optimiser update.

Backward through ReLU

The cached score decides whether each gradient passes through.

Gradient returned | Local derivative | Incoming gradient

$z_h > 0$

$g_{z,h} = g_{a,h}$ — × 1 — $g_{a,h}$

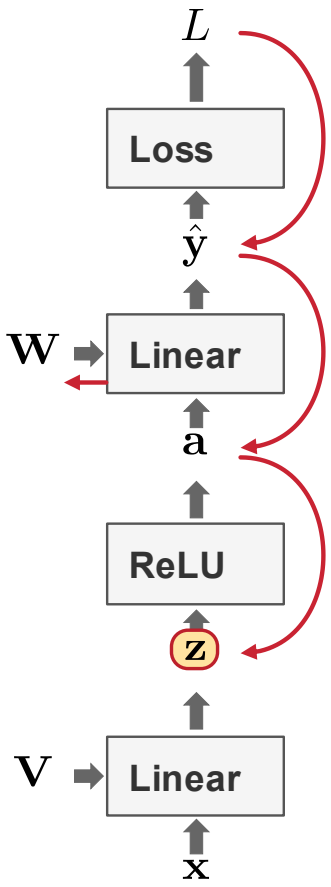
$z_h \leq 0$

$g_{z,h} = 0$ — × 0 — $g_{a,h}$

$$\mathbf{g}_z = \mathbf{g}_a \odot \mathbb{I}(\mathbf{z} > 0) \in \mathbb{R}^{H \times 1}$$

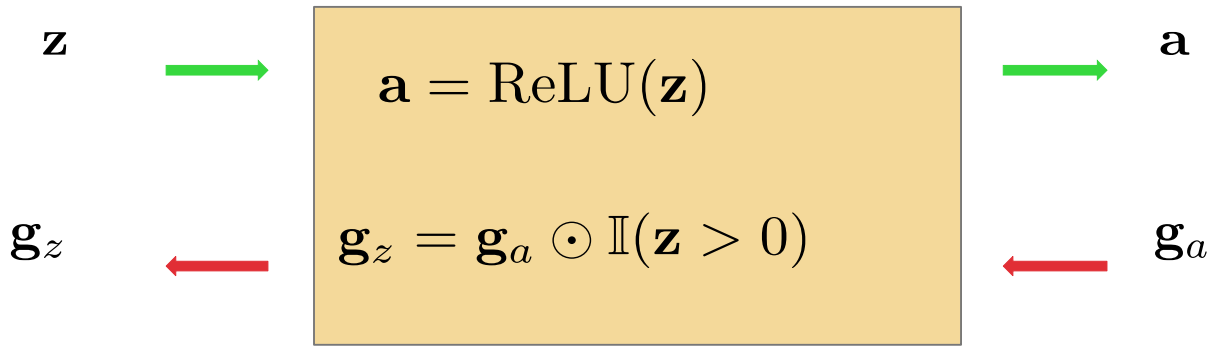
$$\mathbf{z} = (2, -1)^T,\ \mathbf{g}_a = (-3, 4)^T \implies \mathbf{g}_z = (-3, 0)^T$$

$\odot$: elementwise product. At zero, choose backward value 0.

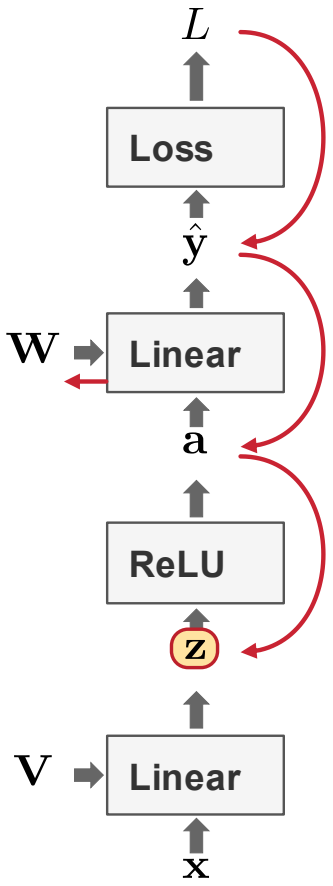


Pointwise activation derivatives

ReLU sends each incoming gradient through its cached activation gate.



$$g_{z,h} = g_{a,h}\,\mathbb{I}(z_h > 0)$$



Positive score: multiply by 1. Negative score: multiply by 0.

At the kink $z_h = 0$, choose backward value 0.

The first layer uses the same backward rule

Reuse the affine-layer rule with the hidden-score gradient and the input.

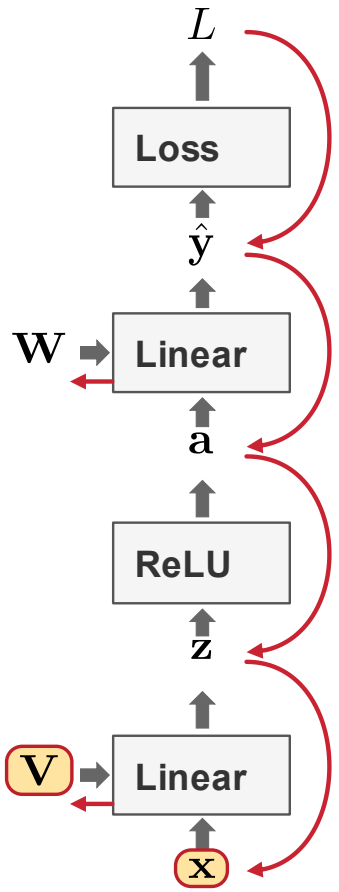
$$\mathbf{z} = \mathbf{V}\mathbf{x}, \qquad \mathbf{V} \in \mathbb{R}^{H \times D}$$

$$\frac{\partial L}{\partial V_{hi}} = g_{z,h} x_i$$

$$\underbrace{\nabla_{\mathbf{V}} L}_{H \times D} = \underbrace{\mathbf{g}_z}_{H \times 1} \underbrace{\mathbf{x}^T}_{1 \times D}$$

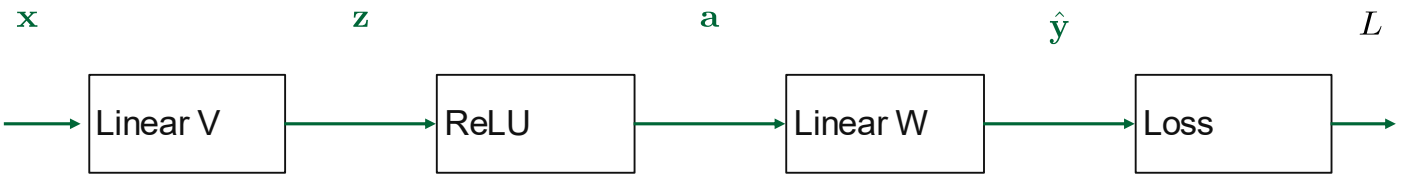
If a bias is added $\nabla_{\mathbf{b}} L = \mathbf{g}_z$

Optional input gradient $\mathbf{g}_x = \mathbf{V}^T \mathbf{g}_z$



The completed backward pass

Cache x, z and a. Compute all gradients at the same fixed parameter values.

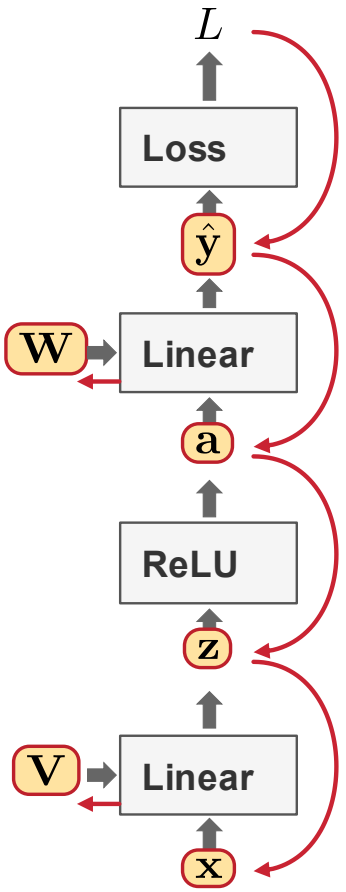


Output $\mathbf{g}_{\hat{y}} = \hat{\mathbf{y}} - \mathbf{y}$

Output linear $\nabla_{\mathbf{W}} L = \mathbf{g}_{\hat{y}} \mathbf{a}^T, \qquad \mathbf{g}_a = \mathbf{W}^T \mathbf{g}_{\hat{y}}$

ReLU $\mathbf{g}_z = \mathbf{g}_a \odot \mathbb{I}(\mathbf{z} > 0)$

First linear $\nabla_{\mathbf{V}} L = \mathbf{g}_z \mathbf{x}^T$



Backpropagation computes gradients. The optimiser then changes the parameters.

Appendix: bias and variance

Optional appendix material

Bias and variance

Forest applications

Historical context

Kernels and margins

Risk and generalisation

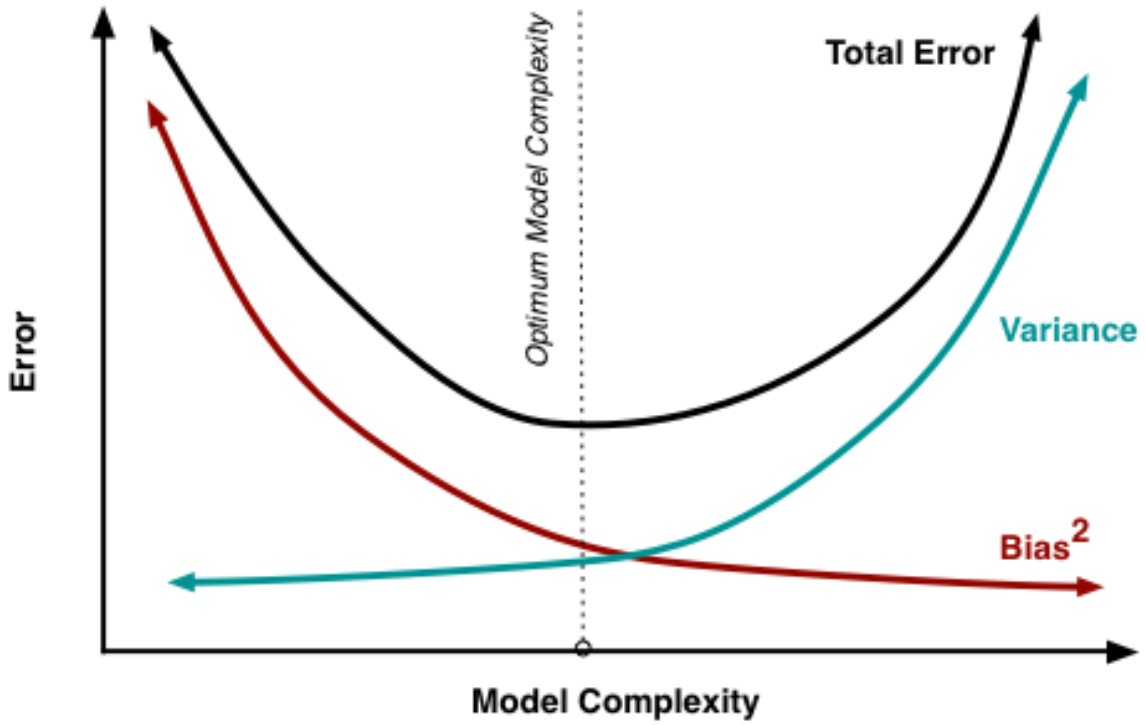
Averaging over training datasets

Bias–variance decomposition

Reading

K. Murphy (2022), §§4.3, 4.7.6 Empirical risk and the bias–variance tradeoff

Bias-variance decomposition



Schematic; noise is not shown.

Empirical risk vs population risk

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \qquad \ell(y, \hat{y}) = (y - \hat{y})^2$$

Training risk: mean squared error on the training set:

$$\hat{R}_{\text{train}}(f) = \frac{1}{N} \sum_{i=1}^{N} \ell(y_i, f(\mathbf{x}_i))$$

Empirical risk minimization:

$$\hat{f}_{\mathcal{D}} \in \arg\min_{f \in \mathcal{F}} \hat{R}_{\text{train}}(f)$$

Population risk:

$$R(f) = \mathbb{E}_{(\mathbf{X},Y)\sim P}[\ell(Y, f(\mathbf{X}))] = \iint \ell(y, f(\mathbf{x}))\, p(\mathbf{x}, y)\, \mathrm{d}y\, \mathrm{d}\mathbf{x}$$

F: candidate function class.
Subscript D: the training set used for this fit.
p(x, y): joint density for continuous inputs and targets.

Bias–variance decomposition at one input

$$Y = f^*(x) + \epsilon, \quad \mathbb{E}[\epsilon \mid x] = 0, \quad \mathbb{E}[\epsilon^2 \mid x] = \sigma^2(x)$$

m(x): mean fitted prediction over training datasets.

Add and subtract the true mean and the mean fit

$$Y - \hat{f}_{\mathcal{D}}(x) = \underbrace{f^*(x) - m(x)}_{\text{mean error}} + \underbrace{m(x) - \hat{f}_{\mathcal{D}}(x)}_{\text{fit fluctuation}} + \epsilon$$

Cross-terms vanish: the fluctuations have mean zero, noise is independent of the fitted predictor. $\mathbb{E}_{\mathcal{D}}[m(x) - \hat{f}_{\mathcal{D}}(x)] = 0, \qquad \mathbb{E}[\epsilon \mid x] = 0$

$$\mathbb{E}_{\mathcal{D},Y}[(Y - \hat{f}_{\mathcal{D}}(x))^2 \mid X = x]$$

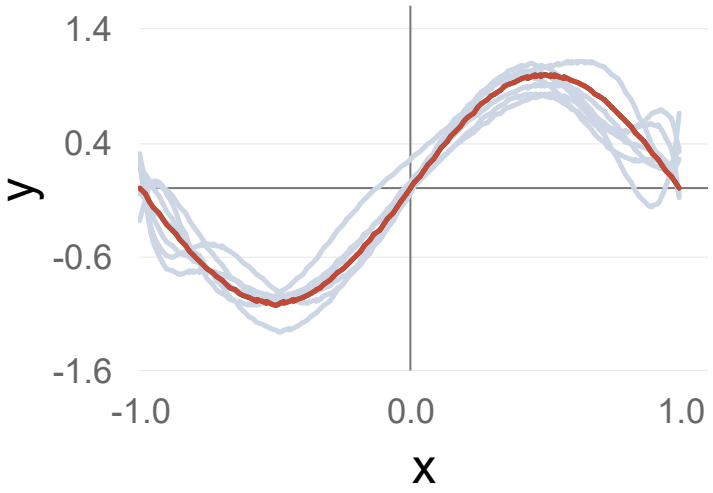
$$= \underbrace{(m(x) - f^*(x))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - m(x))^2]}_{\text{variance}} + \underbrace{\sigma^2(x)}_{\text{noise}}$$

Target noise ε is independent of the training data. σ²(x) is noise variance at x.

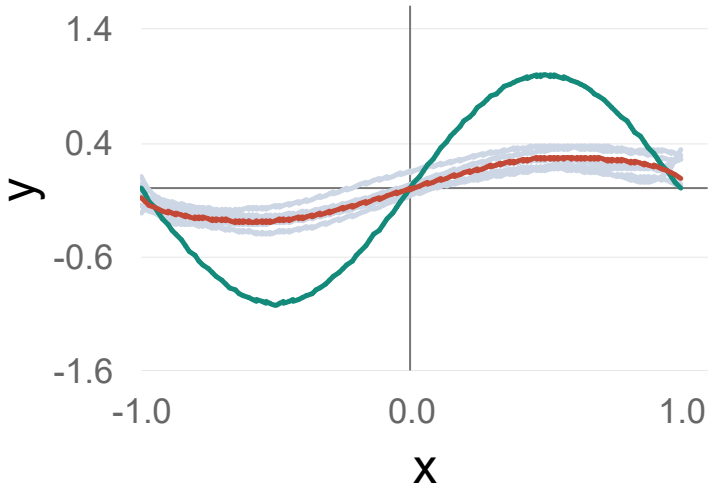
What does “averaging over datasets” mean?

Repeat collecting N examples from the same source and refitting.
Keep the function class and ridge strength λ fixed within each plot.

Weak ridge: λ = 0.0001



Strong ridge: λ = 10



At scalar input x: grey fits, red mean fit m(x), green true mean f*(x).

$$m(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)], \qquad f^*(x) = \mathbb{E}[Y \mid X = x]$$

E with subscript D averages training sets.
Here: fixed inputs, fresh noisy targets. With random inputs, resample those too.

Appendix: forest applications

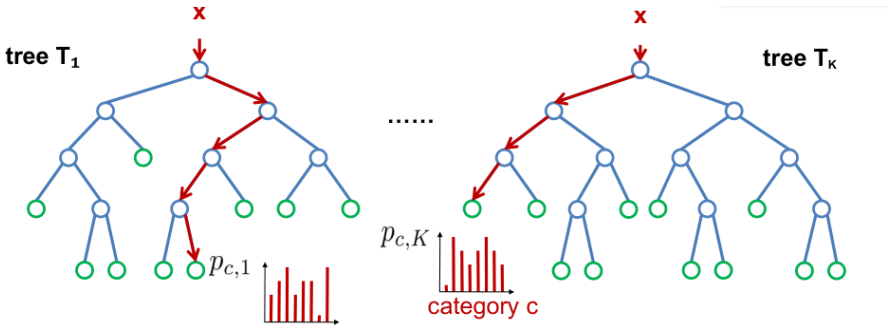
Bias and variance

Forest applications

Historical context

Kernels and margins

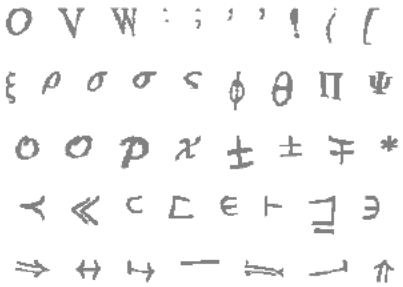
Appendix: optional material



Reading: Forest applications

K. Murphy (2022), §18.4 Random forests

Randomized Forests in Vision



[Amit & Geman, 97]
digit recognition



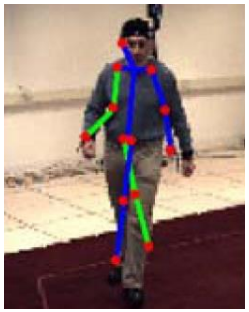
[Lepetit *et al.*, 06]
keypoint recognition



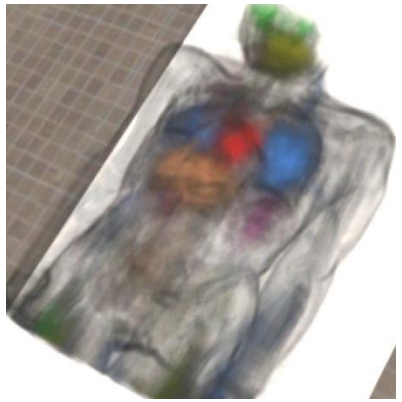
[Moosmann *et al.*, 06]
visual word clustering



[Shotton *et al.*, 08]
object segmentation



[Rogez *et al.*, 08]
pose estimation



[Criminisi *et al.*, 09]
organ detection

Selected applications

Kinect's pipeline







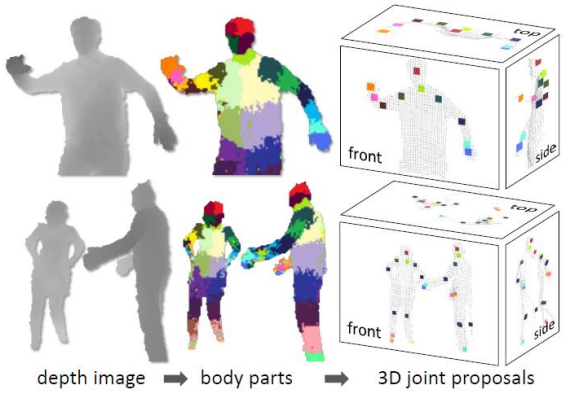


Figure 1. **Overview.** From an single input depth image, a per-pixel body part distribution is inferred. (Colors indicate the most likely part labels at each pixel, and correspond in the joint proposals). Local modes of this signal are estimated to give high-quality proposals for the 3D locations of body joints, even for multiple users.

A single depth image supplies a classification problem at each pixel.

A random forest predicts body-part labels.

The body-part distributions support 3D joint estimates.

Shotton et al., CVPR 2011: Real-Time Human Pose Recognition in Parts from a Single Depth Image

Appendix: kernels and margins

Optional appendix material

Bias and variance

Forest applications

Historical context

Kernels and margins

Large margins

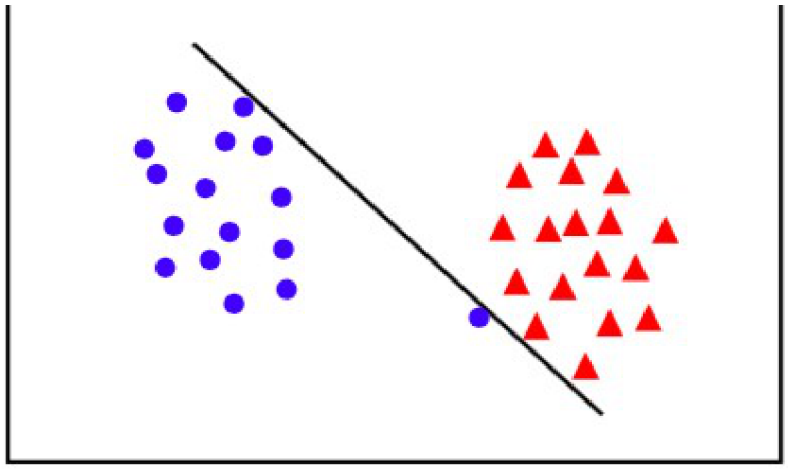
Kernel evaluations

Gaussian feature expansion

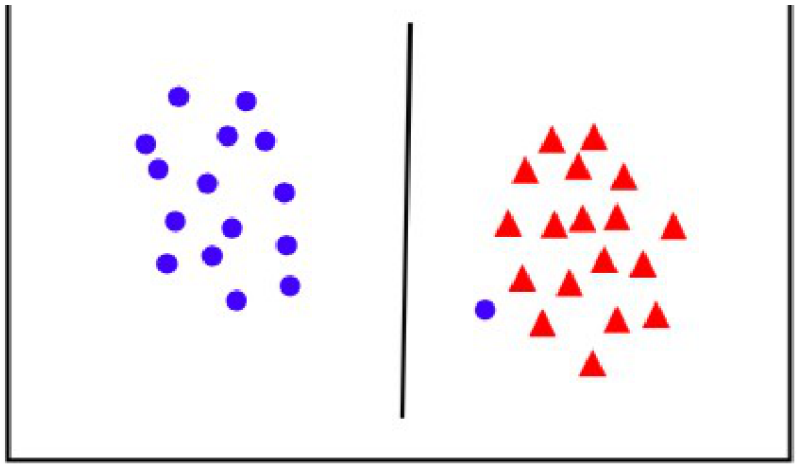
Reading

K. Murphy (2022), §§17.1, 17.3

Large margins and imperfect separation



All points on the correct side!

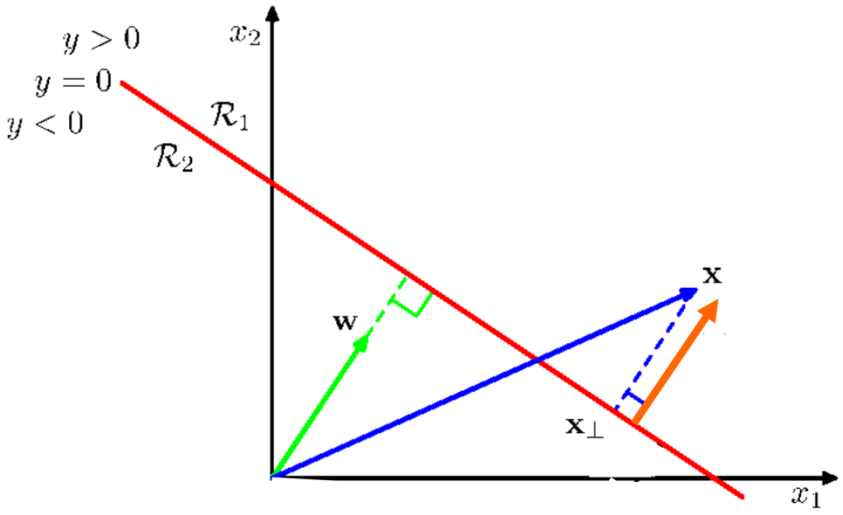


But this looks better overall!

Prefer a separator with the largest minimum geometric margin.

Allow some violations when they improve the overall tradeoff.
Choose the tradeoff using validation data.

Geometric margin



The signed distance to the decision boundary measures how clearly an example is classified.

$$\gamma_i = \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|}$$

Labels $y_i \in \{-1, +1\}$. A positive margin means correct classification.
Positive rescaling of w and b leaves the geometric margin unchanged.

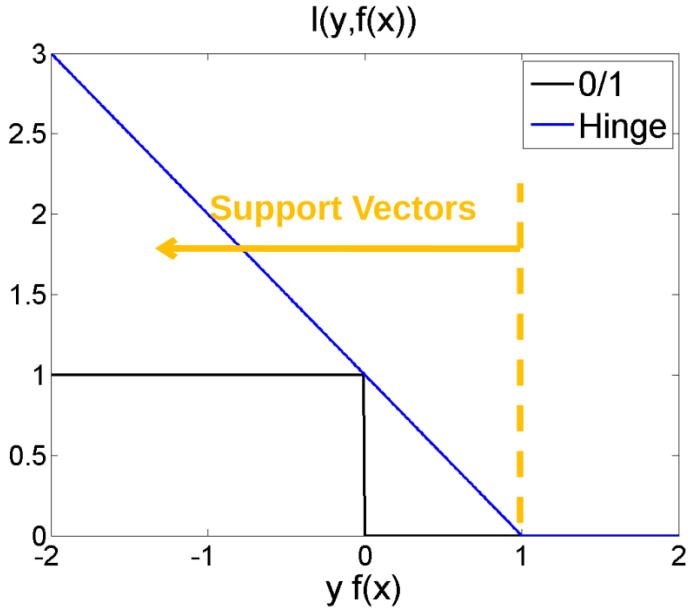
Hinge loss and regularization

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\max(0, 1 - y_i f(\mathbf{x}_i))$$

$f(x) = w^T x + b$
Labels $y = \pm 1$; margin $y\, f(x)$.

The weight penalty favours a wide margin.

Hinge loss penalises margins below 1.



Larger C puts more weight on fitting the training examples.

The kernel training problem

Binary labels $y_i = \pm 1$; α_i are learned nonnegative coefficients.

$$k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^T \phi(\mathbf{z})$$

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j)$$

$$0 \leq \alpha_i \leq C, \qquad \sum_i \alpha_i y_i = 0$$

Only inner products are needed. A kernel computes them directly, without constructing the feature vector explicitly.

Kernel examples

Linear kernel

$$k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$$

Polynomial kernel

$$k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + 1)^p$$

Radial basis function (Gaussian) kernel

$$k(\mathbf{x}, \mathbf{z}) = \exp\left[-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}\right], \qquad \sigma > 0$$

Underlying feature dimension for the Gaussian kernel: infinite

The Gaussian feature expansion

$$\gamma = \frac{1}{2\sigma^2} > 0, \qquad x, z \in \mathbb{R}$$

$$\begin{aligned} k(x,z) &= e^{-\gamma(x-z)^2} \\ &= e^{-\gamma x^2} e^{-\gamma z^2} e^{2\gamma xz} \\ &= e^{-\gamma x^2} e^{-\gamma z^2} \sum_{r=0}^{\infty} \frac{(2\gamma xz)^r}{r!} \\ &= \sum_{r=0}^{\infty} \left[e^{-\gamma x^2} \sqrt{\frac{(2\gamma)^r}{r!}} x^r \right] \left[e^{-\gamma z^2} \sqrt{\frac{(2\gamma)^r}{r!}} z^r \right] \\ &= \phi(x)^T \phi(z). \end{aligned}$$

The feature vector contains one coordinate for every non-negative integer:

$$\phi(x) = e^{-\gamma x^2} \left(1, \sqrt{2\gamma}\, x, \sqrt{\frac{(2\gamma)^2}{2!}} x^2, \sqrt{\frac{(2\gamma)^3}{3!}} x^3, \ldots \right)^T$$

References: the eight ML milestones

[1] Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408.

[2] Breiman, L., Friedman, J. H., Olshen, R. A. & Stone, C. J. (1984). Classification and Regression Trees. Wadsworth & Brooks/Cole.

[3] Rumelhart, D. E., Hinton, G. E. & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536.

[4] Cortes, C. & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297.

[5] LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.

[6] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

[7] Krizhevsky, A., Sutskever, I. & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems, 25, 1097–1105.

[8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998–6008.